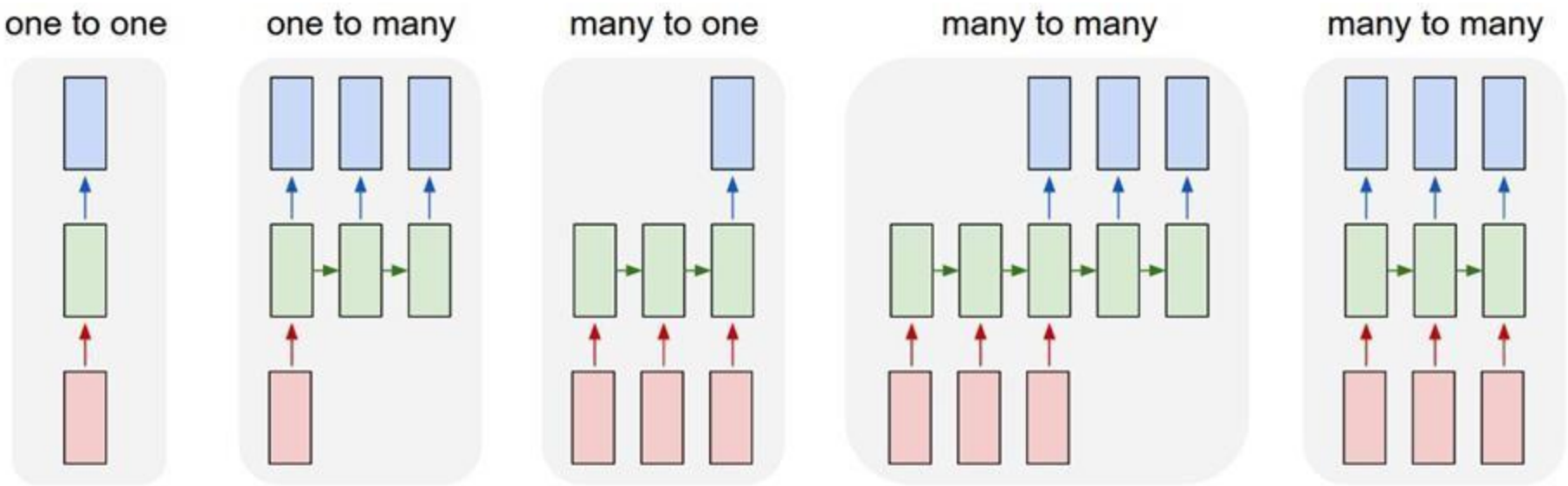
Лекция 7: Трансформеры в деталях

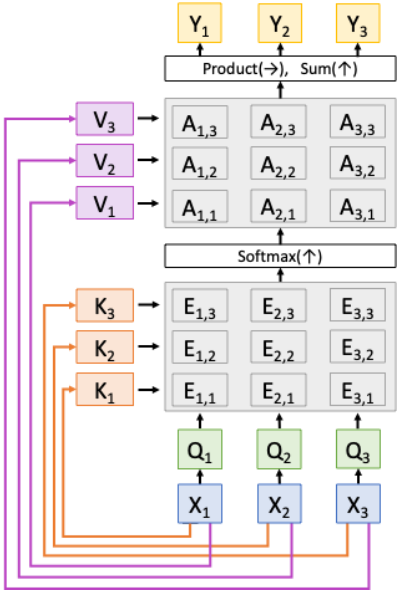
RNN -> Attention -> Transformer -> 🤗

Last Time: Recurrent Neural Networks

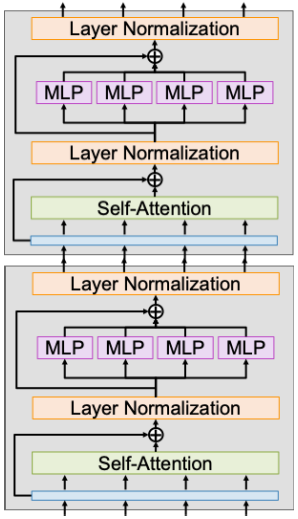


Today: Attention + Transformers

Attention: A new primitive that operates on sets of vectors

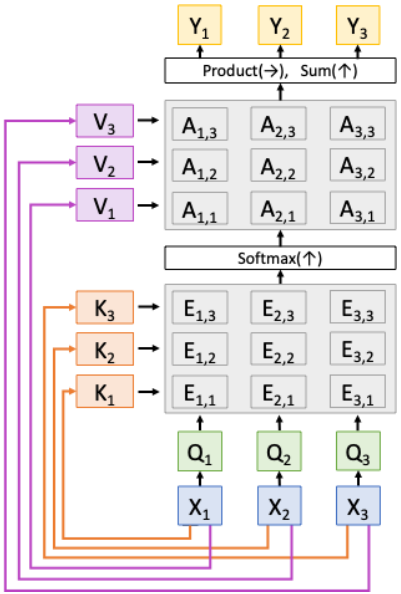


Transformer: A neural network architecture that uses attention everywhere



Today: Attention + Transformers

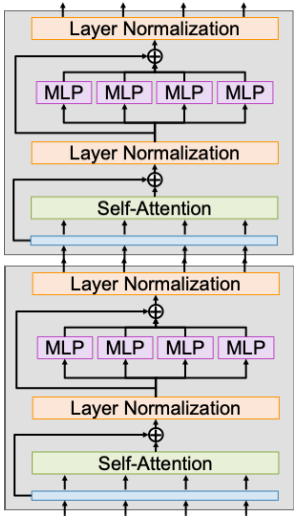
Attention: A new primitive that operates on sets of vectors



Transformers are used everywhere today!

But they developed as an offshoot of RNNs so let's start there

Transformer: A neural network architecture that uses attention everywhere

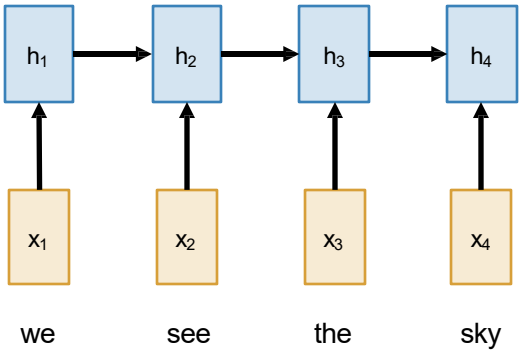


Sequence to Sequence with RNNs: Encoder - Decoder

Input: Sequence $x_1, \ldots x_T$
Output: Sequence $y_1, \ldots, y_{T'}$

A motivating example for today's discussion – machine translation! English → Italian

Encoder: $h_t = f_W(x_t, h_{t-1})$



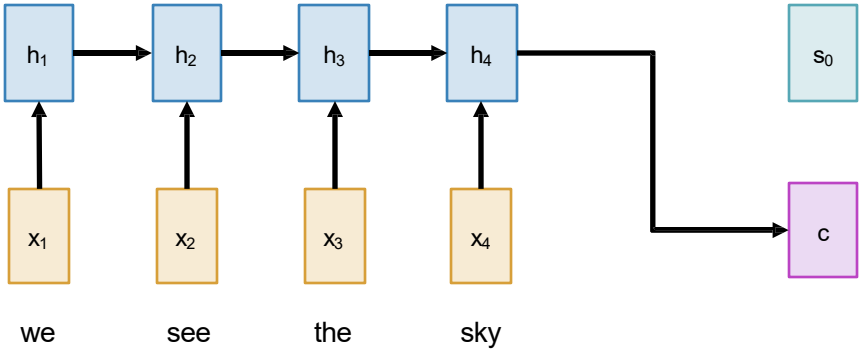
Sutskever et al, "Sequence to sequence learning with neural networks", NeurIPS 2014

Sequence to Sequence with RNNs

Input: Sequence $x_1, \ldots x_T$
Output: Sequence $y_1, \ldots, y_{T'}$

Encoder: $h_t = f_W(x_t, h_{t-1})$

From final hidden state predict:
Initial decoder state s_0
Context vector c (often $c=h_T$)



Sutskever et al, “Sequence to sequence learning with neural networks”, NeurIPS 2014

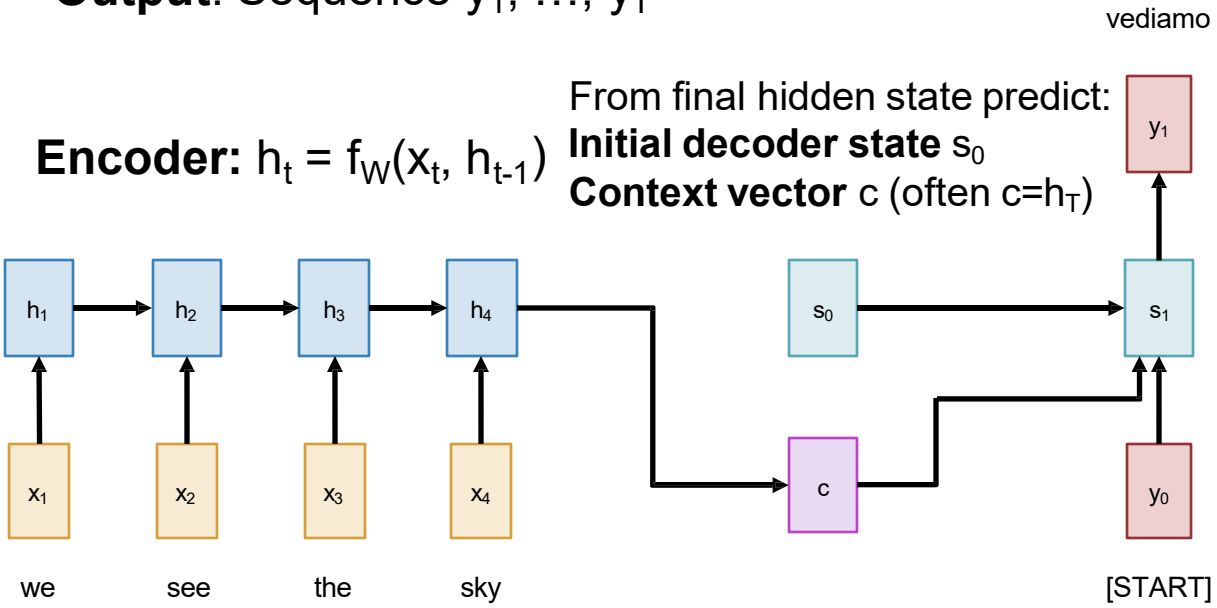
Sequence to Sequence with RNNs

Input: Sequence $x_1, \ldots x_T$
Output: Sequence $y_1, \ldots, y_{T'}$

Decoder: $s_t = g_U(y_{t-1}, s_{t-1}, c)$

Encoder: $h_t = f_W(x_t, h_{t-1})$

From final hidden state predict:
Initial decoder state s_0
Context vector c (often $c=h_T$)



Sutskever et al, “Sequence to sequence learning with neural networks”, NeurIPS 2014

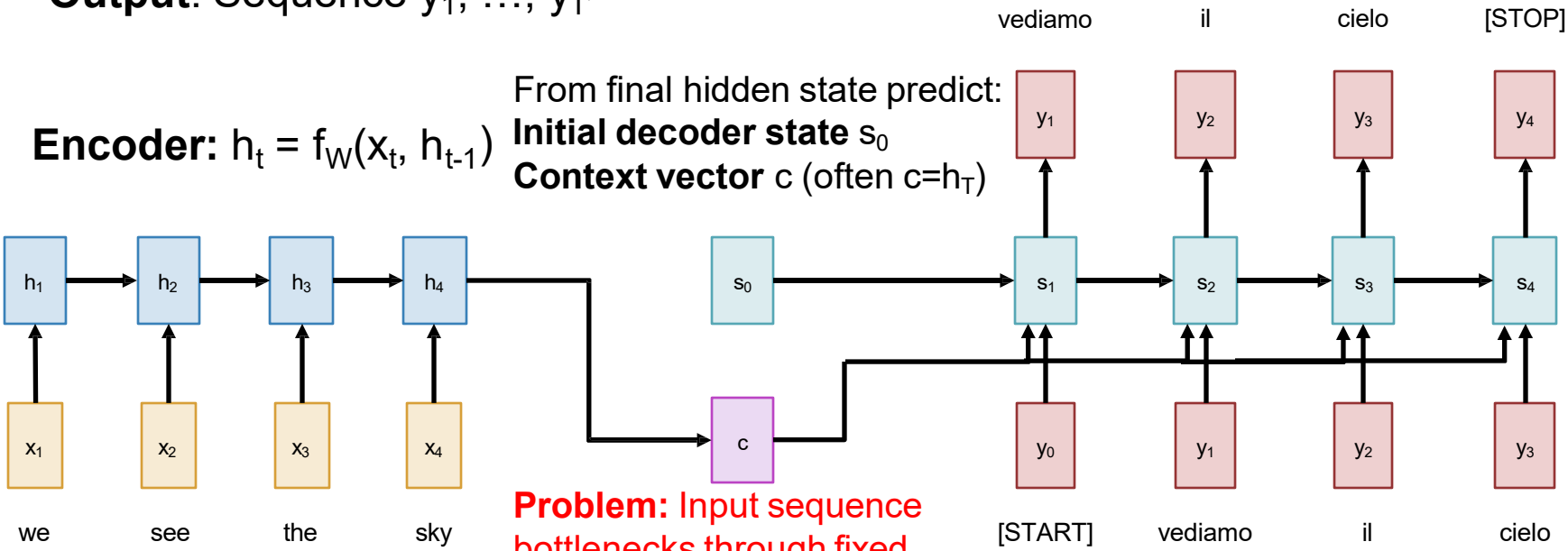
Sequence to Sequence with RNNs

Input: Sequence $x_1, \ldots x_T$
Output: Sequence $y_1, \ldots, y_{T'}$

Decoder: $s_t = g_U(y_{t-1}, s_{t-1}, c)$

Encoder: $h_t = f_W(x_t, h_{t-1})$

From final hidden state predict:
Initial decoder state s_0
Context vector c (often $c=h_T$)



Problem: Input sequence bottlenecks through fixed sized c. What if T=1000?

Sutskever et al, "Sequence to sequence learning with neural networks", NeurIPS 2014

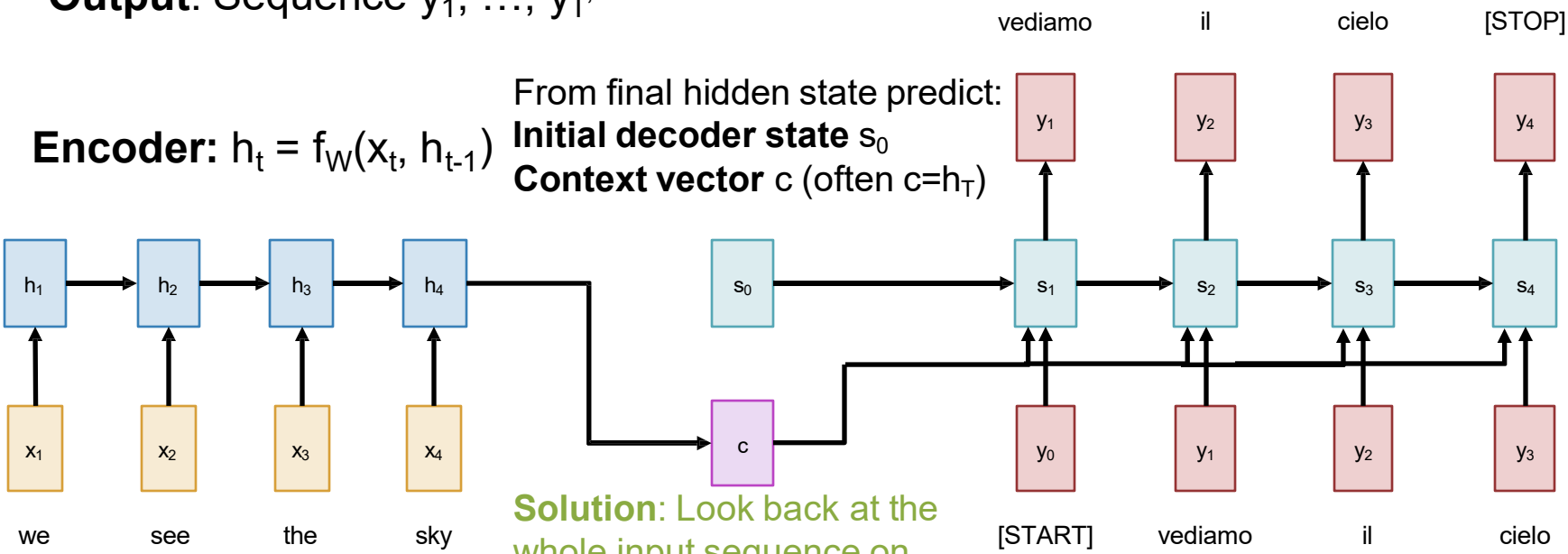
Sequence to Sequence with RNNs

Input: Sequence $x_1, \ldots x_T$
Output: Sequence $y_1, \ldots, y_{T'}$

Decoder: $s_t = g_U(y_{t-1}, s_{t-1}, c)$

Encoder: $h_t = f_W(x_t, h_{t-1})$

From final hidden state predict:
Initial decoder state s_0
Context vector c (often $c=h_T$)



Solution: Look back at the whole input sequence on each step of the output

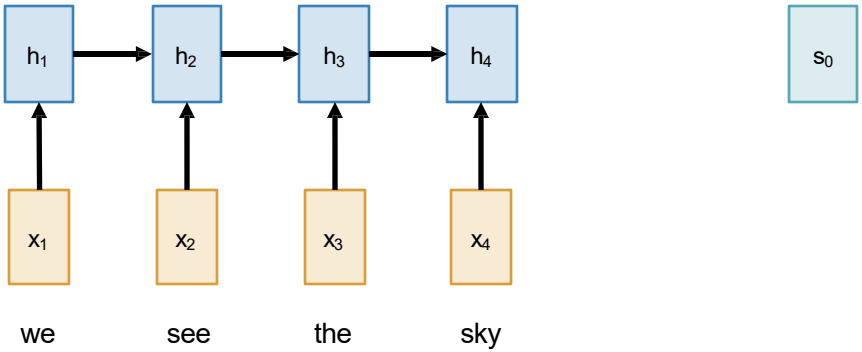
Sutskever et al, "Sequence to sequence learning with neural networks", NeurIPS 2014

Sequence to Sequence with RNNs and Attention

Input: Sequence $x_1, \ldots x_T$
Output: Sequence $y_1, \ldots, y_{T'}$

Encoder: $h_t = f_W(x_t, h_{t-1})$

From final hidden state:
Initial decoder state s_0

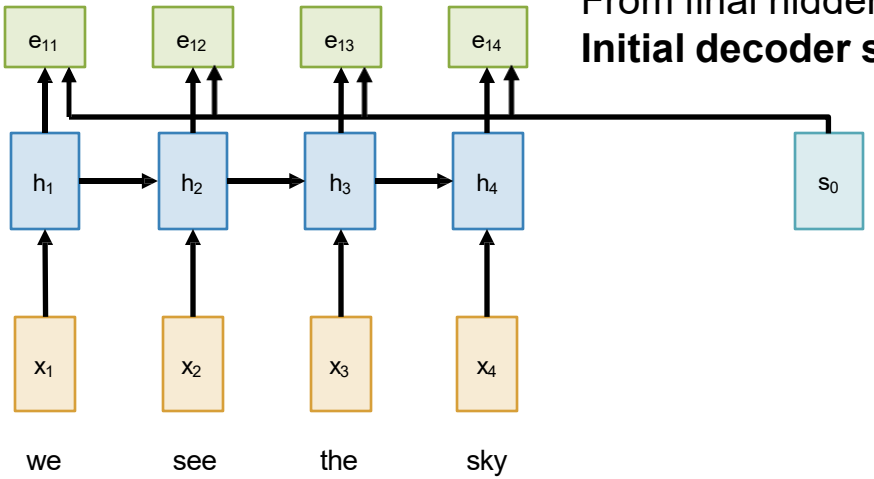


Bahdanau et al, "Neural machine translation by jointly learning to align and translate", ICLR 2015

Sequence to Sequence with RNNs and Attention

Compute (scalar) **alignment scores**

$e_{t,i} = f_{att}(s_{t-1}, h_i)$ (f_{att} is a Linear Layer)

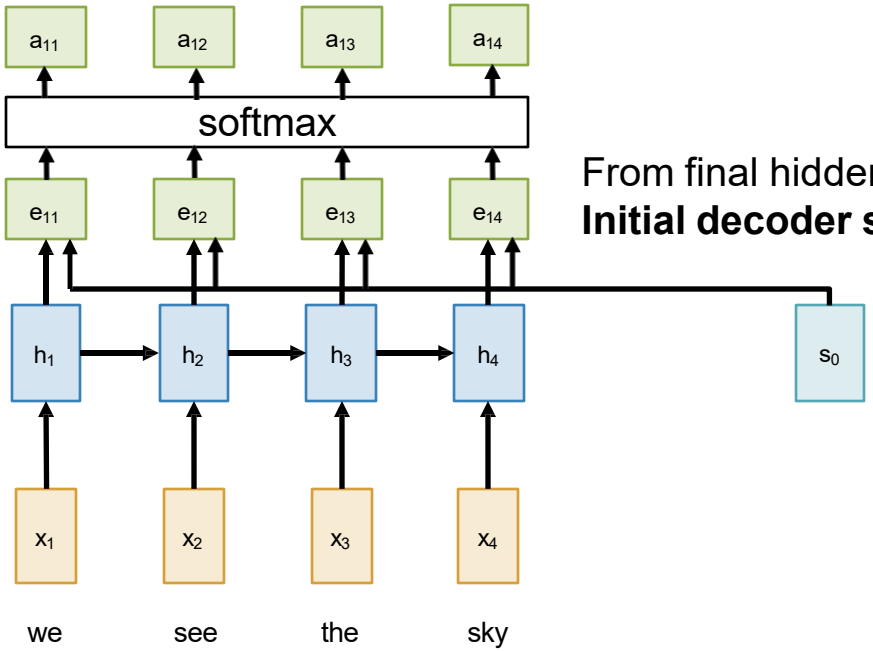


From final hidden state:

Initial decoder state s_0

Bahdanau et al, “Neural machine translation by jointly learning to align and translate”, ICLR 2015

Sequence to Sequence with RNNs and Attention



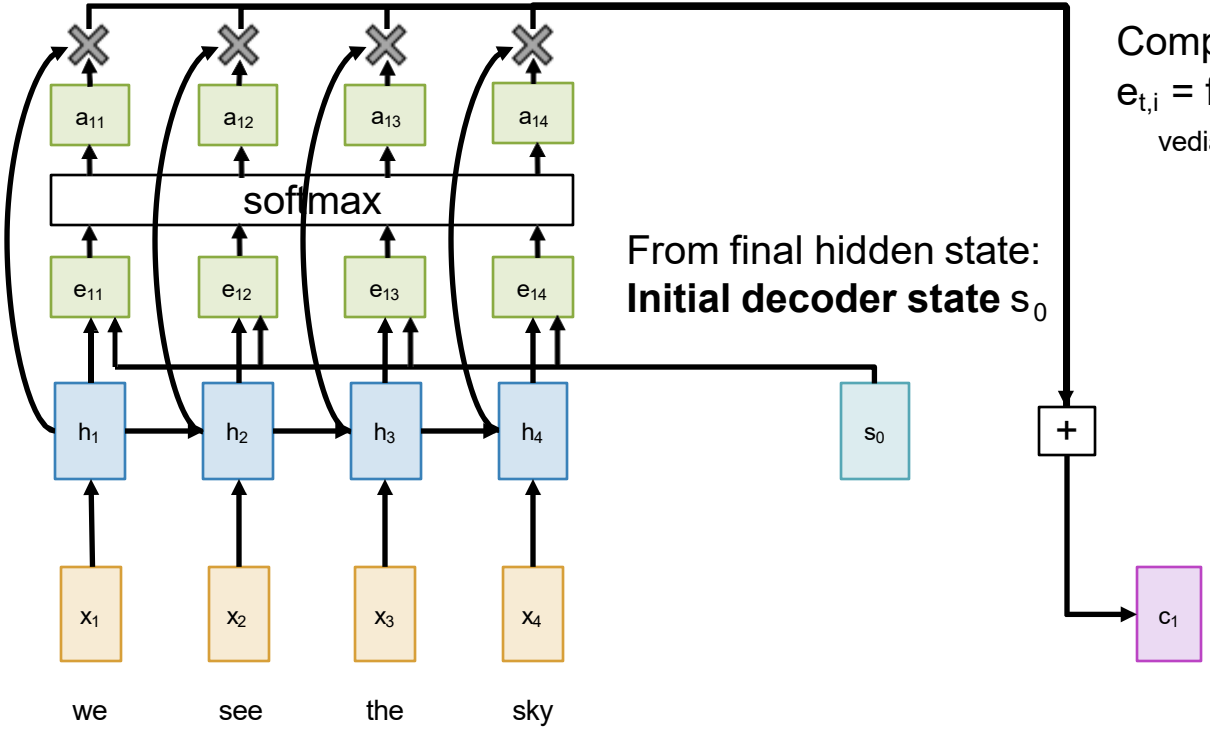
From final hidden state:
Initial decoder state s_0

Compute (scalar) **alignment scores**
$e_{t,i} = f_{att}(s_{t-1}, h_i)$ (f_{att} is a Linear Layer)

Normalize alignment scores to get **attention weights**
$0 < a_{t,i} < 1$ $\sum_i a_{t,i} = 1$

Bahdanau et al, "Neural machine translation by jointly learning to align and translate", ICLR 2015

Sequence to Sequence with RNNs and Attention



From final hidden state:
Initial decoder state s_0

Compute (scalar) **alignment scores**
$e_{t,i} = f_{att}(s_{t-1}, h_i)$ (f_{att} is a Linear Layer)

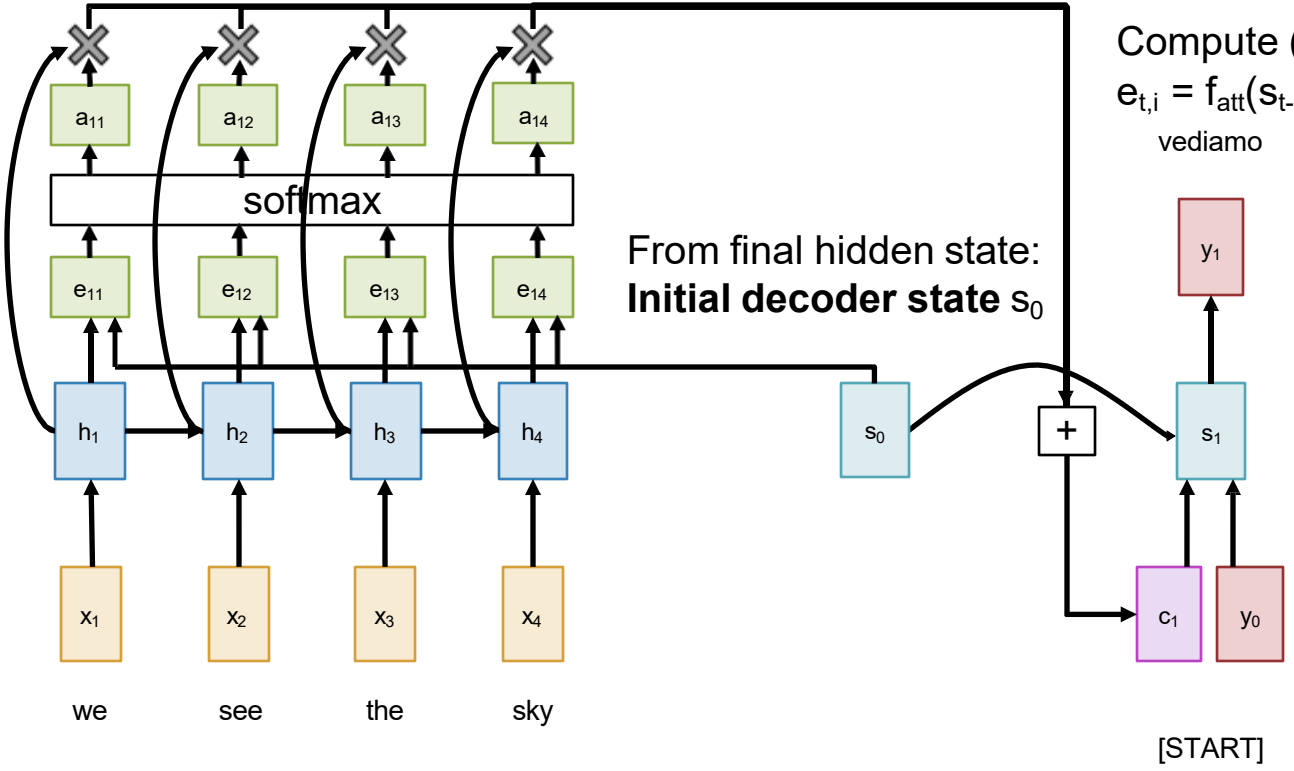
vediamo

Normalize alignment scores to get **attention weights**
$0 < a_{t,i} < 1 \quad \sum_i a_{t,i} = 1$

Compute context vector as **weighted sum of hidden states**
$c_t = \sum_i a_{t,i} h_i$

Bahdanau et al, "Neural machine translation by jointly learning to align and translate", ICLR 2015

Sequence to Sequence with RNNs and Attention



Compute (scalar) **alignment scores**

$e_{t,i} = f_{att}(s_{t-1}, h_i)$ (f_{att} is a Linear Layer)

vediamo

Normalize alignment scores to get **attention weights**

$0 < a_{t,i} < 1 \quad \sum_i a_{t,i} = 1$

Compute **context vector** as weighted sum of hidden states

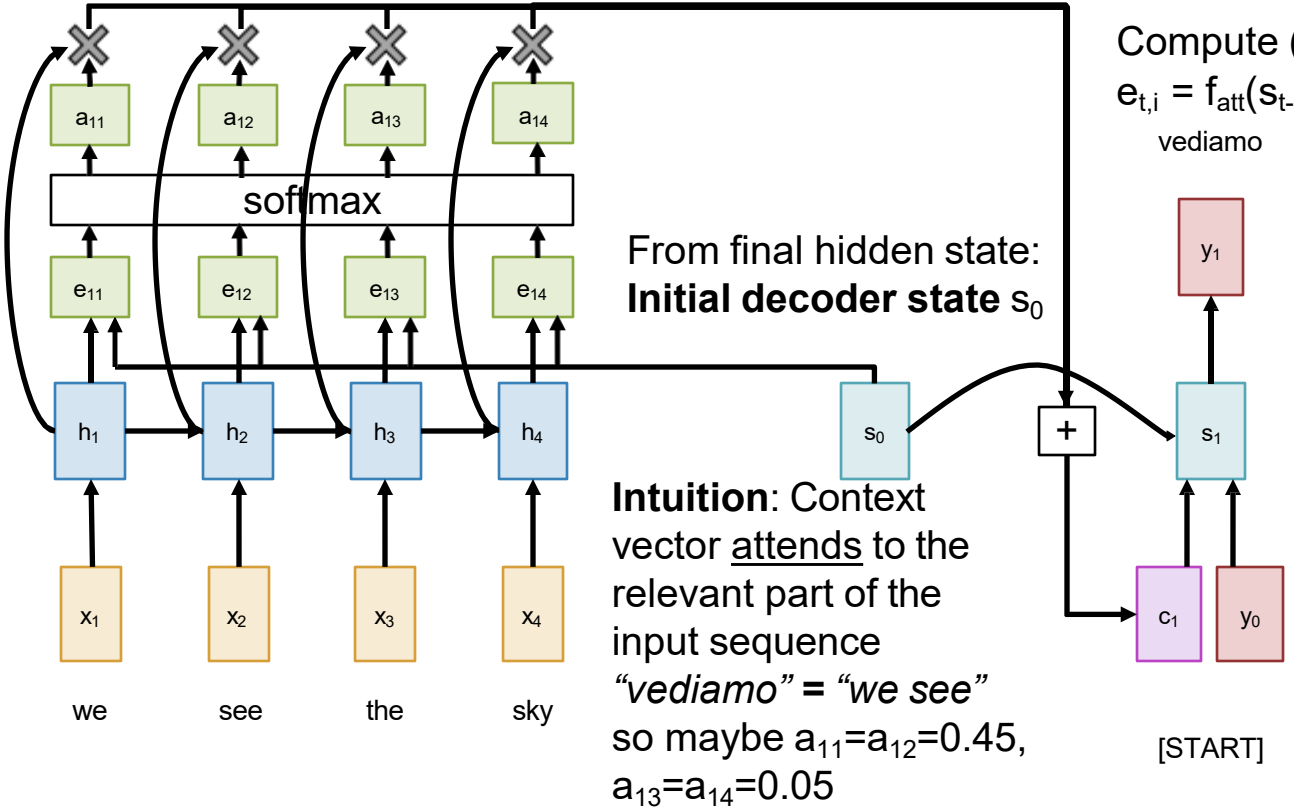
$c_t = \sum_i a_{t,i} h_i$

Use context vector in decoder: $s_t = g_U(y_{t-1}, s_{t-1}, c_t)$

g_U is an RNN unit (e.g. LSTM, GRU)

Bahdanau et al, "Neural machine translation by jointly learning to align and translate", ICLR 2015

Sequence to Sequence with RNNs and Attention



From final hidden state:
Initial decoder state s_0

Intuition: Context vector attends to the relevant part of the input sequence
"vediamo" **=** *"we see"*
so maybe a_{11}=a_{12}=0.45, a_{13}=a_{14}=0.05

Compute (scalar) **alignment scores**
$e_{t,i} = f_{att}(s_{t-1}, h_i)$ (f_{att} is a Linear Layer)

vediamo

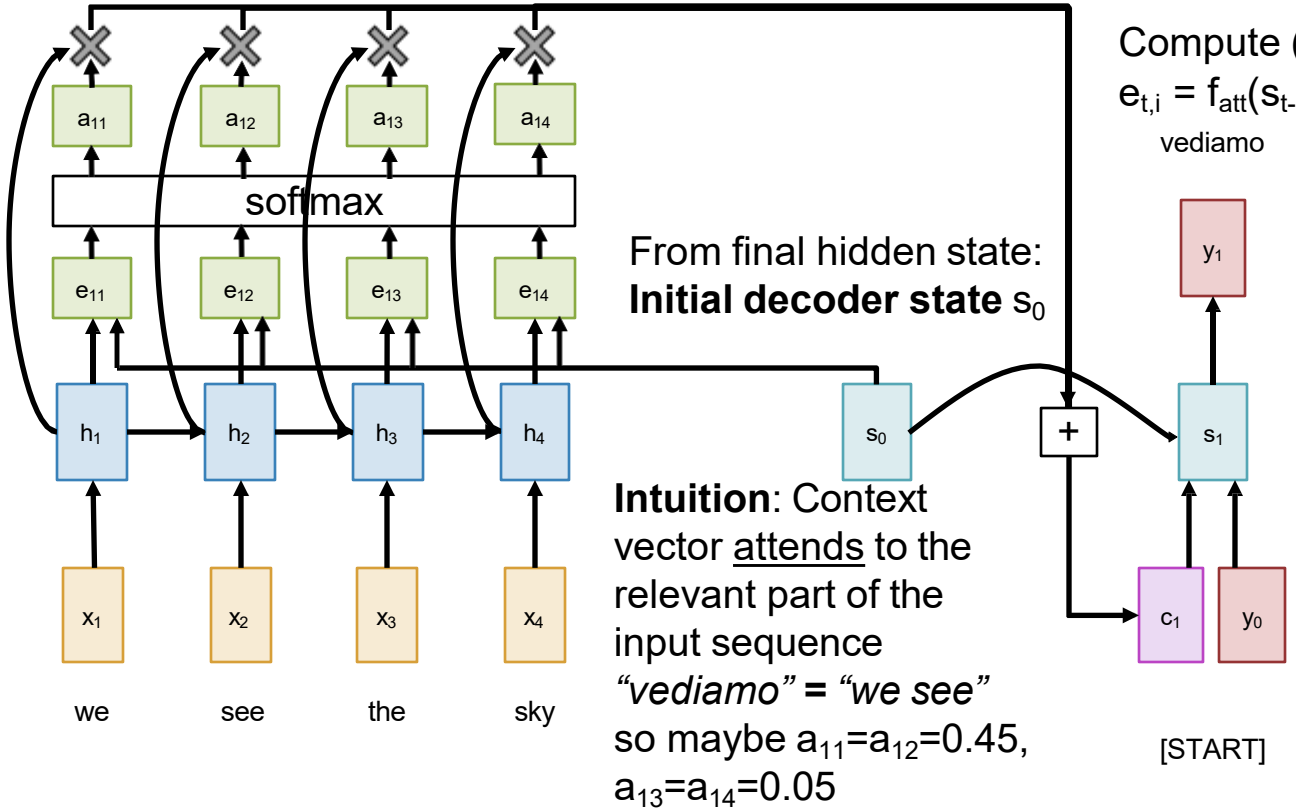
Normalize alignment scores to get **attention weights**
$0 < a_{t,i} < 1$ $\sum_i a_{t,i} = 1$

Compute **context vector** as weighted sum of hidden states
$c_t = \sum_i a_{t,i} h_i$

Use context vector in decoder: $s_t = g_U(y_{t-1}, s_{t-1}, c_t)$

Bahdanau et al, "Neural machine translation by jointly learning to align and translate", ICLR 2015

Sequence to Sequence with RNNs and Attention



From final hidden state:
Initial decoder state s_0

Intuition: Context vector attends to the relevant part of the input sequence
"vediamo" **=** *"we see"*
so maybe a_{11}=a_{12}=0.45, a_{13}=a_{14}=0.05

Compute (scalar) **alignment scores**
$e_{t,i} = f_{att}(s_{t-1}, h_i)$ (f_{att} is a Linear Layer)

vediamo

Normalize alignment scores to get **attention weights**
$0 < a_{t,i} < 1$ $\sum_i a_{t,i} = 1$

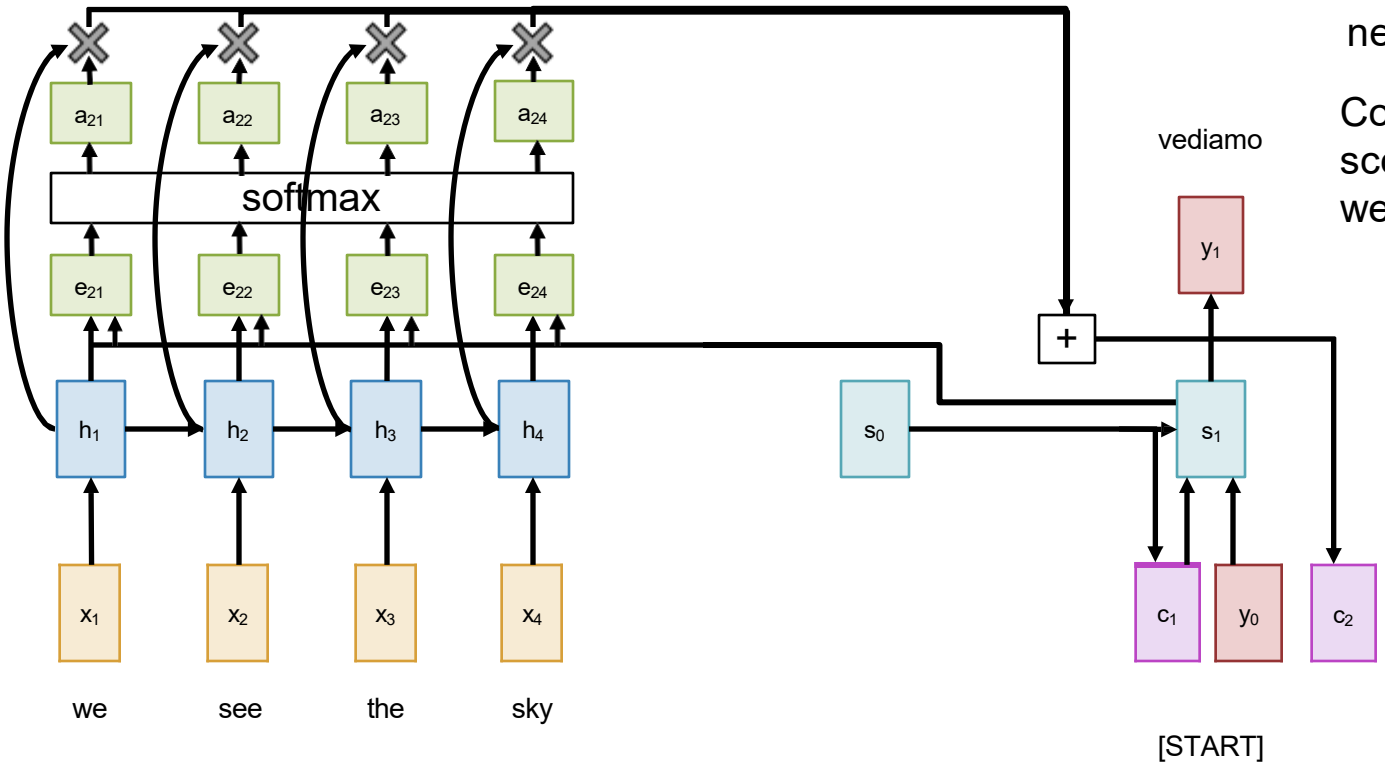
Compute **context vector** as weighted sum of hidden states
$c_t = \sum_i a_{t,i} h_i$

Use context vector in decoder: $s_t = g_U(y_{t-1}, s_{t-1}, c_t)$

All differentiable! No supervision on attention weights. Backprop through everything

Bahdanau et al, "Neural machine translation by jointly learning to align and translate", ICLR 2015

Sequence to Sequence with RNNs and Attention

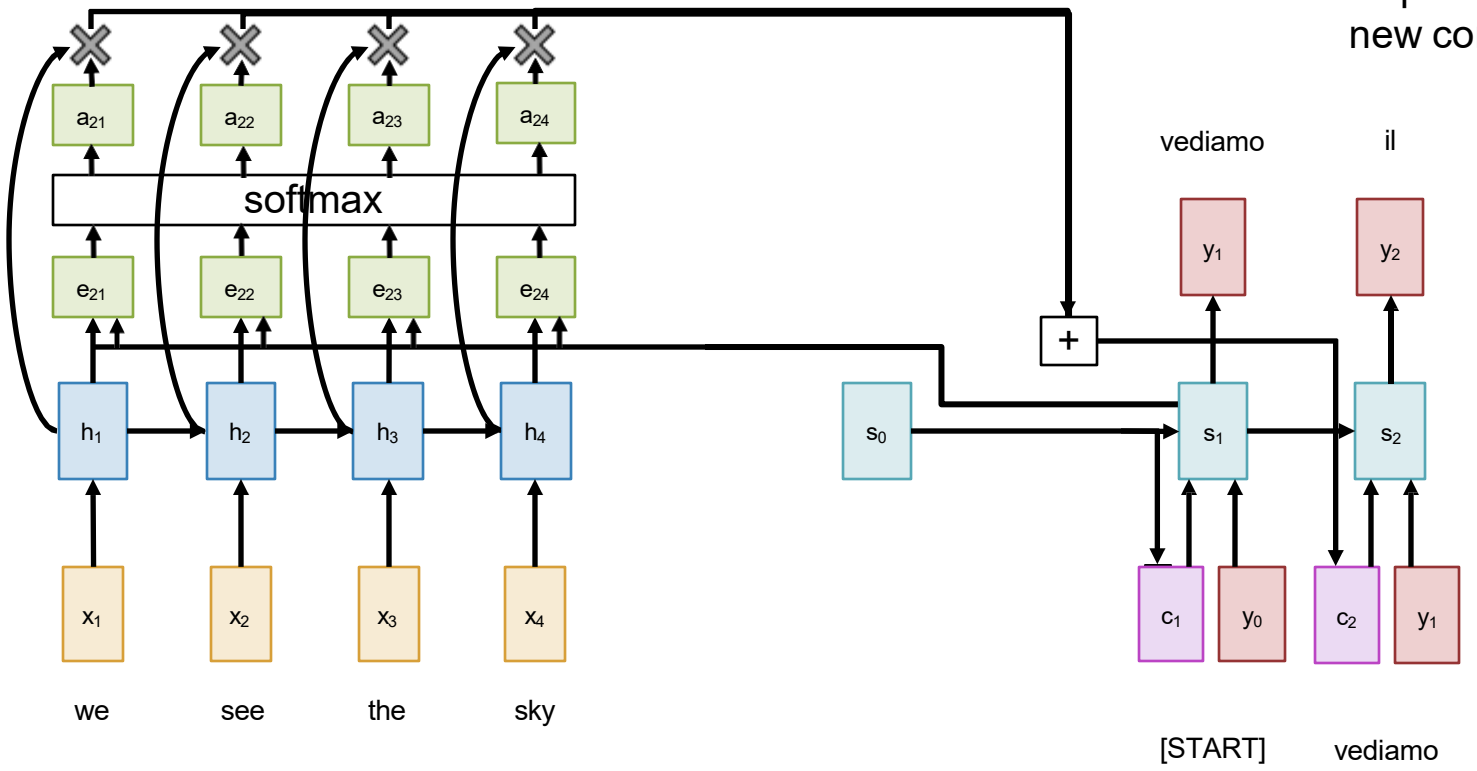


Repeat: Use s_1 to compute new context vector c_2

Compute new alignment scores $e_{2,i}$ and attention weights $a_{2,i}$

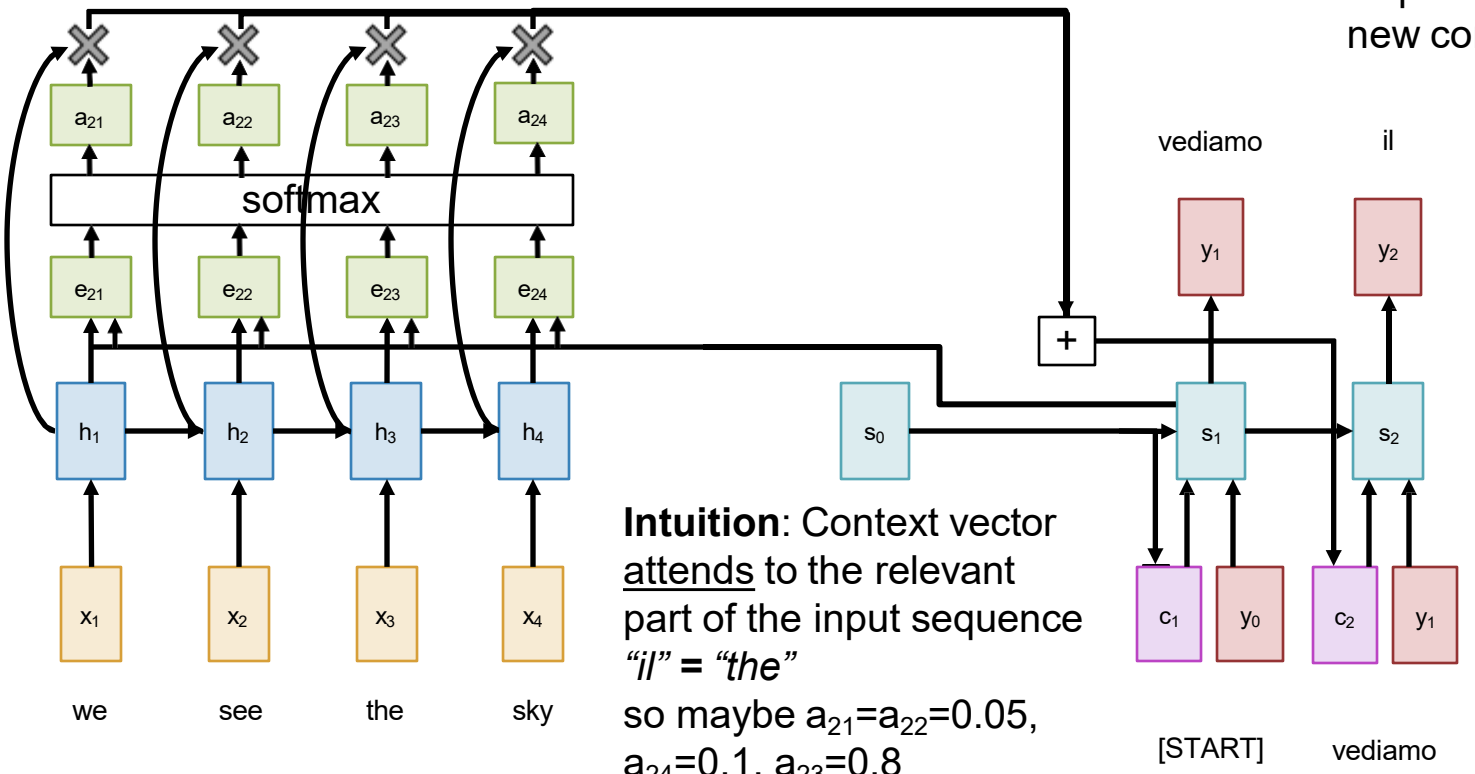
Bahdanau et al, “Neural machine translation by jointly learning to align and translate”, ICLR 2015

Sequence to Sequence with RNNs and Attention



Bahdanau et al, "Neural machine translation by jointly learning to align and translate", ICLR 2015

Sequence to Sequence with RNNs and Attention



Repeat: Use s_1 to compute new context vector c_2

Use context vector in decoder: $s_t = g_U(y_{t-1}, s_{t-1}, c_t)$

Intuition: Context vector attends to the relevant part of the input sequence

"il" = "the"

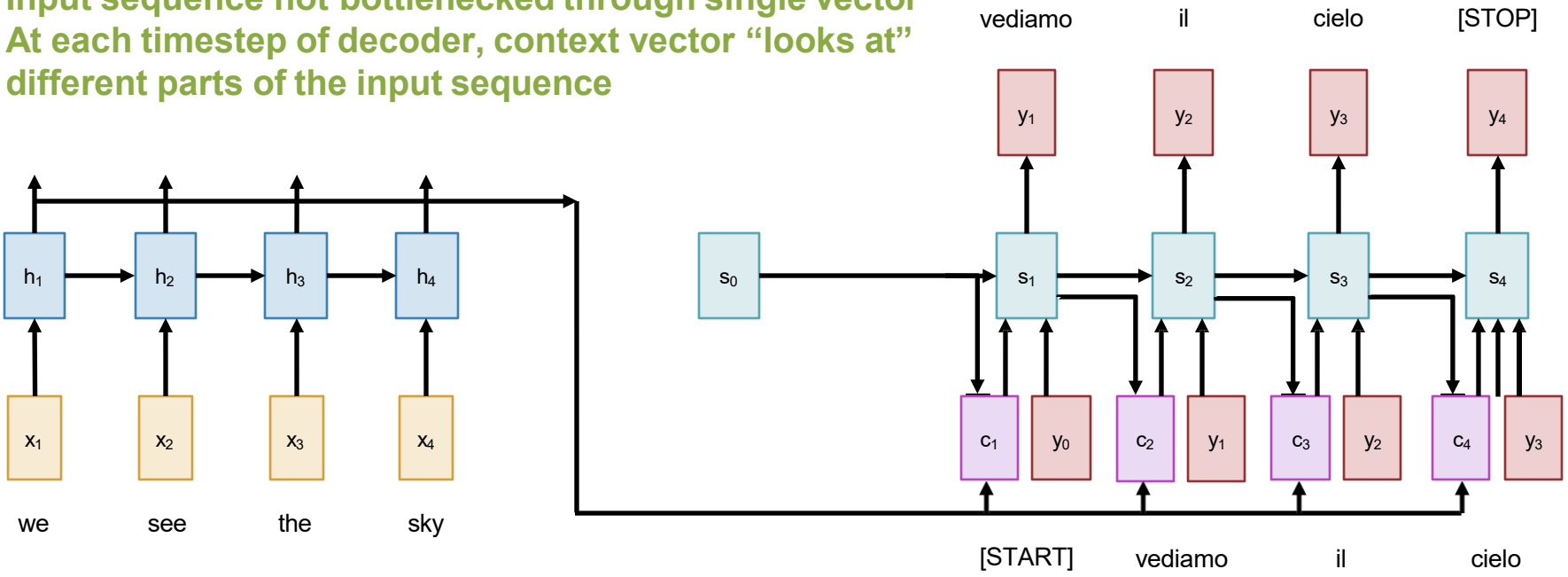
so maybe $a_{21}=a_{22}=0.05$, $a_{24}=0.1$, $a_{23}=0.8$

Bahdanau et al, "Neural machine translation by jointly learning to align and translate", ICLR 2015

Sequence to Sequence with RNNs and Attention

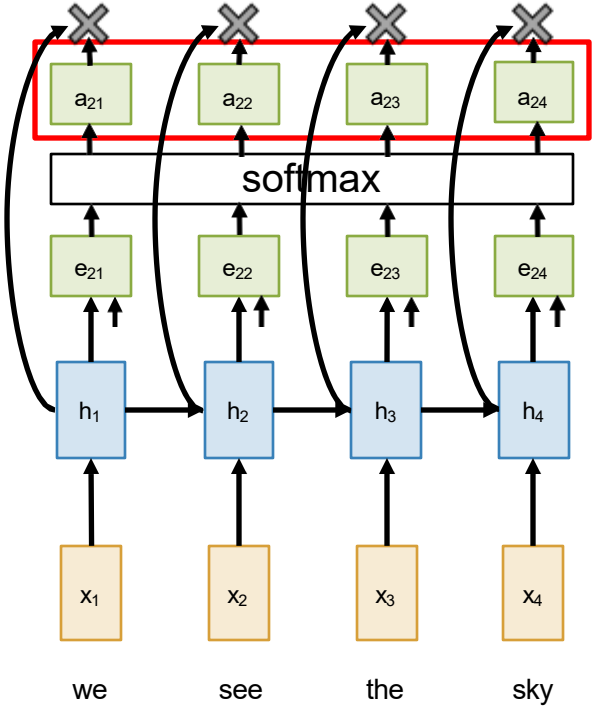
Use a different context vector in each timestep of decoder

- **Input sequence not bottlenecked through single vector**
- **At each timestep of decoder, context vector "looks at" different parts of the input sequence**



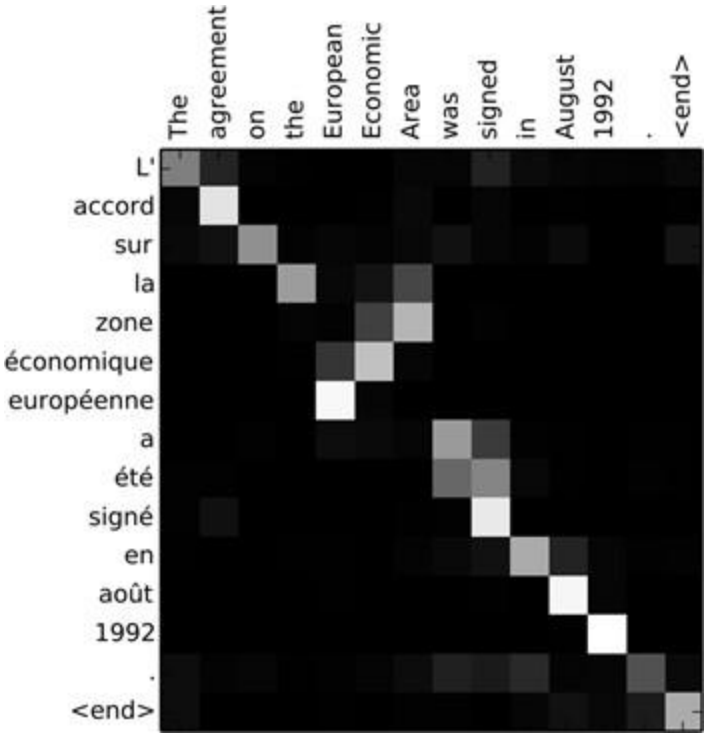
Bahdanau et al, "Neural machine translation by jointly learning to align and translate", ICLR 2015

Sequence to Sequence with RNNs and Attention



Example: English to French translation

Visualize attention weights $a_{t,i}$

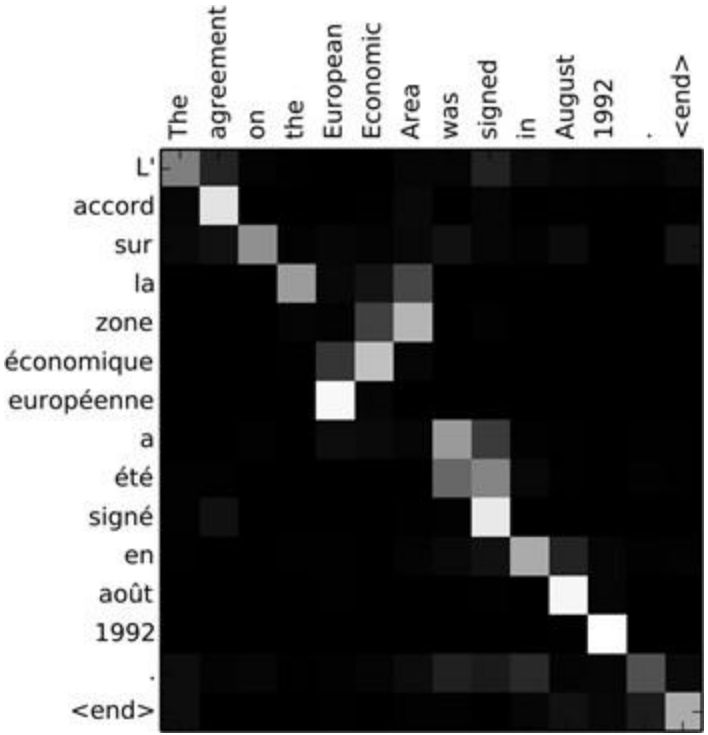


Bahdanau et al, “Neural machine translation by jointly learning to align and translate”, ICLR 2015

Sequence to Sequence with RNNs and Attention

Example: English to French translation

Visualize attention weights $a_{t,i}$



Input: “The agreement on the European Economic Area was signed in August 1992.”

Output: “L’accord sur la zone économique européenne a été signé en août 1992.”

Bahdanau et al, “Neural machine translation by jointly learning to align and translate”, ICLR 2015

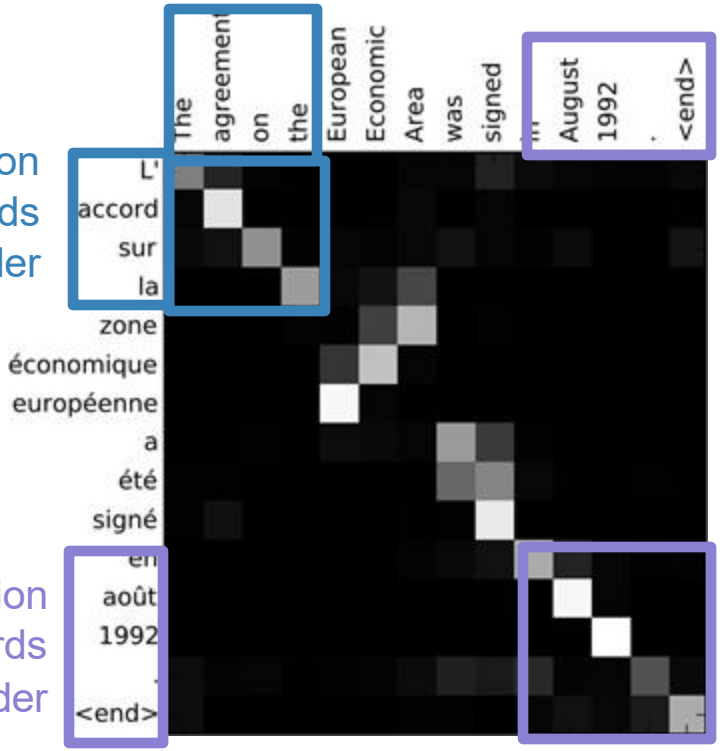
Sequence to Sequence with RNNs and Attention

Example: English to French translation

Input: "**The agreement on the** European Economic Area was signed in **August 1992.**"

Output: "**L'accord sur la** zone économique européenne a été signé **en août 1992.**"

Visualize attention weights $a_{t,i}$



Bahdanau et al, "Neural machine translation by jointly learning to align and translate", ICLR 2015

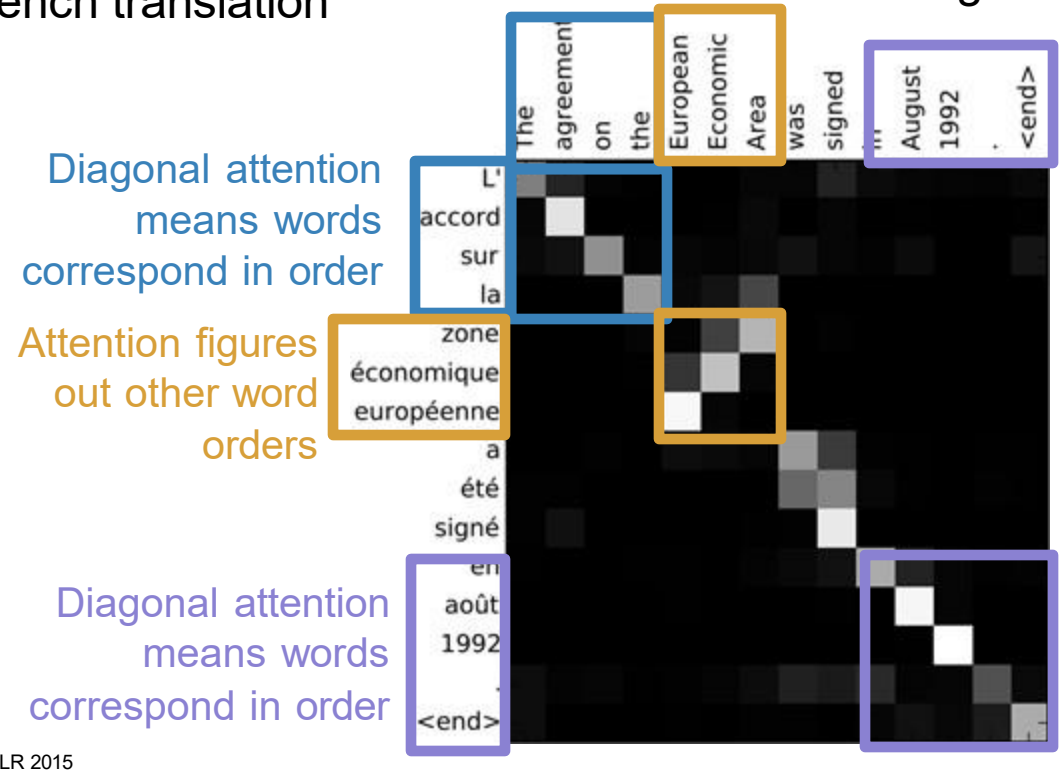
Sequence to Sequence with RNNs and Attention

Example: English to French translation

Input: "The agreement on the European Economic Area was signed in August 1992."

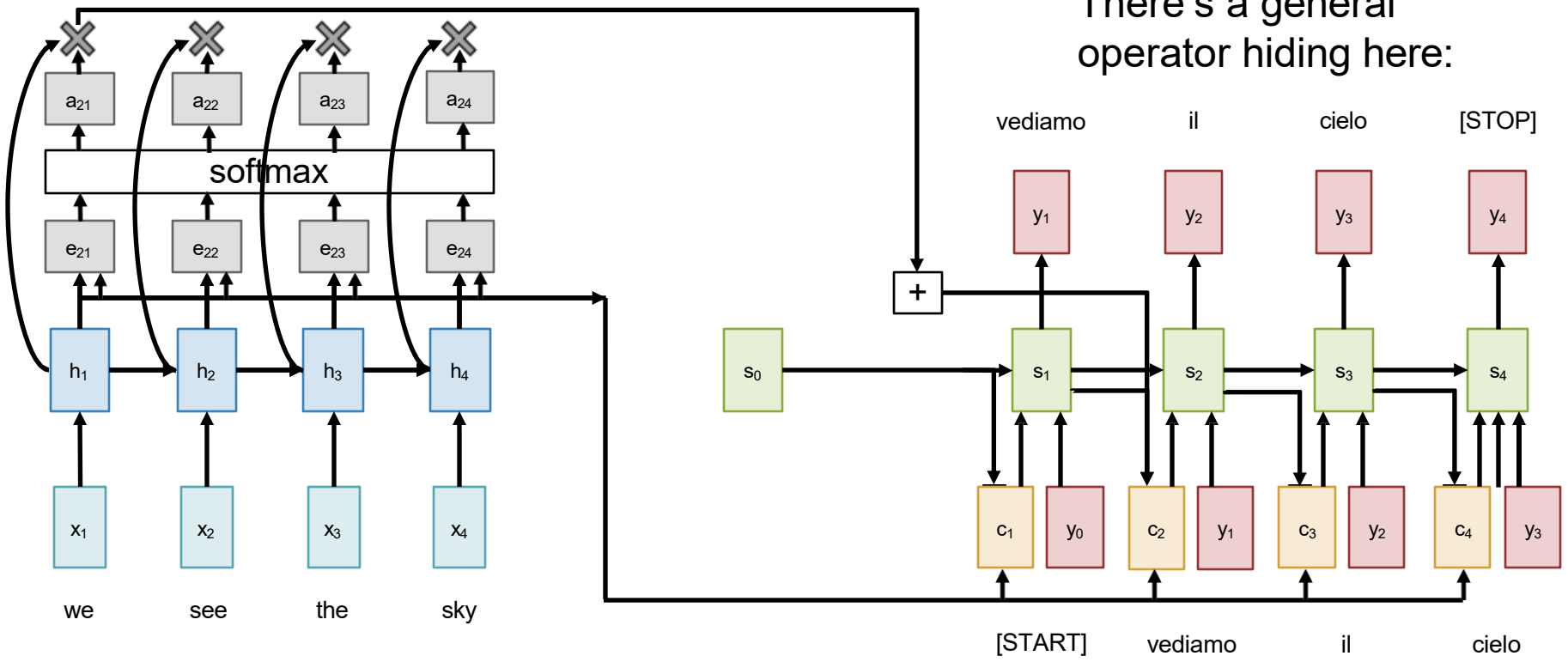
Output: "L'accord sur la zone économique européenne a été signé en août 1992."

Visualize attention weights $a_{t,i}$



Bahdanau et al, "Neural machine translation by jointly learning to align and translate", ICLR 2015

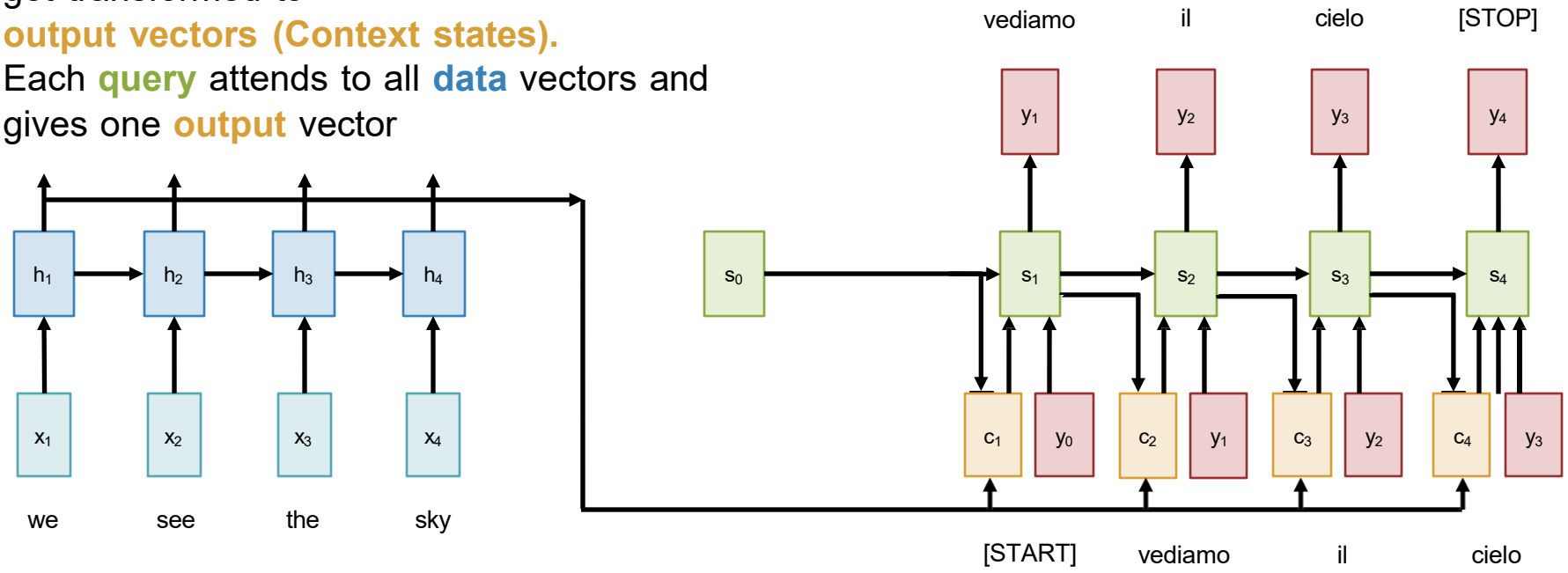
Sequence to Sequence with RNNs and Attention



Sequence to Sequence with RNNs and Attention

Query vectors (decoder RNN states) and **data vectors (encoder RNN states)** get transformed to **output vectors (Context states).** Each **query** attends to all **data** vectors and gives one **output** vector

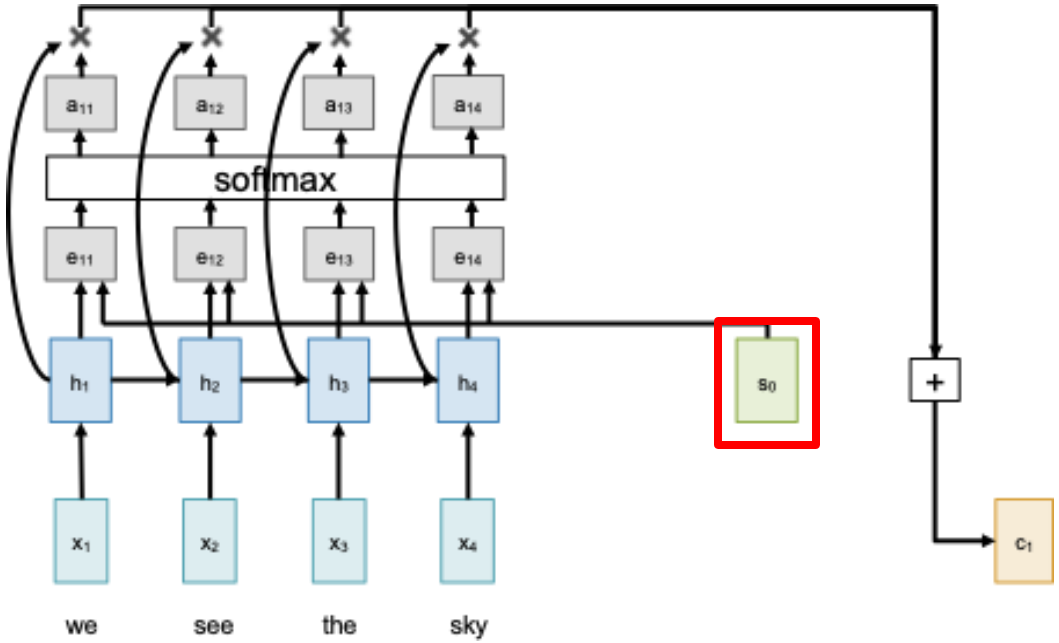
There's a general operator hiding here:



Attention Layer

Inputs:

Query vector: **q** [D_Q]

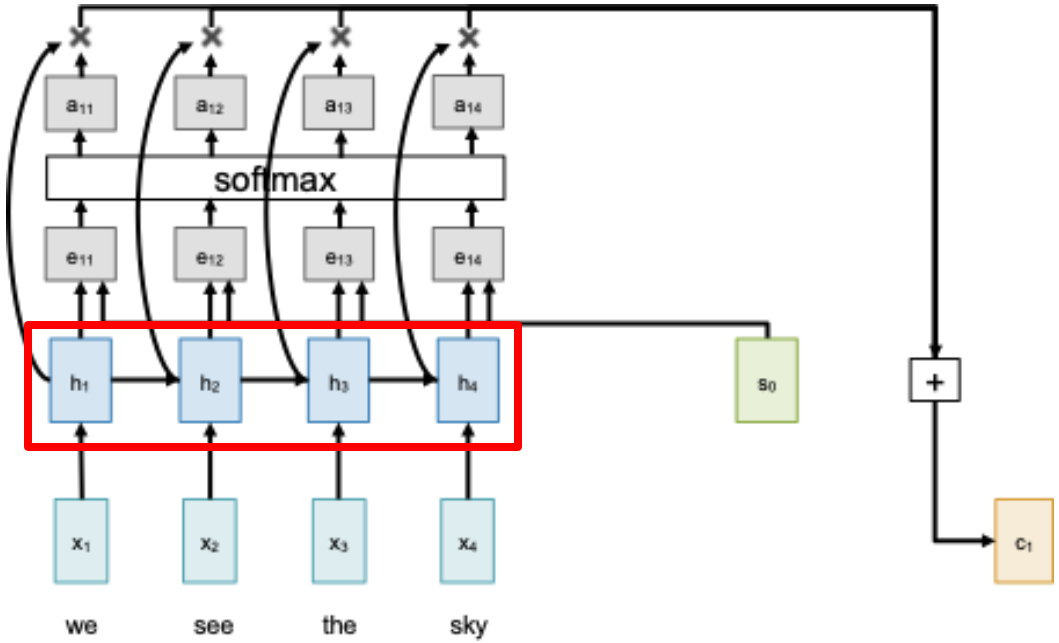


Attention Layer

Inputs:

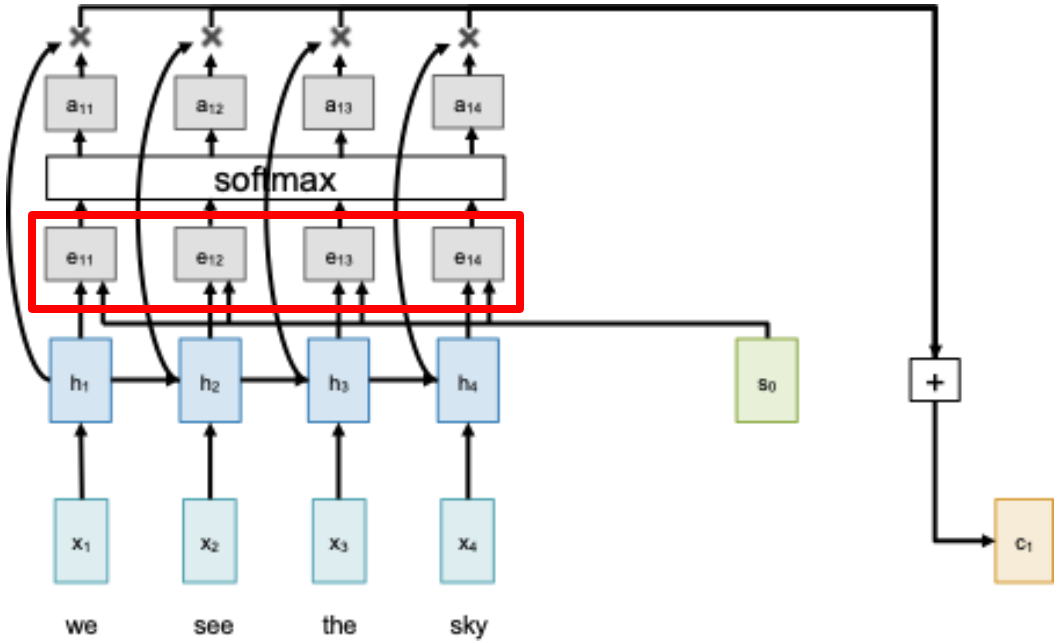
Query vector: q [D_Q]

Data vectors: X [N_X x D_X]



Attention Layer

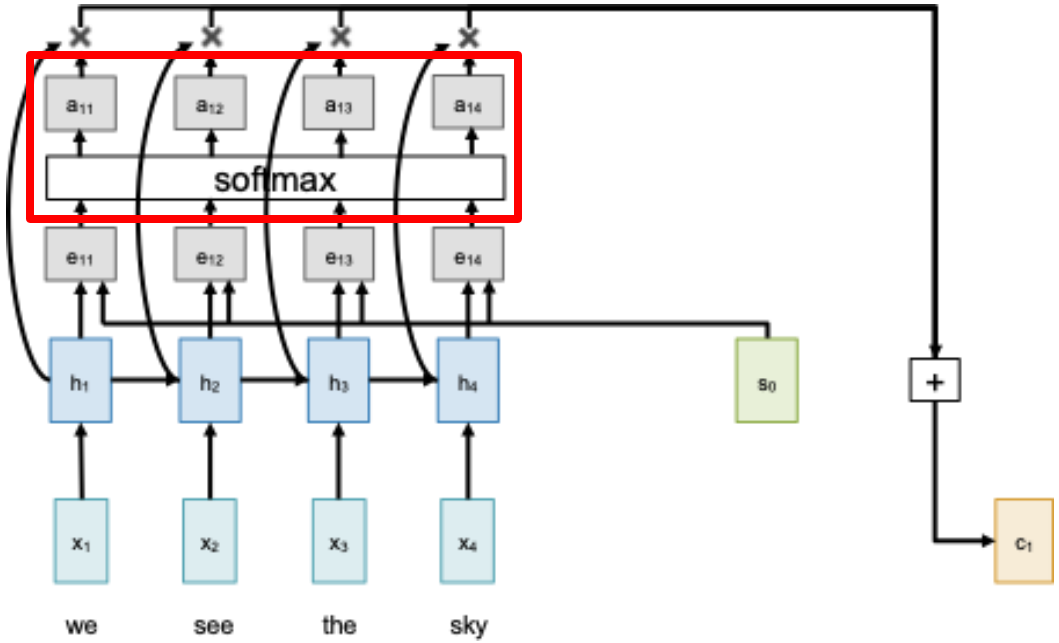
Inputs:
Query vector: q $[D_Q]$
Data vectors: X $[N_X \times D_X]$



Computation:
Similarities: e $[N_X]$ $e_i = f_{att}(q, X_i)$

Attention Layer

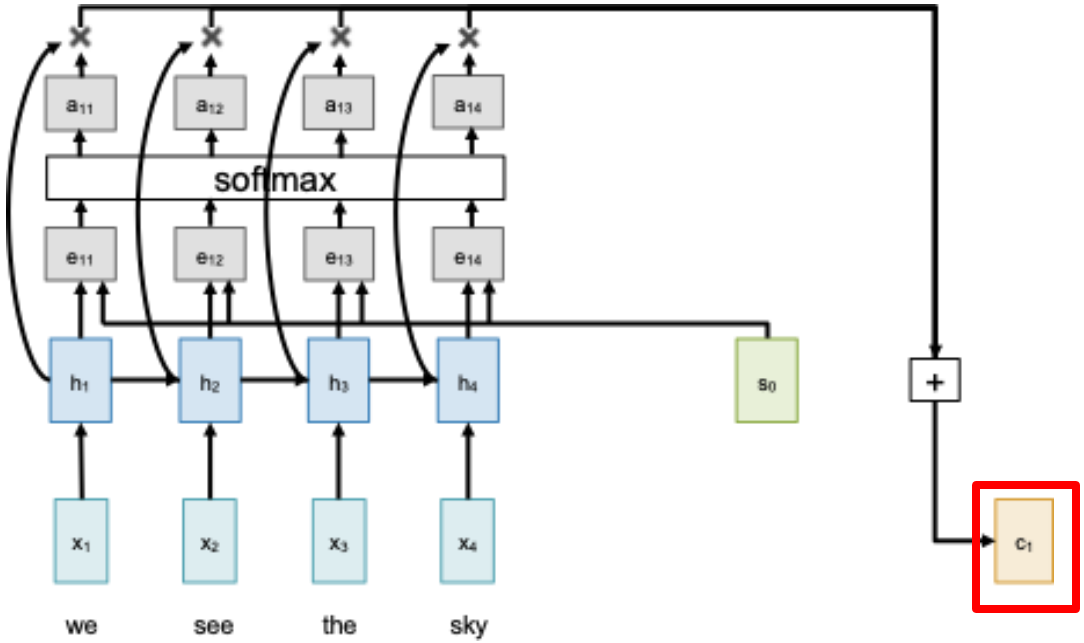
Inputs:
Query vector: **q** $[D_Q]$
Data vectors: **X** $[N_X \times D_X]$



Computation:
Similarities: e $[N_X]$ $e_i = f_{att}(\mathbf{q}, \mathbf{X}_i)$
Attention weights: a = softmax(e) $[N_X]$

Attention Layer

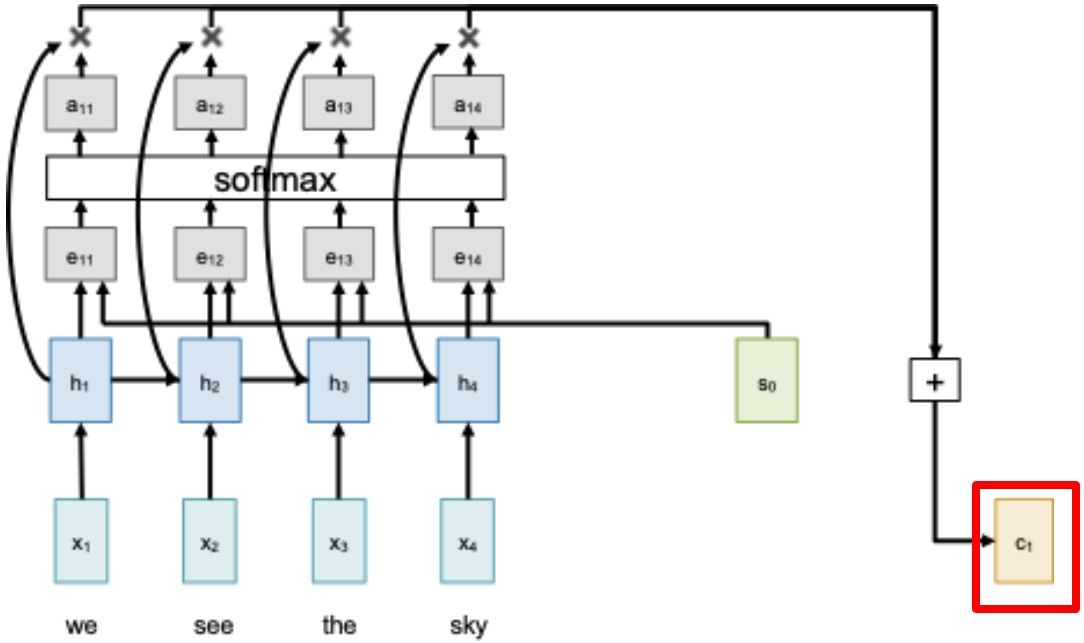
Inputs:
Query vector: q $[D_Q]$
Data vectors: X $[N_X \times D_X]$



Computation:
Similarities: e $[N_X]$ $e_i = f_{att}(q, X_i)$
Attention weights: a = softmax(e) $[N_X]$
Output vector: y = $\sum_i a_i X_i$ $[D_X]$

Attention Layer

Inputs:
Query vector: q $[D_Q]$
Data vectors: X $[N_X \times D_X]$

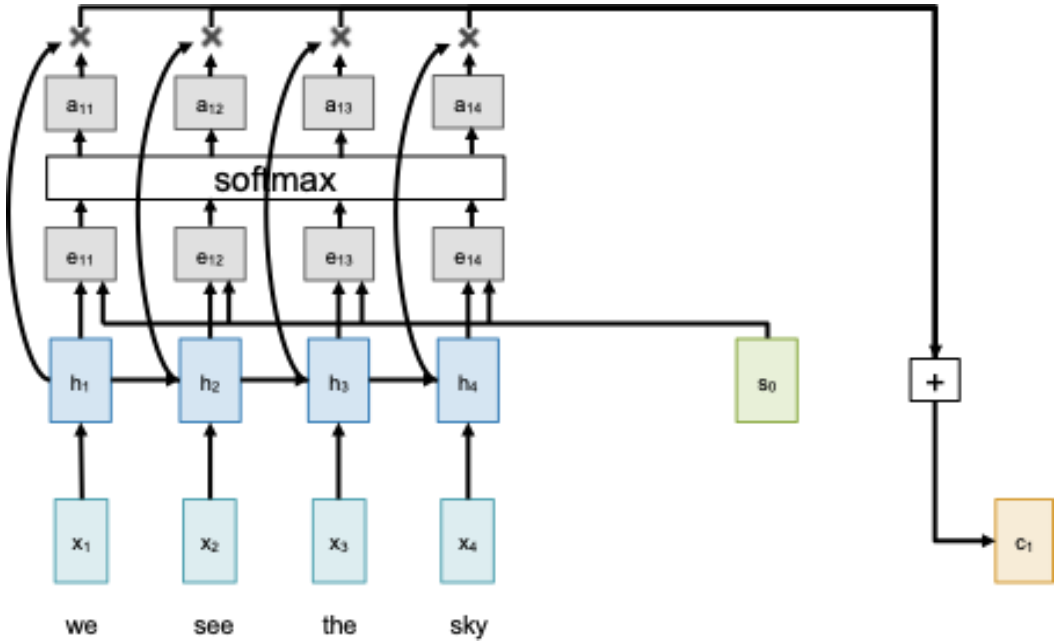


Computation:
Similarities: e $[N_X]$ $e_i = f_{att}(q, X_i)$
Attention weights: a = softmax(e) $[N_X]$
Output vector: y = $\sum_i a_i X_i$ $[D_X]$

Let's generalize this!

Attention Layer

Inputs:
Query vector: q $[D_X]$
Data vectors: X $[N_X \times D_X]$

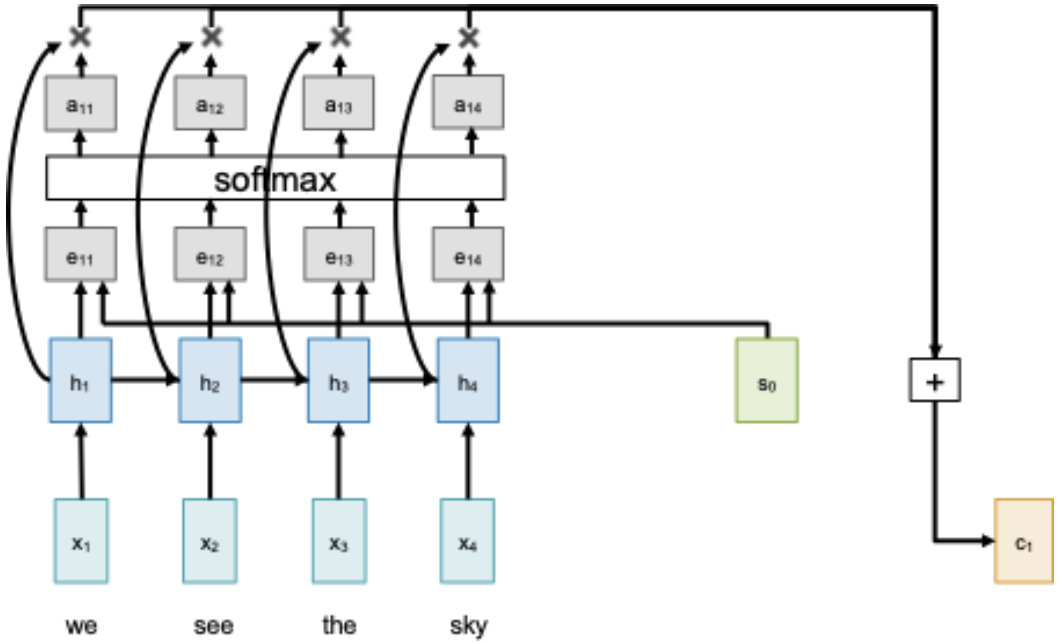


Computation:
Similarities: e $[N_X]$ $e_i = q \cdot X_i$
Attention weights: a = softmax(e) $[N_X]$
Output vector: y = $\sum_i a_i X_i$ $[D_X]$

Changes
- Use dot product for similarity

Attention Layer

Inputs:
Query vector: q $[D_X]$
Data vectors: X $[N_X \times D_X]$



Computation:
Similarities: e $[N_X]$ $e_i = q \cdot X_i / \sqrt{D_X}$
Attention weights: a = softmax(e) $[N_X]$
Output vector: y = $\sum_i a_i X_i$ $[D_X]$

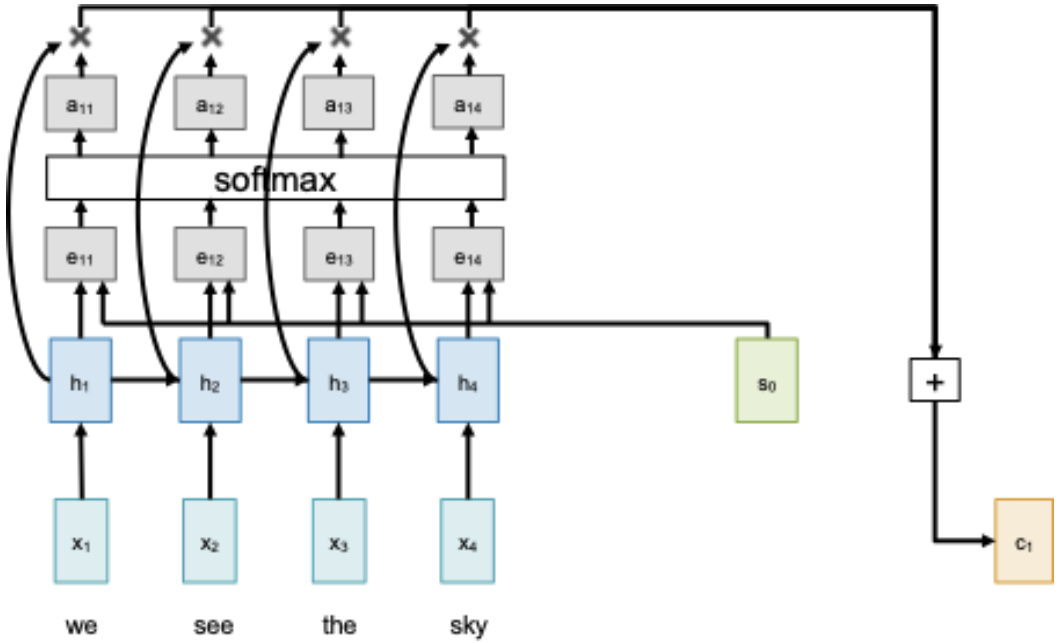
Changes
- Use **scaled** dot product for similarity

Attention Layer

Inputs:
Query vector: q $[D_X]$
Data vectors: X $[N_X \times D_X]$

Large similarities will cause softmax to saturate and give vanishing gradients
Recall $a \cdot b = |a||b| \cos(\text{angle})$
Suppose that a and b are constant vectors of dimension D
Then $|a| = (\sum_i a^2)^{1/2} = a\sqrt{D}$

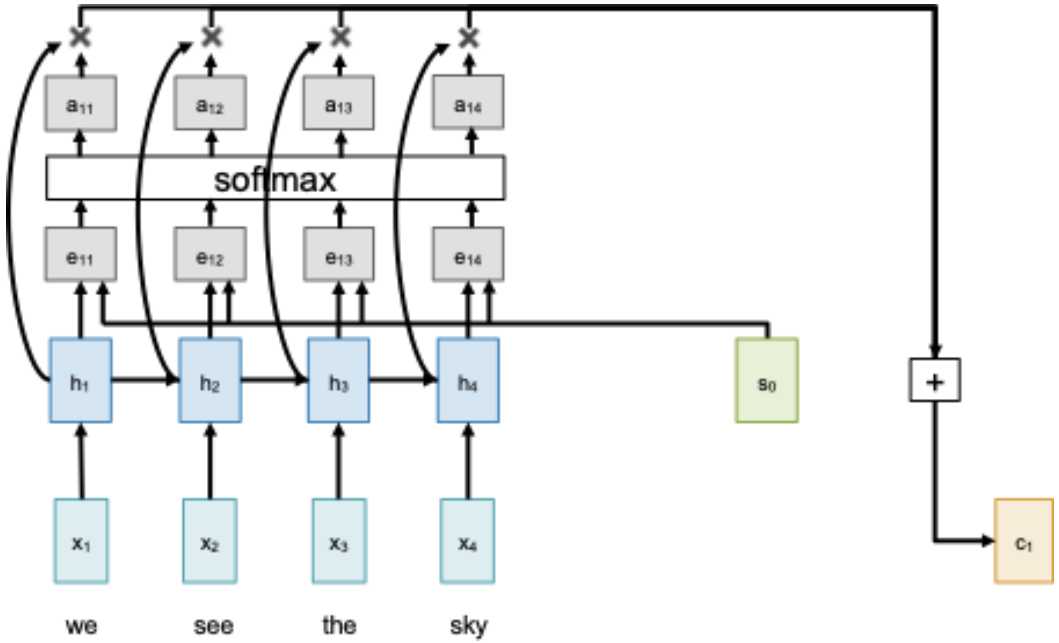
Computation:
Similarities: e $[N_X]$ $e_i = q \cdot X_i / \sqrt{D_X}$
Attention weights: a = softmax(e) $[N_X]$
Output vector: y = $\sum_i a_i X_i$ $[D_X]$



Changes
- Use **scaled** dot product for similarity

Attention Layer

Inputs:
Query vector: **Q** $[N_Q \times D_X]$
Data vectors: **X** $[N_X \times D_X]$



Computation:

Similarities: $E = QX^T / \sqrt{D_X}$ $[N_Q \times N_X]$

$E_{ij} = Q_i \cdot X_j / \sqrt{D_X}$

Attention weights: $A = \text{softmax}(E, \text{dim}=1)$ $[N_Q \times N_X]$

Output vector: $Y = AX$ $[N_Q \times D_X]$

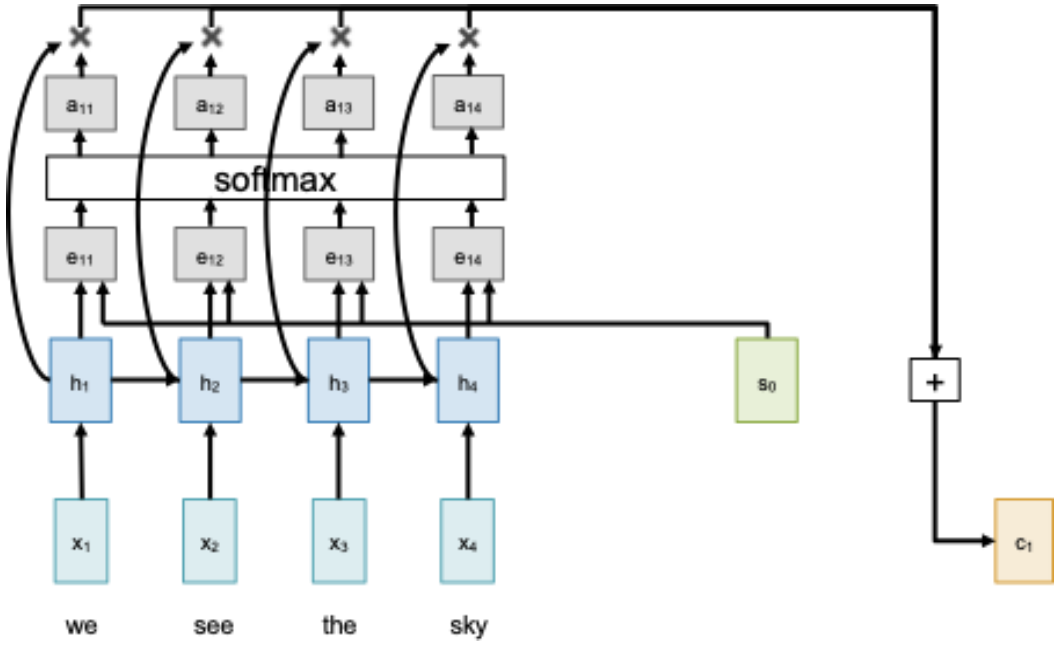
$Y_i = \sum_j A_{ij} X_j$

Changes

- Use scaled dot product for similarity
- Multiple **query** vectors

Attention Layer

Inputs:
Query vector: Q [N_Q x D_Q]
Data vectors: X [N_X x D_X]
Key matrix: W_K [D_X x D_Q]
Value matrix: W_V [D_X x D_V]



Computation:
Keys: $K = XW_K$ [N_X x D_Q]
Values: $V = XW_V$ [N_X x D_V]
Similarities: $E = QK^T / \sqrt{D_Q}$ [N_Q x N_X]
$E_{ij} = Q_i \cdot K_j / \sqrt{D_Q}$
Attention weights: A = softmax(E, dim=1) [N_Q x N_X]
Output vector: $Y = AV$ [N_Q x D_V]
$Y_i = \sum_j A_{ij} V_j$

Changes
- Use scaled dot product for similarity
- Multiple **query** vectors
- Separate **key** and **value**

Attention Layer

Inputs:

Query vector: **Q** $[N_Q \times D_Q]$

Data vectors: **X** $[N_X \times D_X]$

Key matrix: W_K $[D_X \times D_Q]$

Value matrix: W_V $[D_X \times D_V]$

Computation:

Keys: $K = XW_K$ $[N_X \times D_Q]$

Values: $V = XW_V$ $[N_X \times D_V]$

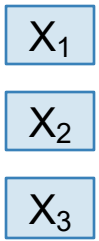
Similarities: $E = QK^T / \sqrt{D_Q}$ $[N_Q \times N_X]$

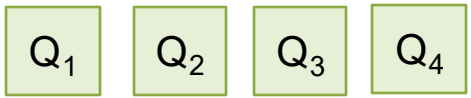
$E_{ij} = Q_i \cdot K_j / \sqrt{D_Q}$

Attention weights: $A = \text{softmax}(E, \text{dim}=1)$ $[N_Q \times N_X]$

Output vector: $Y = AV$ $[N_Q \times D_V]$

$Y_i = \sum_j A_{ij} V_j$





Attention Layer

Inputs:

Query vector: **Q** $[N_Q \times D_Q]$

Data vectors: **X** $[N_X \times D_X]$

Key matrix: $\mathbf{W_K}$ $[D_X \times D_Q]$

Value matrix: $\mathbf{W_V}$ $[D_X \times D_V]$

Computation:

Keys: $\mathbf{K} = \mathbf{X}\mathbf{W_K}$ $[N_X \times D_Q]$

Values: $\mathbf{V} = \mathbf{X}\mathbf{W_V}$ $[N_X \times D_V]$

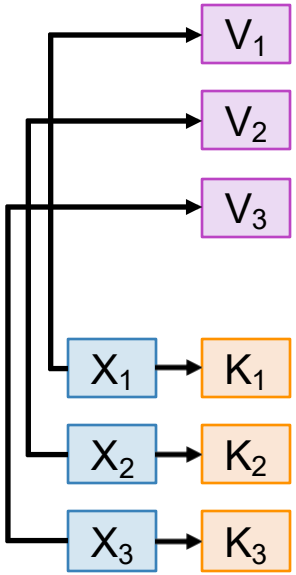
Similarities: $E = \mathbf{Q}\mathbf{K}^T / \sqrt{D_Q}$ $[N_Q \times N_X]$

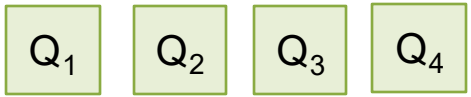
$E_{ij} = \mathbf{Q}_i \cdot \mathbf{K}_j / \sqrt{D_Q}$

Attention weights: A = softmax(E, dim=1) $[N_Q \times N_X]$

Output vector: $\mathbf{Y} = A\mathbf{V}$ $[N_Q \times D_V]$

$\mathbf{Y}_i = \sum_j A_{ij} \mathbf{V}_j$





Attention Layer

Inputs:

Query vector: Q $[N_Q \times D_Q]$

Data vectors: X $[N_X \times D_X]$

Key matrix: W_K $[D_X \times D_Q]$

Value matrix: W_V $[D_X \times D_V]$

Computation:

Keys: $K = XW_K$ $[N_X \times D_Q]$

Values: $V = XW_V$ $[N_X \times D_V]$

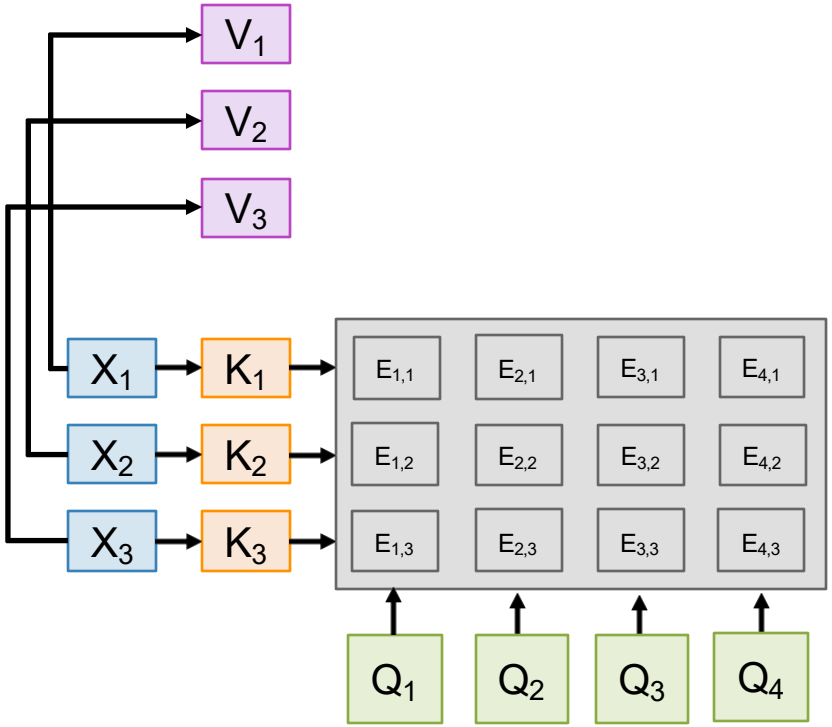
Similarities: $E = QK^T / \sqrt{D_Q}$ $[N_Q \times N_X]$

$E_{ij} = Q_i \cdot K_j / \sqrt{D_Q}$

Attention weights: $A = \text{softmax}(E, \text{dim}=1)$ $[N_Q \times N_X]$

Output vector: $Y = AV$ $[N_Q \times D_V]$

$Y_i = \sum_j A_{ij} V_j$



Attention Layer

Inputs:
Query vector: Q [N_Q x D_Q]
Data vectors: X [N_X x D_X]
Key matrix: W_K [D_X x D_Q]
Value matrix: W_V [D_X x D_V]

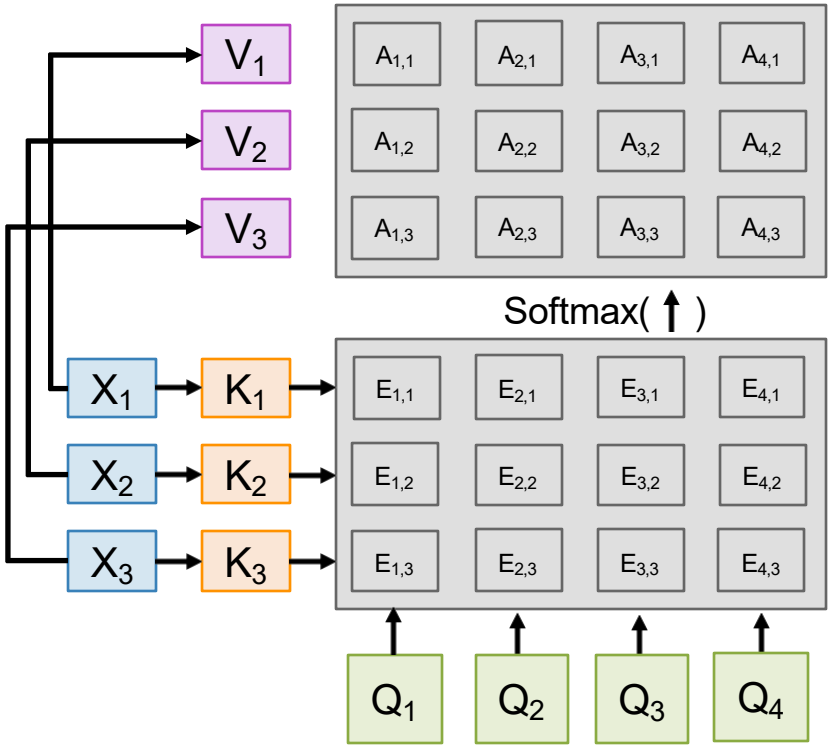
Computation:
Keys: $K = XW_K$ [N_X x D_Q]
Values: $V = XW_V$ [N_X x D_V]
Similarities: $E = QK^T / \sqrt{D_Q}$ [N_Q x N_X]

$$E_{ij} = Q_i \cdot K_j / \sqrt{D_Q}$$

Attention weights: A = softmax(E, dim=1) [N_Q x N_X]
Output vector: Y = AV [N_Q x D_V]

$$Y_i = \sum_j A_{ij} V_j$$

Softmax normalizes each column: each **query** predicts a distribution over the **keys**



Attention Layer

Inputs:

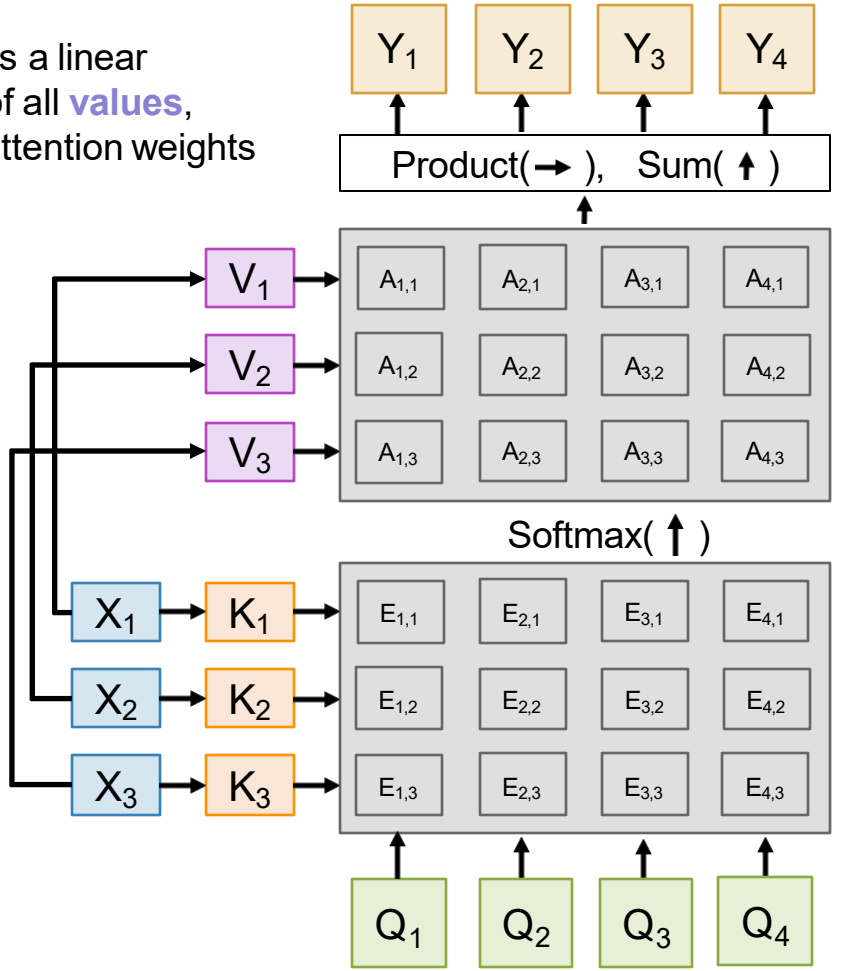
Query vector: Q [N_Q x D_Q]

Data vectors: X [N_X x D_X]

Key matrix: W_K [D_X x D_Q]

Value matrix: W_V [D_X x D_V]

Each **output** is a linear combination of all **values**, weighted by attention weights



Computation:

Keys: $K = XW_K$ [N_X x D_Q]

Values: $V = XW_V$ [N_X x D_V]

Similarities: $E = QK^T / \sqrt{D_Q}$ [N_Q x N_X]

$E_{ij} = Q_i \cdot K_j / \sqrt{D_Q}$

Attention weights: A = softmax(E, dim=1) [N_Q x N_X]

Output vector: $Y = AV$ [N_Q x D_V]

$Y_i = \sum_j A_{ij} V_j$

Cross-Attention Layer

Inputs:

Query vector: Q [N_Q x D_Q]

Data vectors: X [N_X x D_X]

Key matrix: W_K [D_X x D_Q]

Value matrix: W_V [D_X x D_V]

Each **query** produces one **output**, which is a mix of information in the **data** vectors

Computation:

Keys: $K = XW_K$ [N_X x D_Q]

Values: $V = XW_V$ [N_X x D_V]

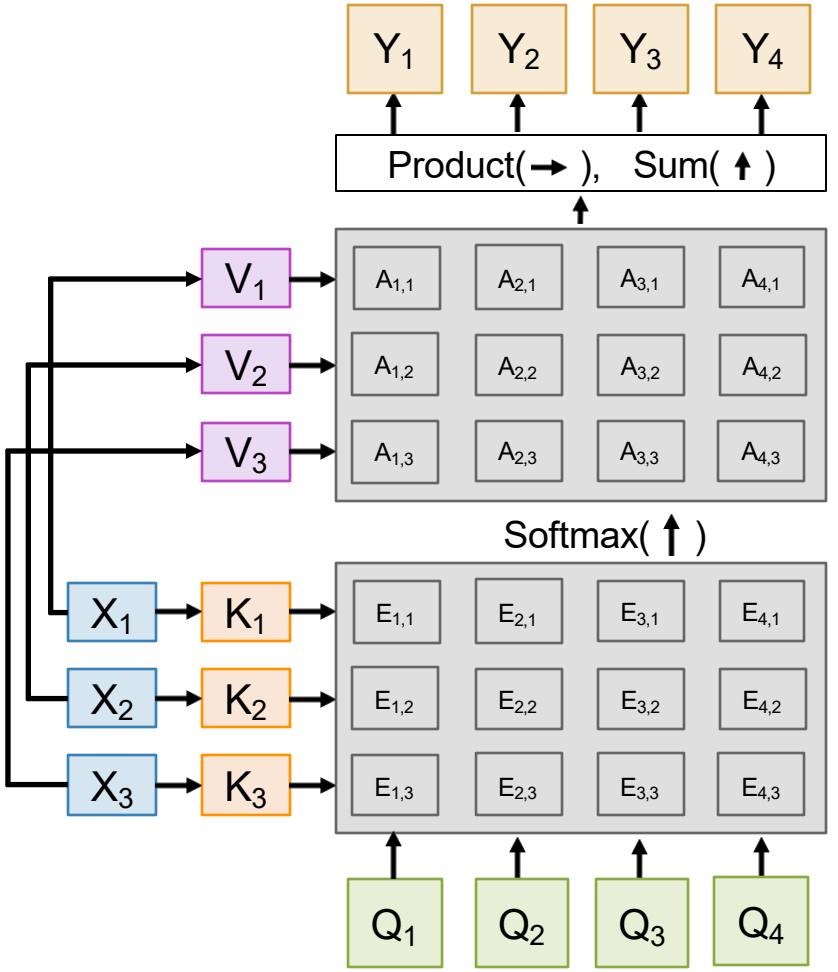
Similarities: $E = QK^T / \sqrt{D_Q}$ [N_Q x N_X]

$E_{ij} = Q_i \cdot K_j / \sqrt{D_Q}$

Attention weights: A = softmax(E, dim=1) [N_Q x N_X]

Output vector: $Y = AV$ [N_Q x D_V]

$Y_i = \sum_j A_{ij} V_j$



Self-Attention Layer

Inputs:

Input vectors: X [N x D_{in}]

Key matrix: W_K [D_{in} x D_{out}]

Value matrix: W_V [D_{in} x D_{out}]

Query matrix: W_Q [D_{in} x D_{out}]

Each **input** produces one **output**, which is a mix of information from all **inputs**

Shapes get a little simpler:

- N input vectors, each D_{in}
- Almost always $D_Q = D_V = D_{out}$

Computation:

Queries: $Q = XW_Q$ [N x D_{out}]

Keys: $K = XW_K$ [N x D_{out}]

Values: $V = XW_V$ [N x D_{out}]

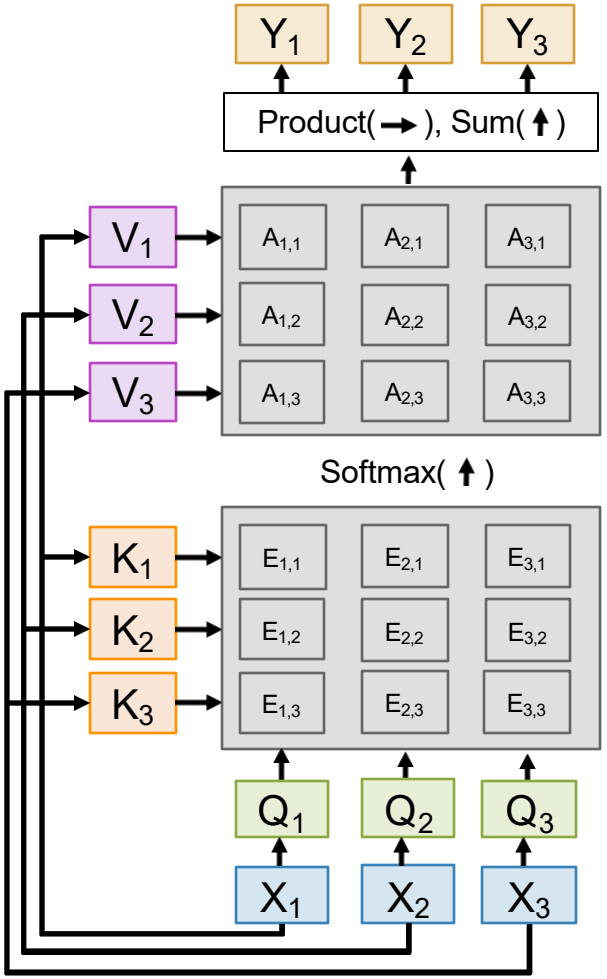
Similarities: $E = QK^T / \sqrt{D_Q}$ [N x N]

$E_{ij} = Q_i \cdot K_j / \sqrt{D_Q}$

Attention weights: A = softmax(E, dim=1) [N x N]

Output vector: $Y = AV$ [N x D_{out}]

$Y_i = \sum_j A_{ij} V_j$

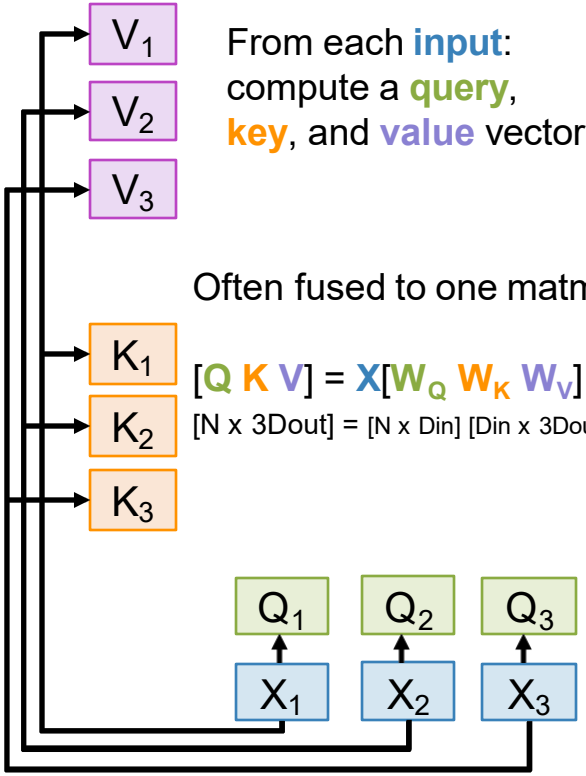


Self-Attention Layer

Inputs:
Input vectors: X [N x D_{in}]
Key matrix: W_K [D_{in} x D_{out}]
Value matrix: W_V [D_{in} x D_{out}]
Query matrix: W_Q [D_{in} x D_{out}]

Each **input** produces one **output**, which is a mix of information from all **inputs**

Computation:
Queries: $Q = XW_Q$ [N x D_{out}]
Keys: $K = XW_K$ [N x D_{out}]
Values: $V = XW_V$ [N x D_{out}]
Similarities: $E = QK^T / D_Q$ [N x N]
$E_{ij} = Q_i \cdot K_j / D_Q$
Attention weights: A = softmax(E, dim=1) [N x N]
Output vector: $Y = AV$ [N x D_{out}]
$Y_i = \sum_j A_{ij} V_j$



From each **input**: compute a **query**, **key**, and **value** vector

Often fused to one matmul:

[Q K V] = X[W_Q W_K W_V]
[N x 3Dout] = [N x Din] [Din x 3Dout]

Self-Attention Layer

Inputs:
Input vectors: X [N x D_{in}]
Key matrix: W_K [D_{in} x D_{out}]
Value matrix: W_V [D_{in} x D_{out}]
Query matrix: W_Q [D_{in} x D_{out}]

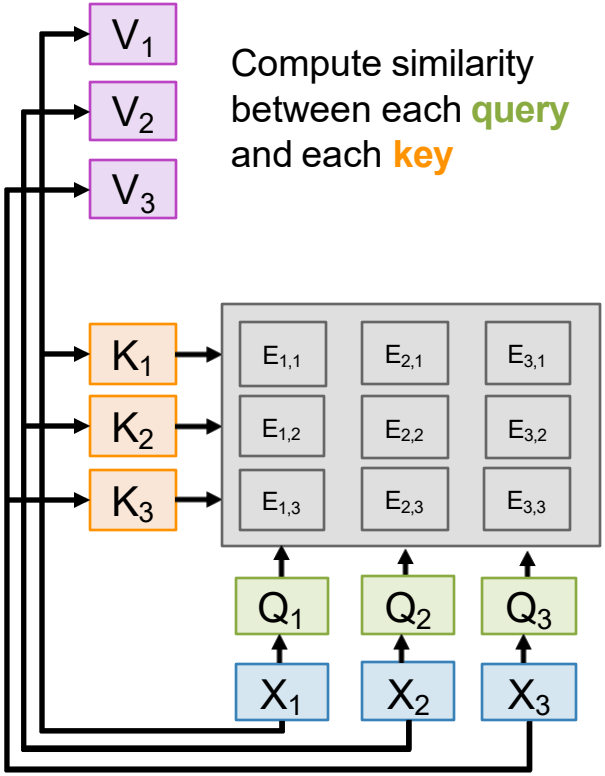
Each **input** produces one **output**, which is a mix of information from all **inputs**

Computation:
Queries: $Q = XW_Q$ [N x D_{out}]
Keys: $K = XW_K$ [N x D_{out}]
Values: $V = XW_V$ [N x D_{out}]
Similarities: $E = QK^T / \sqrt{D_Q}$ [N x N]

$E_{ij} = Q_i \cdot K_j / \sqrt{D_Q}$

Attention weights: A = softmax(E, dim=1) [N x N]
Output vector: Y = AV [N x D_{out}]

$Y_i = \sum_j A_{ij} V_j$



Compute similarity between each **query** and each **key**

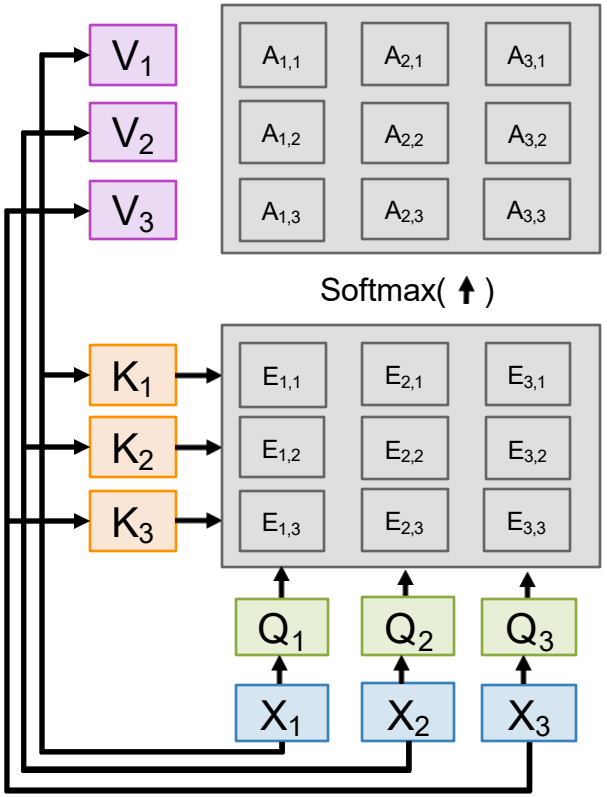
Self-Attention Layer

Inputs:
Input vectors: X [N x D_{in}]
Key matrix: W_K [D_{in} x D_{out}]
Value matrix: W_V [D_{in} x D_{out}]
Query matrix: W_Q [D_{in} x D_{out}]

Each **input** produces one **output**, which is a mix of information from all **inputs**

Computation:
Queries: $Q = XW_Q$ [N x D_{out}]
Keys: $K = XW_K$ [N x D_{out}]
Values: $V = XW_V$ [N x D_{out}]
Similarities: $E = QK^T / \sqrt{D_Q}$ [N x N]
$E_{ij} = Q_i \cdot K_j / \sqrt{D_Q}$
Attention weights: A = softmax(E, dim=1) [N x N]
Output vector: $Y = AV$ [N x D_{out}]
$Y_i = \sum_j A_{ij} V_j$

Normalize over each column: each **query** computes a distribution over **keys**



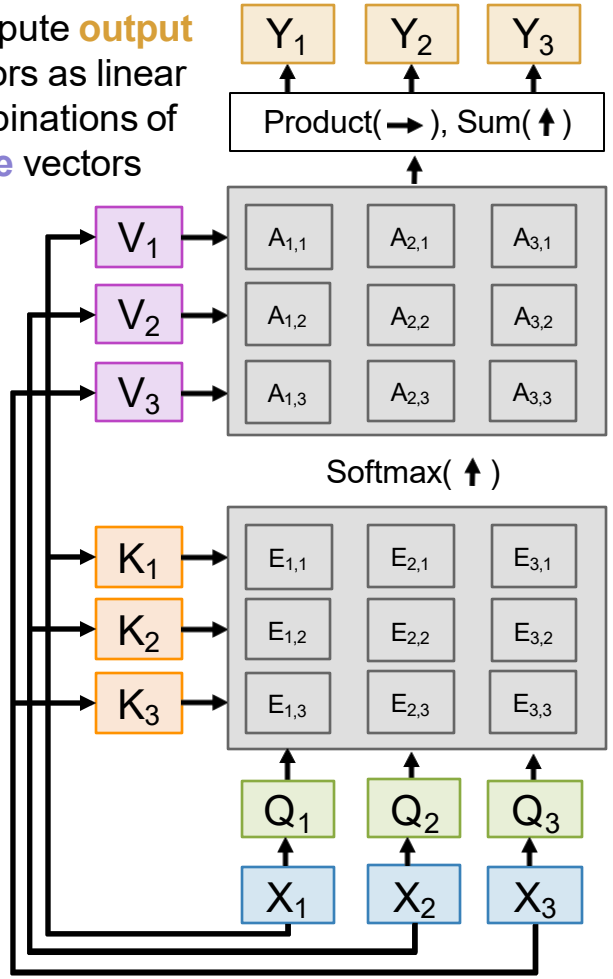
Self-Attention Layer

Inputs:
Input vectors: X [N x D_{in}]
Key matrix: W_K [D_{in} x D_{out}]
Value matrix: W_V [D_{in} x D_{out}]
Query matrix: W_Q [D_{in} x D_{out}]

Each **input** produces one **output**, which is a mix of information from all **inputs**

Computation:
Queries: $Q = XW_Q$ [N x D_{out}]
Keys: $K = XW_K$ [N x D_{out}]
Values: $V = XW_V$ [N x D_{out}]
Similarities: $E = QK^T / \sqrt{D_Q}$ [N x N]
$E_{ij} = Q_i \cdot K_j / \sqrt{D_Q}$
Attention weights: A = softmax(E, dim=1) [N x N]
Output vector: $Y = AV$ [N x D_{out}]
$Y_i = \sum_j A_{ij} V_j$

Compute **output** vectors as linear combinations of **value** vectors

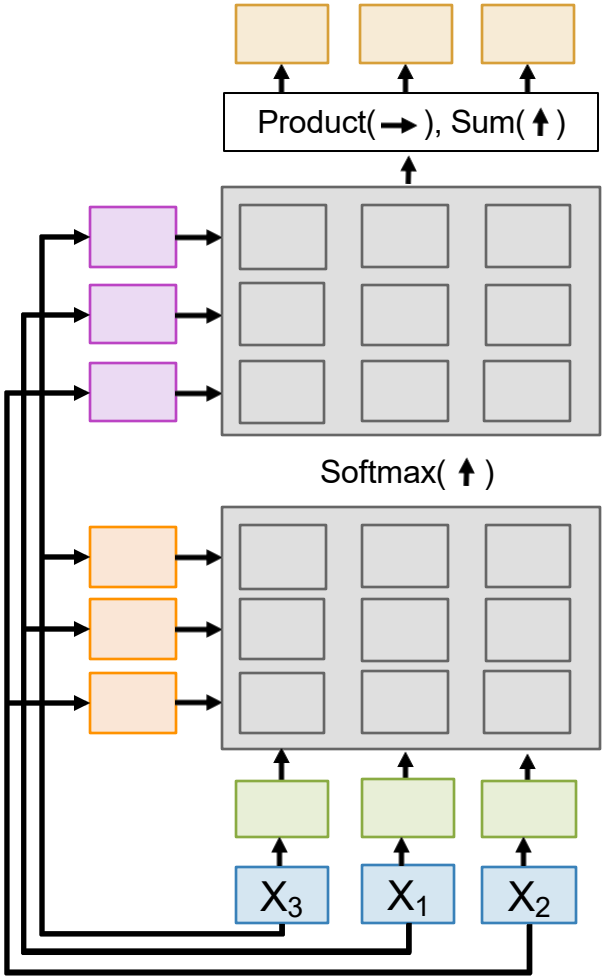


Self-Attention Layer

Consider permuting **inputs**:

Inputs:
Input vectors: X [N x D_{in}]
Key matrix: W_K [D_{in} x D_{out}]
Value matrix: W_V [D_{in} x D_{out}]
Query matrix: W_Q [D_{in} x D_{out}]

Computation:
Queries: $Q = XW_Q$ [N x D_{out}]
Keys: $K = XW_K$ [N x D_{out}]
Values: $V = XW_V$ [N x D_{out}]
Similarities: $E = QK^T / \sqrt{D_Q}$ [N x N]
$E_{ij} = Q_i \cdot K_j / \sqrt{D_Q}$
Attention weights: A = softmax(E, dim=1) [N x N]
Output vector: $Y = AV$ [N x D_{out}]
$Y_i = \sum_j A_{ij} V_j$



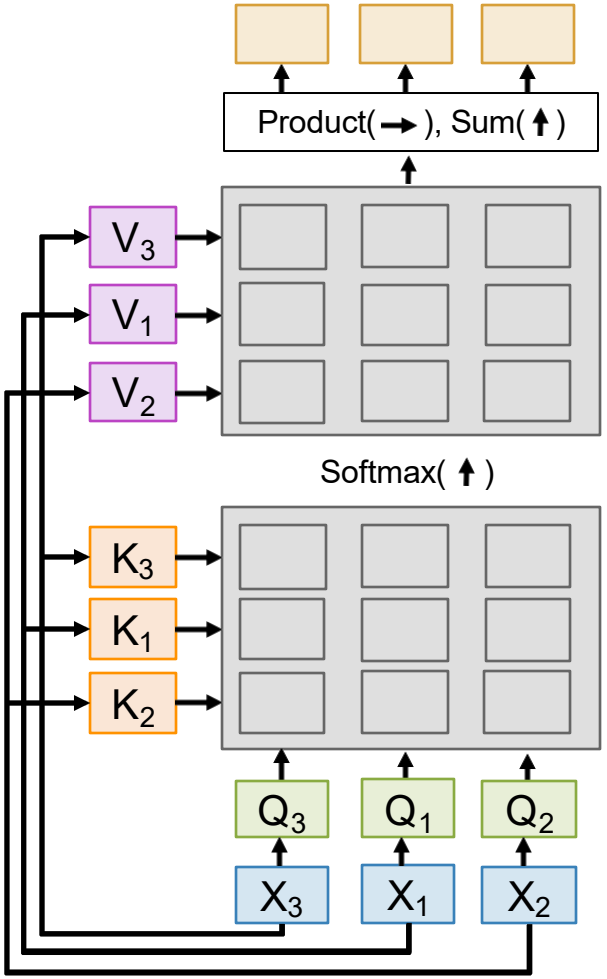
Self-Attention Layer

Inputs:
Input vectors: X [N x D_{in}]
Key matrix: W_K [D_{in} x D_{out}]
Value matrix: W_V [D_{in} x D_{out}]
Query matrix: W_Q [D_{in} x D_{out}]

Computation:
Queries: $Q = XW_Q$ [N x D_{out}]
Keys: $K = XW_K$ [N x D_{out}]
Values: $V = XW_V$ [N x D_{out}]
Similarities: $E = QK^T / \sqrt{D_Q}$ [N x N]
$E_{ij} = Q_i \cdot K_j / \sqrt{D_Q}$
Attention weights: A = softmax(E, dim=1) [N x N]
Output vector: $Y = AV$ [N x D_{out}]
$Y_i = \sum_j A_{ij} V_j$

Consider permuting **inputs**:

Queries, **keys**, and **values** will be the same but permuted



Self-Attention Layer

Inputs:
Input vectors: X [N x D_{in}]
Key matrix: W_K [D_{in} x D_{out}]
Value matrix: W_V [D_{in} x D_{out}]
Query matrix: W_Q [D_{in} x D_{out}]

Computation:
Queries: $Q = XW_Q$ [N x D_{out}]
Keys: $K = XW_K$ [N x D_{out}]
Values: $V = XW_V$ [N x D_{out}]
Similarities: $E = QK^T / \sqrt{D_Q}$ [N x N]

$$E_{ij} = Q_i \cdot K_j / \sqrt{D_Q}$$

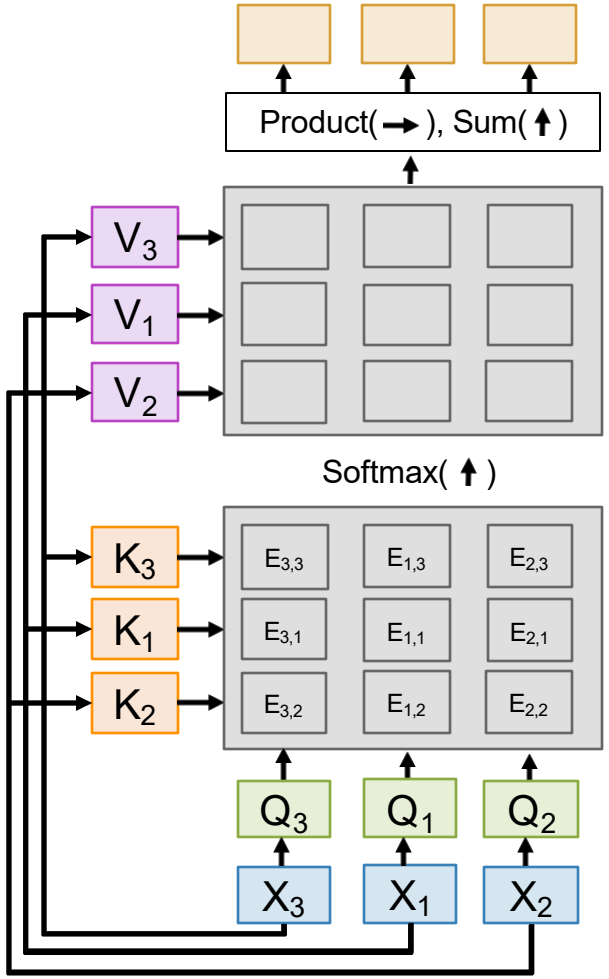
Attention weights: A = softmax(E, dim=1) [N x N]
Output vector: $Y = AV$ [N x D_{out}]

$$Y_i = \sum_j A_{ij} V_j$$

Consider permuting **inputs**:

Queries, **keys**, and **values** will be the same but permuted

Similarities are the same but permuted



Self-Attention Layer

Inputs:
Input vectors: X [N x D_{in}]
Key matrix: W_K [D_{in} x D_{out}]
Value matrix: W_V [D_{in} x D_{out}]
Query matrix: W_Q [D_{in} x D_{out}]

Computation:
Queries: $Q = XW_Q$ [N x D_{out}]
Keys: $K = XW_K$ [N x D_{out}]
Values: $V = XW_V$ [N x D_{out}]
Similarities: $E = QK^T / \sqrt{D_Q}$ [N x N]

$$E_{ij} = Q_i \cdot K_j / \sqrt{D_Q}$$

Attention weights: A = softmax(E, dim=1) [N x N]
Output vector: $Y = AV$ [N x D_{out}]

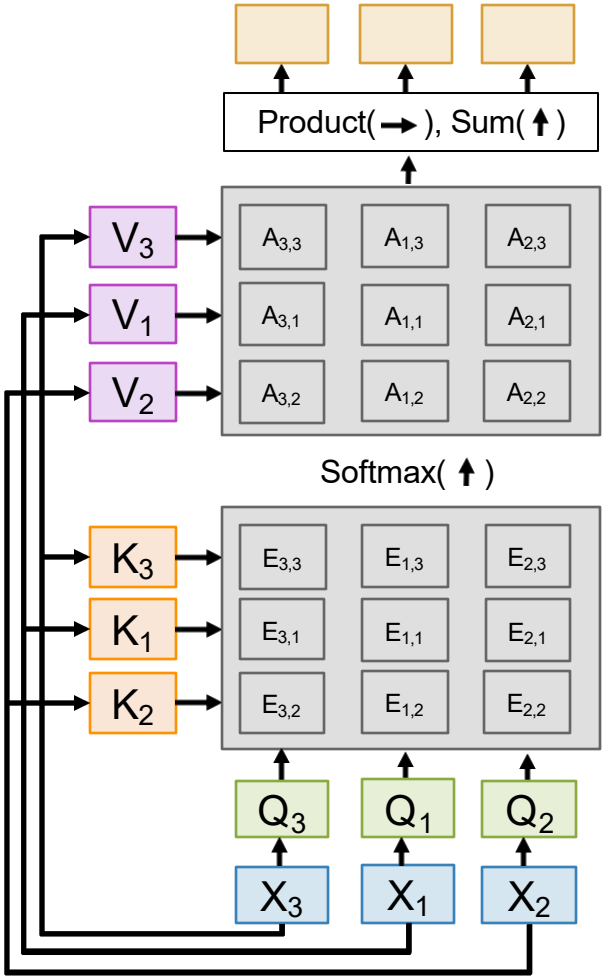
$$Y_i = \sum_j A_{ij} V_j$$

Consider permuting **inputs**:

Queries, **keys**, and **values** will be the same but permuted

Similarities are the same but permuted

Attention weights are the same but permuted



Self-Attention Layer

Inputs:
Input vectors: X [N x D_{in}]
Key matrix: W_K [D_{in} x D_{out}]
Value matrix: W_V [D_{in} x D_{out}]
Query matrix: W_Q [D_{in} x D_{out}]

Computation:
Queries: $Q = XW_Q$ [N x D_{out}]
Keys: $K = XW_K$ [N x D_{out}]
Values: $V = XW_V$ [N x D_{out}]
Similarities: $E = QK^T / \sqrt{D_Q}$ [N x N]

$$E_{ij} = Q_i \cdot K_j / \sqrt{D_Q}$$

Attention weights: A = softmax(E, dim=1) [N x N]
Output vector: $Y = AV$ [N x D_{out}]

$$Y_i = \sum_j A_{ij} V_j$$

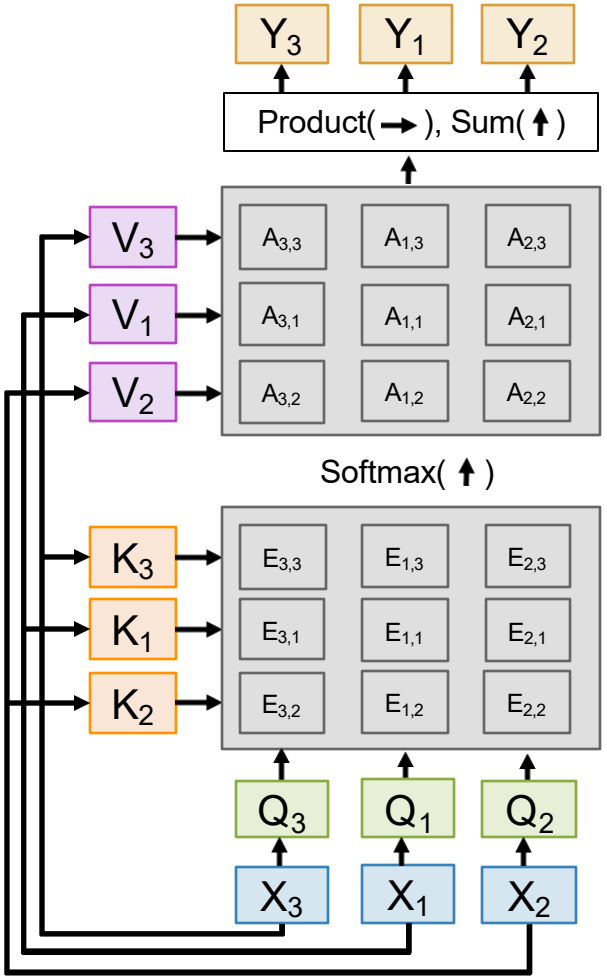
Consider permuting **inputs**:

Queries, **keys**, and **values** will be the same but permuted

Similarities are the same but permuted

Attention weights are the same but permuted

Outputs are the same but permuted



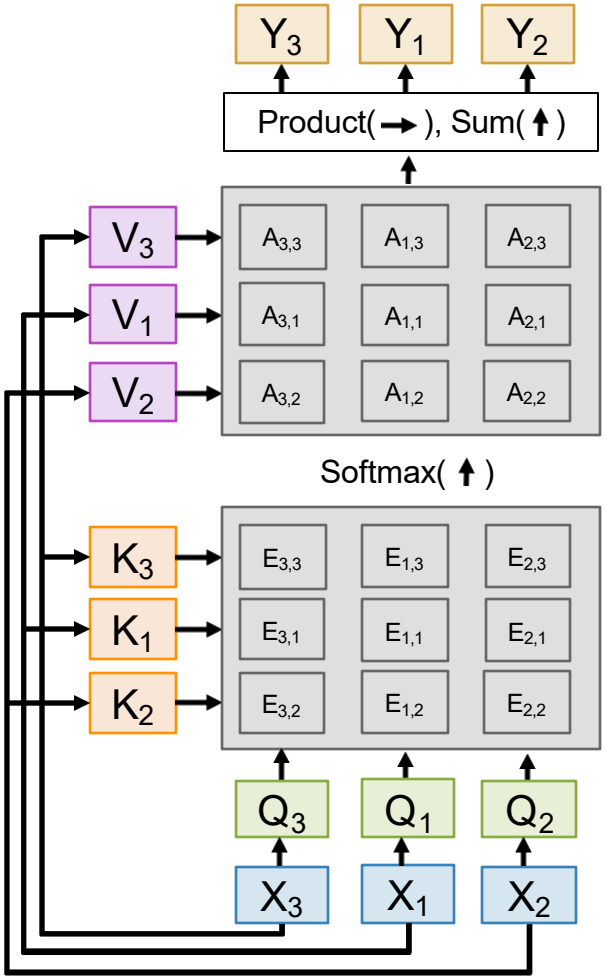
Self-Attention Layer

Inputs:
Input vectors: X [N x D_{in}]
Key matrix: W_K [D_{in} x D_{out}]
Value matrix: W_V [D_{in} x D_{out}]
Query matrix: W_Q [D_{in} x D_{out}]

Self-Attention is permutation equivariant:
$F(\sigma(X)) = \sigma(F(X))$

Computation:
Queries: $Q = XW_Q$ [N x D_{out}]
Keys: $K = XW_K$ [N x D_{out}]
Values: $V = XW_V$ [N x D_{out}]
Similarities: $E = QK^T / \sqrt{D_Q}$ [N x N]
$E_{ij} = Q_i \cdot K_j / \sqrt{D_Q}$
Attention weights: A = softmax(E, dim=1) [N x N]
Output vector: $Y = AV$ [N x D_{out}]
$Y_i = \sum_j A_{ij} V_j$

This means that Self-Attention works on **sets of vectors**

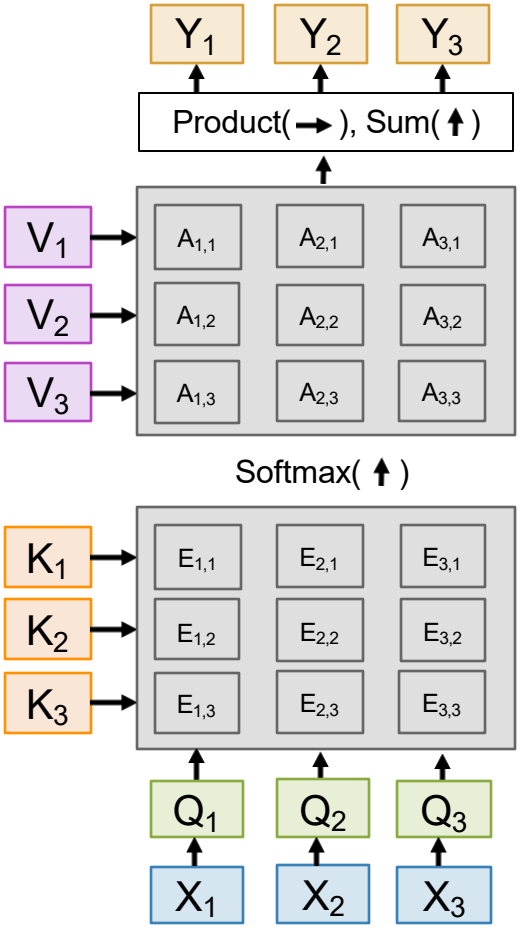


Self-Attention Layer

Inputs:
Input vectors: X [N x D_{in}]
Key matrix: W_K [D_{in} x D_{out}]
Value matrix: W_V [D_{in} x D_{out}]
Query matrix: W_Q [D_{in} x D_{out}]

Problem: Self-Attention does not know the order of the sequence

Computation:
Queries: $Q = XW_Q$ [N x D_{out}]
Keys: $K = XW_K$ [N x D_{out}]
Values: $V = XW_V$ [N x D_{out}]
Similarities: $E = QK^T / \sqrt{D_Q}$ [N x N]
$E_{ij} = Q_i \cdot K_j / \sqrt{D_Q}$
Attention weights: A = softmax(E, dim=1) [N x N]
Output vector: $Y = AV$ [N x D_{out}]
$Y_i = \sum_j A_{ij} V_j$



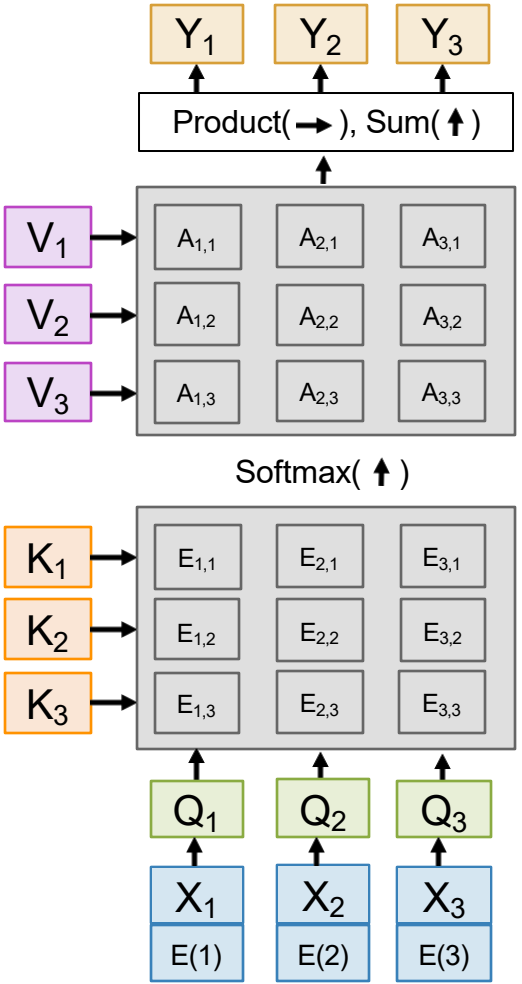
Self-Attention Layer

Inputs:
Input vectors: X [N x D_{in}]
Key matrix: W_K [D_{in} x D_{out}]
Value matrix: W_V [D_{in} x D_{out}]
Query matrix: W_Q [D_{in} x D_{out}]

Computation:
Queries: $Q = XW_Q$ [N x D_{out}]
Keys: $K = XW_K$ [N x D_{out}]
Values: $V = XW_V$ [N x D_{out}]
Similarities: $E = QK^T / \sqrt{D_Q}$ [N x N]
$E_{ij} = Q_i \cdot K_j / \sqrt{D_Q}$
Attention weights: A = softmax(E, dim=1) [N x N]
Output vector: $Y = AV$ [N x D_{out}]
$Y_i = \sum_j A_{ij} V_j$

Problem: Self-Attention does not know the order of the sequence

Solution: Add positional encoding to each input; this is a vector that is a fixed function of the index



Masked Self-Attention Layer

Don't let vectors "look ahead" in the sequence

Inputs:

Input vectors: X [N x D_{in}]

Key matrix: W_K [D_{in} x D_{out}]

Value matrix: W_V [D_{in} x D_{out}]

Query matrix: W_Q [D_{in} x D_{out}]

Override similarities with -inf; this controls which inputs each vector is allowed to look at.

Computation:

Queries: $Q = XW_Q$ [N x D_{out}]

Keys: $K = XW_K$ [N x D_{out}]

Values: $V = XW_V$ [N x D_{out}]

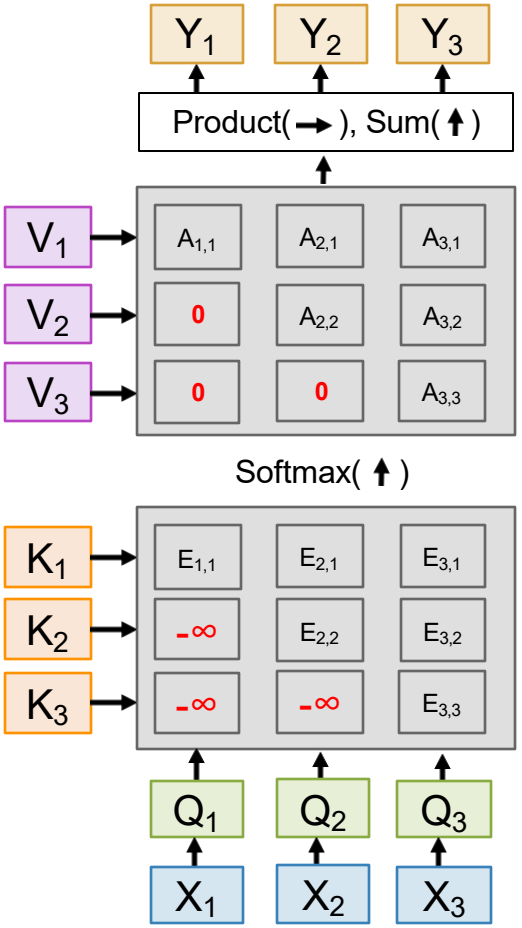
Similarities: $E = QK^T / \sqrt{D_Q}$ [N x N]

$E_{ij} = Q_i \cdot K_j / \sqrt{D_Q}$

Attention weights: A = softmax(E, dim=1) [N x N]

Output vector: $Y = AV$ [N x D_{out}]

$Y_i = \sum_j A_{ij} V_j$



Masked Self-Attention Layer

Don't let vectors "look ahead" in the sequence

Inputs:

Input vectors: X [N x D_{in}]

Key matrix: W_K [D_{in} x D_{out}]

Value matrix: W_V [D_{in} x D_{out}]

Query matrix: W_Q [D_{in} x D_{out}]

Computation:

Queries: $Q = XW_Q$ [N x D_{out}]

Keys: $K = XW_K$ [N x D_{out}]

Values: $V = XW_V$ [N x D_{out}]

Similarities: $E = QK^T / \sqrt{D_Q}$ [N x N]

$E_{ij} = Q_i \cdot K_j / \sqrt{D_Q}$

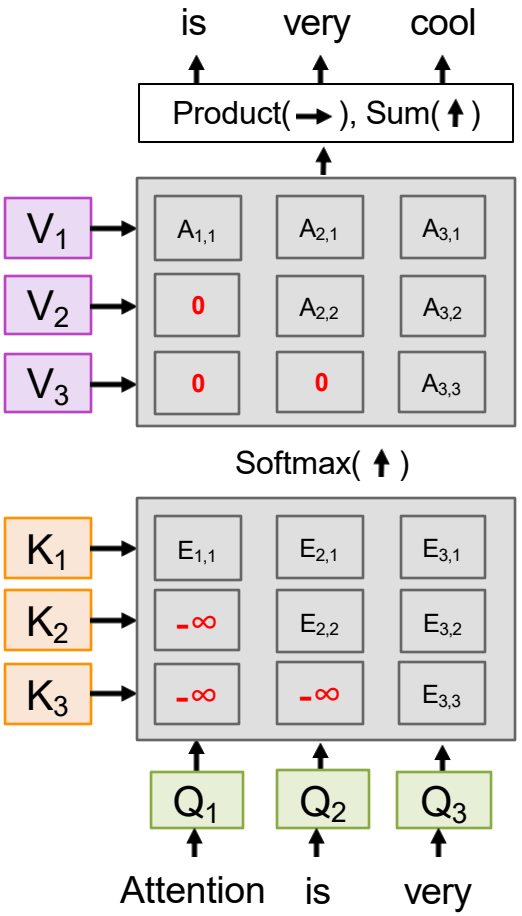
Attention weights: A = softmax(E, dim=1) [N x N]

Output vector: $Y = AV$ [N x D_{out}]

$Y_i = \sum_j A_{ij} V_j$

Override similarities with -inf; this controls which inputs each vector is allowed to look at.

Used for language modeling where you want to predict the next word



Multiheaded Self-Attention Layer

Run H copies of Self-Attention in parallel

Inputs:

Input vectors: X [N x D_{in}]

Key matrix: W_K [D_{in} x D_{out}]

Value matrix: W_V [D_{in} x D_{out}]

Query matrix: W_Q [D_{in} x D_{out}]

Computation:

Queries: $Q = XW_Q$ [N x D_{out}]

Keys: $K = XW_K$ [N x D_{out}]

Values: $V = XW_V$ [N x D_{out}]

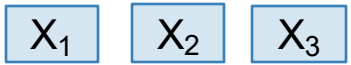
Similarities: $E = QK^T / \sqrt{D_Q}$ [N x N]

$$E_{ij} = Q_i \cdot K_j / \sqrt{D_Q}$$

Attention weights: A = softmax(E, dim=1) [N x N]

Output vector: $Y = AX$ [N x D_{out}]

$$Y_i = \sum_j A_{ij} V_j$$



Multiheaded Self-Attention Layer

Run H copies of Self-Attention in parallel

Inputs:

Input vectors: X [N x D_{in}]

Key matrix: W_K [D_{in} x D_{out}]

Value matrix: W_V [D_{in} x D_{out}]

Query matrix: W_Q [D_{in} x D_{out}]

Computation:

Queries: $Q = XW_Q$ [N x D_{out}]

Keys: $K = XW_K$ [N x D_{out}]

Values: $V = XW_V$ [N x D_{out}]

Similarities: $E = QK^T / \sqrt{D_Q}$ [N x N]

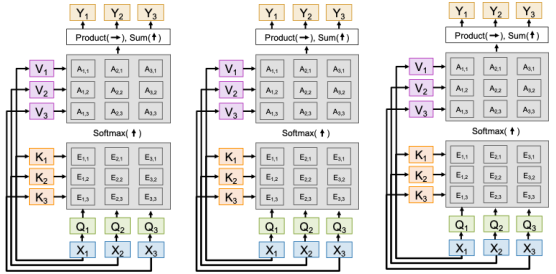
$E_{ij} = Q_i \cdot K_j / \sqrt{D_Q}$

Attention weights: A = softmax(E, dim=1) [N x N]

Output vector: $Y = AX$ [N x D_{out}]

$Y_i = \sum_j A_{ij} V_j$

H = 3 independent self-attention layers (called heads), each with their own weights



X_1 X_2 X_3

Multiheaded Self-Attention Layer

Run H copies of Self-Attention in parallel

Inputs:

Input vectors: X [N x D_{in}]

Key matrix: W_K [D_{in} x D_{out}]

Value matrix: W_V [D_{in} x D_{out}]

Query matrix: W_Q [D_{in} x D_{out}]

Computation:

Queries: $Q = XW_Q$ [N x D_{out}]

Keys: $K = XW_K$ [N x D_{out}]

Values: $V = XW_V$ [N x D_{out}]

Similarities: $E = QK^T / \sqrt{D_Q}$ [N x N]

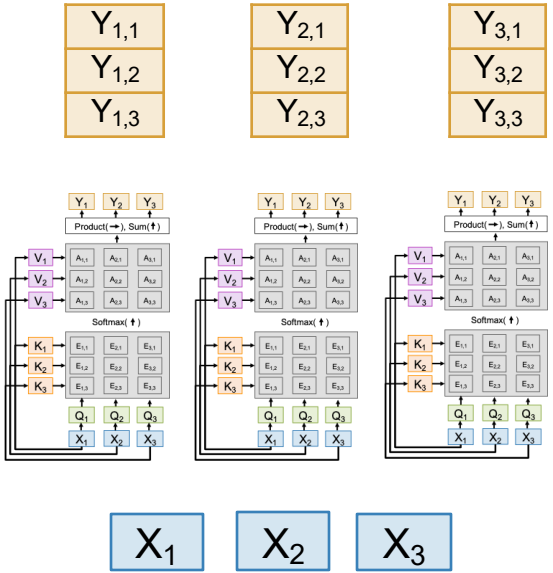
$E_{ij} = Q_i \cdot K_j / \sqrt{D_Q}$

Attention weights: A = softmax(E, dim=1) [N x N]

Output vector: $Y = AX$ [N x D_{out}]

$Y_i = \sum_j A_{ij} V_j$

Stack up the H independent outputs for each input X



H = 3 independent self-attention layers (called heads), each with their own weights

Multiheaded Self-Attention Layer

Run H copies of Self-Attention in parallel

Inputs:

Input vectors: X [N x D_{in}]

Key matrix: W_K [D_{in} x D_{out}]

Value matrix: W_V [D_{in} x D_{out}]

Query matrix: W_Q [D_{in} x D_{out}]

Computation:

Queries: $Q = XW_Q$ [N x D_{out}]

Keys: $K = XW_K$ [N x D_{out}]

Values: $V = XW_V$ [N x D_{out}]

Similarities: $E = QK^T / D_Q$ [N x N]

$E_{ij} = Q_i \cdot K_j / D_Q$

Attention weights: A = softmax(E, dim=1) [N x N]

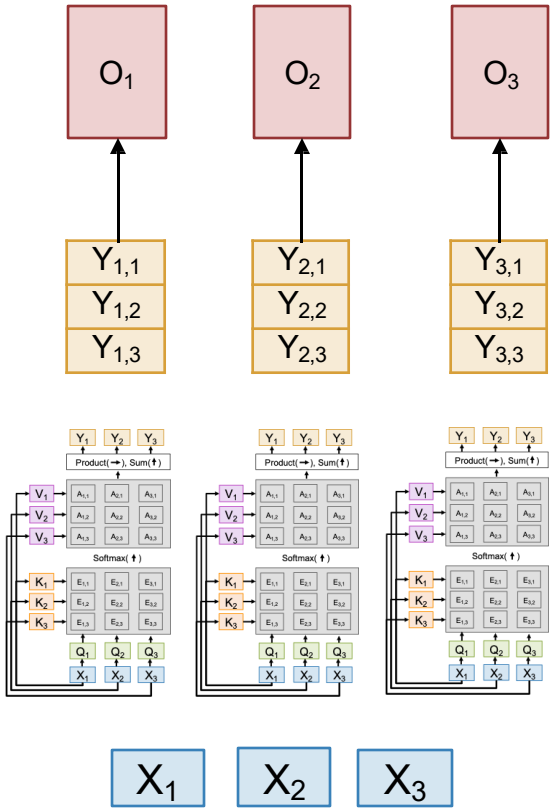
Output vector: $Y = AX$ [N x D_{out}]

$Y_i = \sum_j A_{ij} V_j$

Output projection fuses data from each head

Stack up the H independent outputs for each input X

H = 3 independent self-attention layers (called heads), each with their own weights



Multiheaded Self-Attention Layer

Run H copies of Self-Attention in parallel

Inputs:

Input vectors: X [N x D]

Key matrix: W_K [D x HD_H]

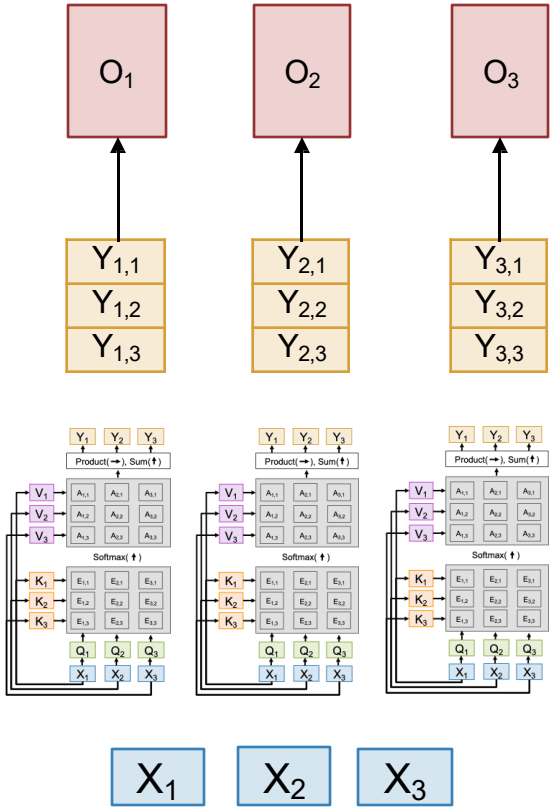
Value matrix: W_V [D x HD_H]

Query matrix: W_Q [D x HD_H]

Output matrix: W_O [HD_H x D]

Each of the H parallel layers use a qkv dim of D_H = "head dim"

Usually D_H = D / H, so inputs and outputs have the same dimension



Computation:

Queries: $Q = XW_Q$ [H x N x D_H]

Keys: $K = XW_K$ [H x N x D_H]

Values: $V = XW_V$ [H x N x D_H]

Similarities: $E = QK^T / \sqrt{D_Q}$ [H x N x N]

Attention weights: A = softmax(E, dim=2) [H x N x N]

Head outputs: $Y = AV$ [H x N x D_H] => [N x HD_H]

Outputs: $O = YW_O$ [N x D]

Multiheaded Self-Attention Layer

Run H copies of Self-Attention in parallel

Inputs:

Input vectors: X [N x D]

Key matrix: W_K [D x HD_H]

Value matrix: W_V [D x HD_H]

Query matrix: W_Q [D x HD_H]

Output matrix: W_O [HD_H x D]

In practice, compute all H heads in parallel using batched matrix multiply operations.

Computation:

Queries: $Q = XW_Q$ [H x N x D_H]

Keys: $K = XW_K$ [H x N x D_H]

Values: $V = XW_V$ [H x N x D_H]

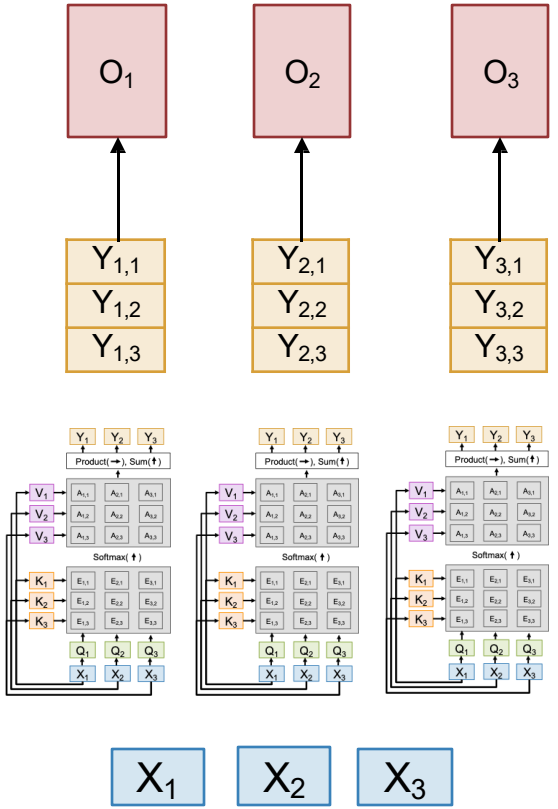
Used everywhere in practice.

Similarities: $E = QK^T / \sqrt{D_Q}$ [H x N x N]

Attention weights: A = softmax(E, dim=2) [H x N x N]

Head outputs: Y = AV [H x N x D_H] => [N x HD_H]

Outputs: $O = YW_O$ [N x D]



Self-Attention is Four Matrix Multiplies!

Inputs:
Input vectors: X [N x D]
Key matrix: W_K [D x HD_H]
Value matrix: W_V [D x HD_H]
Query matrix: W_Q [D x HD_H]
Output matrix: W_O [HD_H x D]

Computation:
Queries: $Q = XW_Q$ [H x N x D_H]
Keys: $K = XW_K$ [H x N x D_H]
Values: $V = XW_V$ [H x N x D_H]
Similarities: $E = QK^T / \sqrt{D_Q}$ [H x N x N]
Attention weights: A = softmax(E, dim=2) [H x N x N]
Head outputs: $Y = AV$ [H x N x D_H] => [N x HD_H]
Outputs: $O = YW_O$ [N x D]

Self-Attention is Four Matrix Multiplies!

Inputs:
Input vectors: **X** [N x D]
Key matrix: W_K [D x HD_H]
Value matrix: W_V [D x HD_H]
Query matrix: W_Q [D x HD_H]
Output matrix: W_O [HD_H x D]

Computation:
Queries: **Q** = XW_Q [H x N x D_H]
Keys: **K** = XW_K [H x N x D_H]
Values: **V** = XW_V [H x N x D_H]
Similarities: **E** = $QK^T / \sqrt{D_Q}$ [H x N x N]
Attention weights: **A** = softmax(E, dim=2) [H x N x N]
Head outputs: **Y** = **AV** [H x N x D_H] => [N x HD_H]
Outputs: **O** = YW_O [N x D]

1. QKV Projection
   [N x D] [D x $3HD_H$] => [N x $3HD_H$]
   Split and reshape to get **Q**, **K**, **V** each of shape [H x N x D_H]

Self-Attention is Four Matrix Multiplies!

Inputs:
Input vectors: **X** [N x D]
Key matrix: W_K [D x HD_H]
Value matrix: W_V [D x HD_H]
Query matrix: W_Q [D x HD_H]
Output matrix: W_O [HD_H x D]

Computation:
Queries: **Q** = XW_Q [H x N x D_H]
Keys: **K** = XW_K [H x N x D_H]
Values: **V** = XW_V [H x N x D_H]
Similarities: **E** = $QK^T / \sqrt{D_Q}$ [H x N x N]
Attention weights: **A** = softmax(E, dim=2) [H x N x N]
Head outputs: **Y** = **AV** [H x N x D_H] => [N x HD_H]
Outputs: **O** = YW_O [N x D]

1. QKV Projection
   [N x D] [D x $3HD_H$] => [N x $3HD_H$]
   Split and reshape to get **Q**, **K**, **V** each of shape [H x N x D_H]
2. QK Similarity
   [H x N x D_H] **[H x D_H x N]** => **[H x N x N]**

Self-Attention is Four Matrix Multiplies!

Inputs:
Input vectors: X [N x D]
Key matrix: W_K [D x HD_H]
Value matrix: W_V [D x HD_H]
Query matrix: W_Q [D x HD_H]
Output matrix: W_O [HD_H x D]

Computation:
Queries: $Q = XW_Q$ [H x N x D_H]
Keys: $K = XW_K$ [H x N x D_H]
Values: $V = XW_V$ [H x N x D_H]
Similarities: $E = QK^T / \sqrt{D_Q}$ [H x N x N]
Attention weights: A = softmax(E, dim=2) [H x N x N]
Head outputs: $Y = AV$ [H x N x D_H] => [N x HD_H]
Outputs: $O = YW_O$ [N x D]

1. QKV Projection
   [N x D] [D x $3HD_H$] => [N x $3HD_H$]
   Split and reshape to get Q, K, V each of shape [H x N x D_H]
2. QK Similarity
   [H x N x D_H] [H x D_H x N] => [H x N x N]
3. V-Weighting
   [H x N x N] [H x N x D_H] => [H x N x D_H]
   Reshape to [N x HD_H]

Self-Attention is Four Matrix Multiplies!

Inputs:
Input vectors: X [N x D]
Key matrix: W_K [D x HD_H]
Value matrix: W_V [D x HD_H]
Query matrix: W_Q [D x HD_H]
Output matrix: W_O [HD_H x D]

Computation:
Queries: $Q = XW_Q$ [H x N x D_H]
Keys: $K = XW_K$ [H x N x D_H]
Values: $V = XW_V$ [H x N x D_H]
Similarities: $E = QK^T / \sqrt{D_Q}$ [H x N x N]
Attention weights: A = softmax(E, dim=2) [H x N x N]
Head outputs: $Y = AV$ [H x N x D_H] => [N x HD_H]
Outputs: $O = YW_O$ [N x D]

1. QKV Projection
   [N x D] [D x $3HD_H$] => [N x $3HD_H$]
   Split and reshape to get Q, K, V each of shape [H x N x D_H]
2. QK Similarity
   [H x N x D_H] [H x D_H x N] => [H x N x N]
3. V-Weighting
   [H x N x N] [H x N x D_H] => [H x N x D_H]
   Reshape to [N x HD_H]
4. Output Projection
   [N x HD_H] [HD_H x D] => [N x D]

Self-Attention is Four Matrix Multiplies!

Inputs:
Input vectors: **X** [N x D]
Key matrix: W_K [D x HD_H]
Value matrix: W_V [D x HD_H]
Query matrix: W_Q [D x HD_H]
Output matrix: W_O [HD_H x D]

Computation:
Queries: **Q** = XW_Q [H x N x D_H]
Keys: **K** = XW_K [H x N x D_H]
Values: **V** = XW_V [H x N x D_H]
Similarities: **E** = $QK^T / \sqrt{D_Q}$ [H x N x N]
Attention weights: **A** = softmax(E, dim=2) [H x N x N]
Head outputs: **Y** = **AV** [H x N x D_H] => [N x HD_H]
Outputs: **O** = YW_O [N x D]

1. QKV Projection
   [N x D] [D x $3HD_H$] => [N x $3HD_H$]
   Split and reshape to get **Q**, **K**, **V** each of shape [H x N x D_H]
2. QK Similarity
   [H x N x D_H] [H x D_H x N] => [H x N x N]
3. V-Weighting
   [H x N x N] [H x N x D_H] => [H x N x D_H]
   Reshape to [N x HD_H]
4. Output Projection
   [N x HD_H] [HD_H x D] => [N x D]

Q: How much compute does this take as the number of vectors N increases?

Self-Attention is Four Matrix Multiplies!

Inputs:
Input vectors: **X** [N x D]
Key matrix: W_K [D x HD_H]
Value matrix: W_V [D x HD_H]
Query matrix: W_Q [D x HD_H]
Output matrix: W_O [HD_H x D]

Computation:
Queries: $Q = XW_Q$ [H x N x D_H]
Keys: $K = XW_K$ [H x N x D_H]
Values: $V = XW_V$ [H x N x D_H]
Similarities: $E = QK^T / \sqrt{D_Q}$ [H x N x N]
Attention weights: A = softmax(E, dim=2) [H x N x N]
Head outputs: $Y = AV$ [H x N x D_H] => [N x HD_H]
Outputs: $O = YW_O$ [N x D]

1. QKV Projection
   [N x D] [D x $3HD_H$] => [N x $3HD_H$]
   Split and reshape to get Q, K, V each of shape [H x N x D_H]
2. QK Similarity
   [H x N x D_H] [H x D_H x N] => [H x N x N]
3. V-Weighting
   [H x N x N] [H x N x D_H] => [H x N x D_H]
   Reshape to [N x HD_H]
4. Output Projection
   [N x HD_H] [HD_H x D] => [N x D]

Q: How much compute does this take as the number of vectors N increases?
A: $O(N^2)$

Self-Attention is Four Matrix Multiplies!

Inputs:
Input vectors: **X** [N x D]
Key matrix: W_K [D x HD_H]
Value matrix: W_V [D x HD_H]
Query matrix: W_Q [D x HD_H]
Output matrix: W_O [HD_H x D]

Computation:
Queries: $Q = XW_Q$ [H x N x D_H]
Keys: $K = XW_K$ [H x N x D_H]
Values: $V = XW_V$ [H x N x D_H]
Similarities: $E = QK^T / \sqrt{D_Q}$ [H x N x N]
Attention weights: A = softmax(E, dim=2) [H x N x N]
Head outputs: Y = AV [H x N x D_H] => [N x HD_H]
Outputs: $O = YW_O$ [N x D]

1. QKV Projection
   [N x D] [D x $3HD_H$] => [N x $3HD_H$]
   Split and reshape to get Q, K, V each of shape [H x N x D_H]
2. QK Similarity
   [H x N x D_H] [H x D_H x N] => [H x N x N]
3. V-Weighting
   [H x N x N] [H x N x D_H] => [H x N x D_H]
   Reshape to [N x HD_H]
4. Output Projection
   [N x HD_H] [HD_H x D] => [N x D]

Q: How much memory does this take as the number of vectors N increases?

Self-Attention is Four Matrix Multiplies!

Inputs:
Input vectors: **X** [N x D]
Key matrix: W_K [D x HD_H]
Value matrix: W_V [D x HD_H]
Query matrix: W_Q [D x HD_H]
Output matrix: W_O [HD_H x D]

Computation:
Queries: $Q = XW_Q$ [H x N x D_H]
Keys: $K = XW_K$ [H x N x D_H]
Values: $V = XW_V$ [H x N x D_H]
Similarities: $E = QK^T / \sqrt{D_Q}$ [H x N x N]
Attention weights: A = softmax(E, dim=2) [H x N x N]
Head outputs: Y = AV [H x N x D_H] => [N x HD_H]
Outputs: $O = YW_O$ [N x D]

1. QKV Projection
   [N x D] [D x $3HD_H$] => [N x $3HD_H$]
   Split and reshape to get Q, K, V each of shape [H x N x D_H]
2. QK Similarity
   [H x N x D_H] [H x D_H x N] => [H x N x N]
3. V-Weighting
   [H x N x N] [H x N x D_H] => [H x N x D_H]
   Reshape to [N x HD_H]
4. Output Projection
   [N x HD_H] [HD_H x D] => [N x D]

Q: How much memory does this take as the number of vectors N increases?
A: $O(N^2)$

Self-Attention is Four Matrix Multiplies!

If N=100K, H=64 then HxNxN attention weights take 1.192 TB! GPUs don't have that much memory…

Inputs:
Input vectors: X [N x D]
Key matrix: W_K [D x HD_H]
Value matrix: W_V [D x HD_H]
Query matrix: W_Q [D x HD_H]
Output matrix: W_O [HD_H x D]

Computation:
Queries: Q = XW_Q [H x N x D_H]
Keys: K = XW_K [H x N x D_H]
Values: V = XW_V [H x N x D_H]
Similarities: E = $QK^T / \sqrt{D_Q}$ [H x N x N]
Attention weights: A = softmax(E, dim=2) [H x N x N]
Head outputs: Y = AV [H x N x D_H] => [N x HD_H]
Outputs: O = YW_O [N x D]

1. QKV Projection
   [N x D] [D x $3HD_H$] => [N x $3HD_H$]
   Split and reshape to get Q, K, V each of shape [H x N x D_H]
2. QK Similarity
   [H x N x D_H] [H x D_H x N] => [H x N x N]
3. V-Weighting
   [H x N x N] [H x N x D_H] => [H x N x D_H]
   Reshape to [N x HD_H]
4. Output Projection
   [N x HD_H] [HD_H x D] => [N x D]

Q: How much memory does this take as the number of vectors N increases?
A: $O(N^2)$

Self-Attention is Four Matrix Multiplies!

If N=100K, H=64 then HxNxN attention weights take 1.192 TB! GPUs don't have that much memory…

Inputs:
Input vectors: X [N x D]
Key matrix: W_K [D x HD_H]
Value matrix: W_V [D x HD_H]
Query matrix: W_Q [D x HD_H]
Output matrix: W_O [HD_H x D]

Flash Attention algorithm computes 2+3 at the same time without storing the full attention matrix!

Makes large N possible

Computation:
Queries: $Q = XW_Q$ [H x N x D_H]
Keys: $K = XW_K$ [H x N x D_H]
Values: $V = XW_V$ [H x N x D_H]
Similarities: $E = QK^T / \sqrt{D_Q}$ [H x N x N]
Attention weights: A = softmax(E, dim=2) [H x N x N]
Head outputs: Y = AV [H x N x D_H] => [N x HD_H]
Outputs: $O = YW_O$ [N x D]

1. QKV Projection
   [N x D] [D x $3HD_H$] => [N x $3HD_H$]
   Split and reshape to get Q, K, V each of shape [H x N x D_H]
2. QK Similarity
   [H x N x D_H] [H x D_H x N] => [H x N x N]
3. V-Weighting
   [H x N x N] [H x N x D_H] => [H x N x D_H]
   Reshape to [N x HD_H]
4. Output Projection
   [N x HD_H] [HD_H x D] => [N x D]

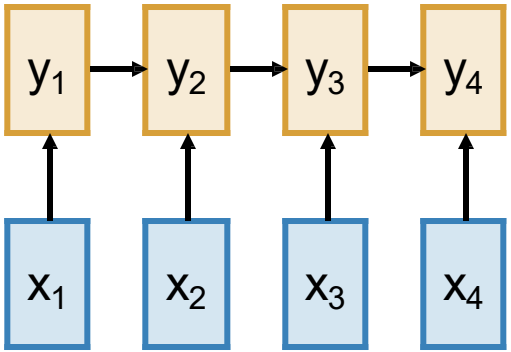
Q: How much memory does this take as the number of vectors N increases?
A: O(N) with Flash Attention

Dao et al, "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness", 2022

Three Ways of Processing Sequences

Three Ways of Processing Sequences

Recurrent Neural Network



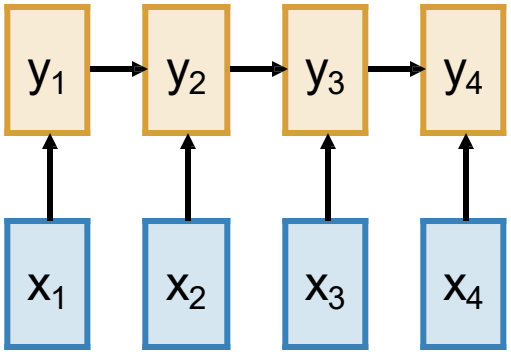
Works on **1D ordered sequences**

(+) Theoretically good at long sequences: O(N) compute and memory for a sequence of length N

(-) Not parallelizable. Need to compute hidden states sequentially

Three Ways of Processing Sequences

Recurrent Neural Network

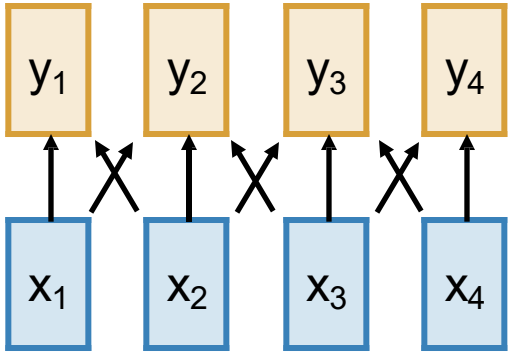


Works on **1D ordered sequences**

(+) Theoretically good at long sequences: O(N) compute and memory for a sequence of length N

(-) Not parallelizable. Need to compute hidden states sequentially

Convolution



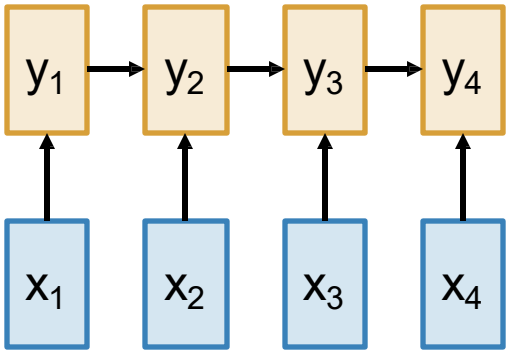
Works on **N-dimensional grids**

(-) Bad for long sequences: need to stack many layers to build up large receptive fields

(+) Parallelizable, outputs can be computed in parallel

Three Ways of Processing Sequences

Recurrent Neural Network

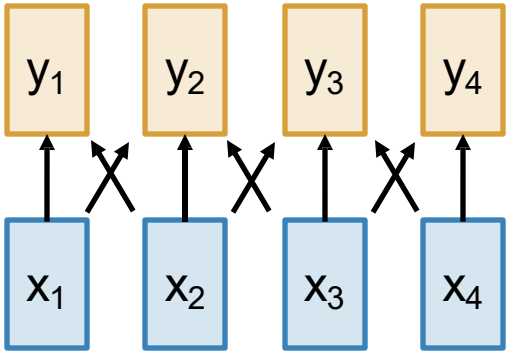


Works on **1D ordered sequences**

(+) Theoretically good at long sequences: O(N) compute and memory for a sequence of length N

(-) Not parallelizable. Need to compute hidden states sequentially

Convolution

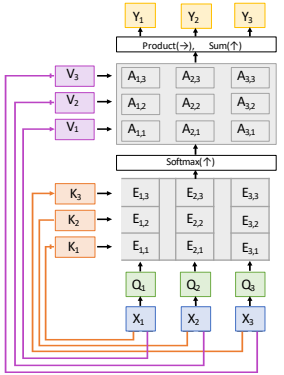


Works on **N-dimensional grids**

(-) Bad for long sequences: need to stack many layers to build up large receptive fields

(+) Parallelizable, outputs can be computed in parallel

Self-Attention



Works on **sets of vectors**

(+) Great for long sequences; each output depends directly on all inputs

(+) Highly parallel, it's just 4 matmuls

(-) Expensive: $O(N^2)$ compute, O(N) memory for sequence of length N

Three Ways of Processing Sequences

Recurrent Neural Network



sequences. O(N) compute and memory for a sequence of length N

(-) Not parallelizable. Need to compute hidden states sequentially

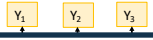
Convolution



stack many layers to build up large receptive fields

(+) Parallelizable, outputs can be computed in parallel

Self-Attention



output depends directly on all inputs

(+) Highly parallel, it's just 4 matmuls

(-) Expensive: $O(N^2)$ compute, $O(N)$ memory for sequence of length N

Attention is All You Need

Vaswani et al, NeurIPS 2017

The Transformer

Transformer Block

Input: Set of vectors x



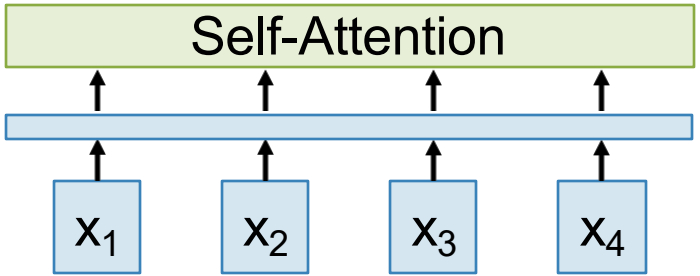
Vaswani et al, “Attention is all you need,” NeurIPS 2017

The Transformer

Transformer Block

Input: Set of vectors x

All vectors interact through (multiheaded) Self-Attention

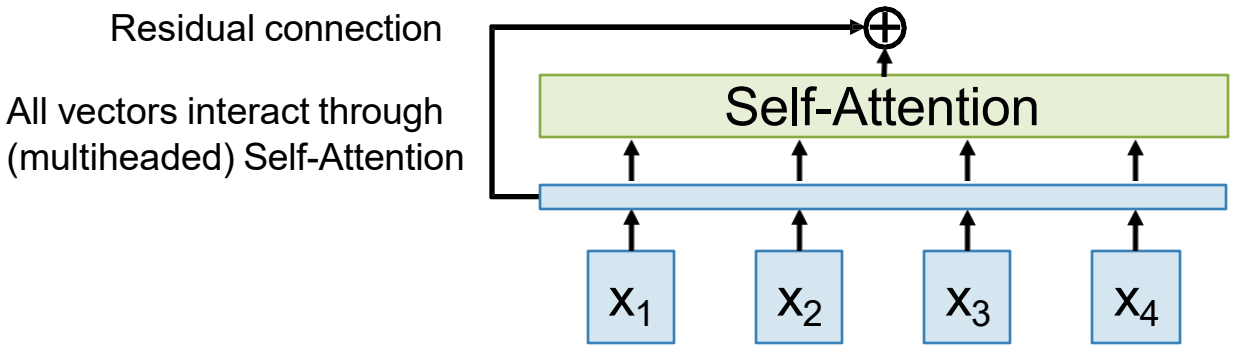


Vaswani et al, “Attention is all you need,” NeurIPS 2017

The Transformer

Transformer Block

Input: Set of vectors x



Vaswani et al, “Attention is all you need,” NeurIPS 2017

The Transformer

Transformer Block

Input: Set of vectors x

Recall **Layer Normalization**:

Given $h_1, \ldots, h_N$ (Shape: D)

scale: γ (Shape: D)

shift: β (Shape: D)

$\mu_i = (\sum_j h_{i,j})/D$ (scalar)

$\sigma_i = (\sum_j (h_{i,j} - \mu_i)^2/D)^{1/2}$ (scalar)

$z_i = (h_i - \mu_i) / \sigma_i$

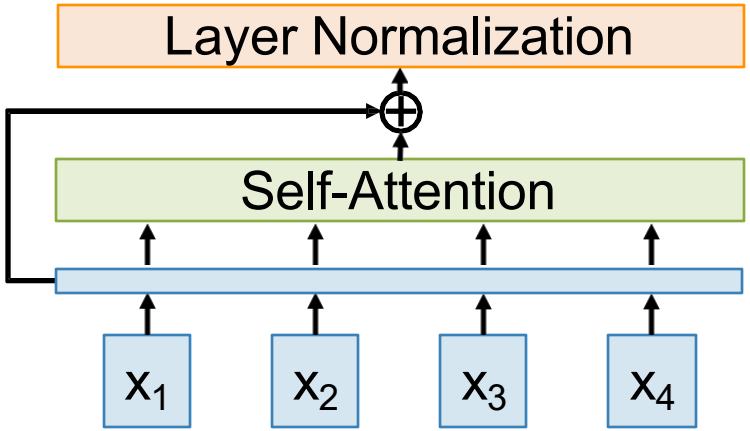
$y_i = \gamma * z_i + \beta$

Ba et al, 2016

Layer normalization normalizes all vectors

Residual connection

All vectors interact through (multiheaded) Self-Attention



Vaswani et al, "Attention is all you need," NeurIPS 2017

The Transformer

Transformer Block

Input: Set of vectors x

Usually a two-layer MLP;
classic setup is
D => 4D => D

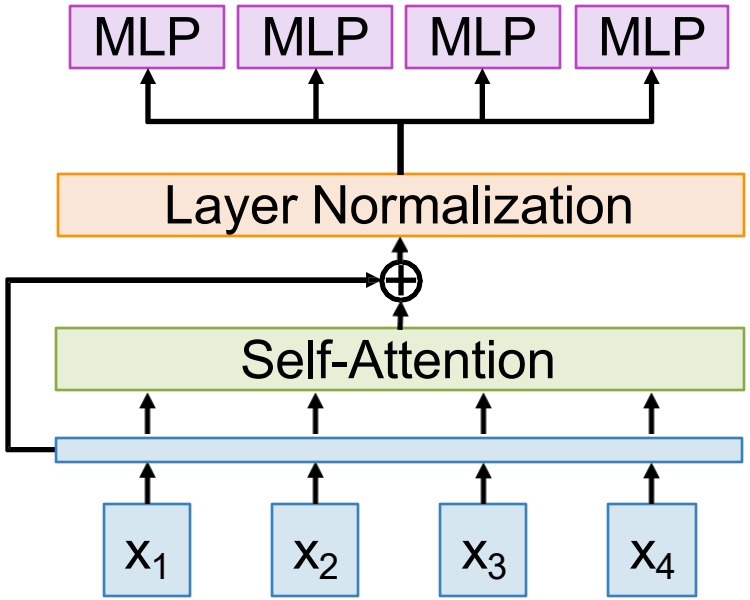
Also sometimes called FFN
(Feed-Forward Network)

MLP independently
on each vector

Layer normalization
normalizes all vectors

Residual connection

All vectors interact through
(multiheaded) Self-Attention

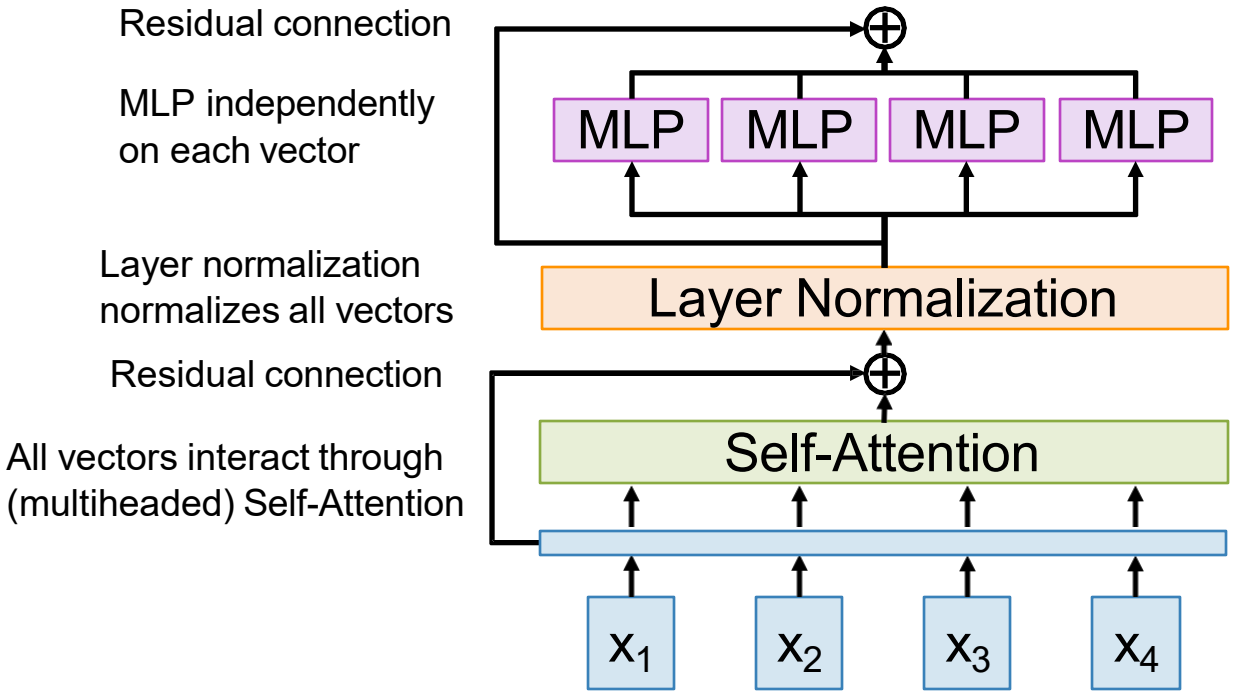


Vaswani et al, “Attention is all you need,” NeurIPS 2017

The Transformer

Transformer Block

Input: Set of vectors x

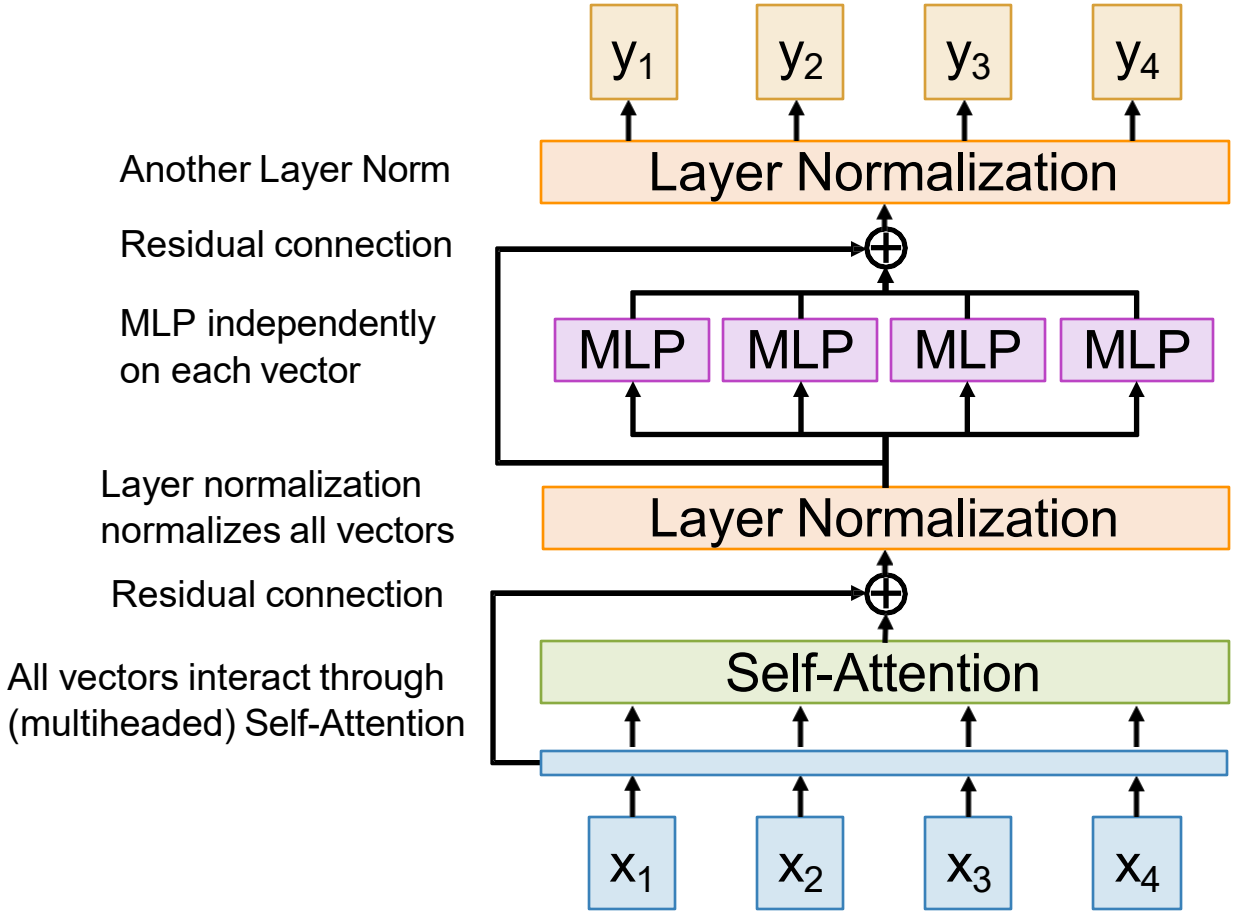


Vaswani et al, “Attention is all you need,” NeurIPS 2017

The Transformer

Transformer Block

Input: Set of vectors x



Vaswani et al, “Attention is all you need,” NeurIPS 2017

The Transformer

Transformer Block

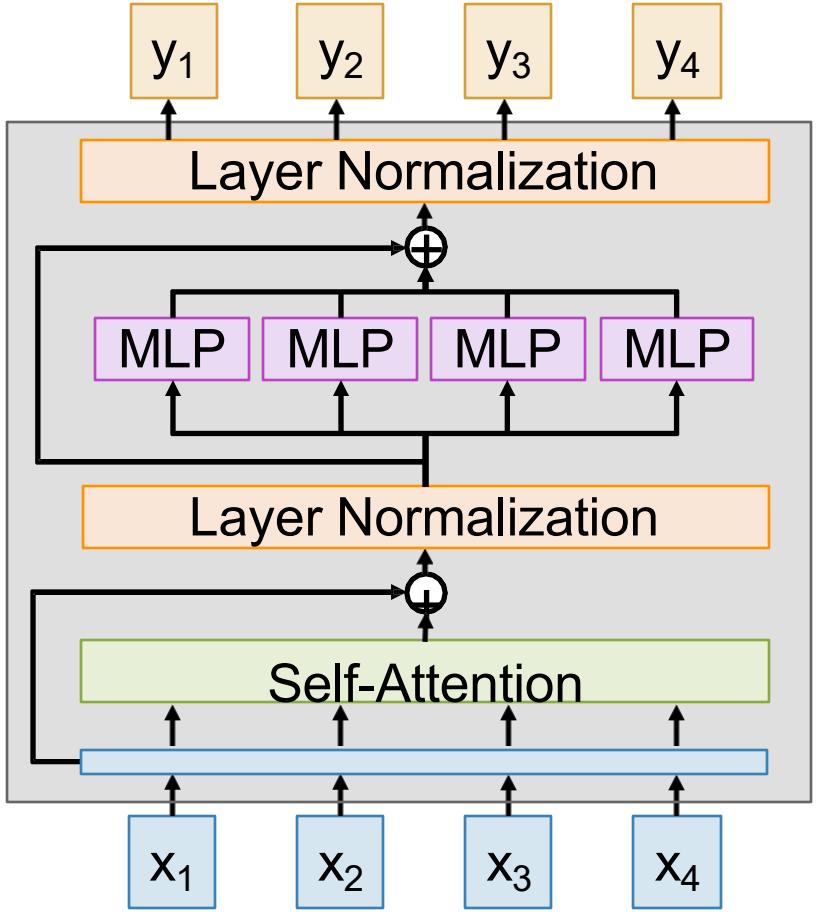
Input: Set of vectors x
Output: Set of vectors y

Self-Attention is the only interaction between vectors

LayerNorm and MLP work on each vector independently

Highly scalable and parallelizable, most of the compute is just 6 matmuls:

4 from Self-Attention
2 from MLP



Vaswani et al, “Attention is all you need,” NeurIPS 2017

The Transformer

Transformer Block

Input: Set of vectors x
Output: Set of vectors y

Self-Attention is the only interaction between vectors

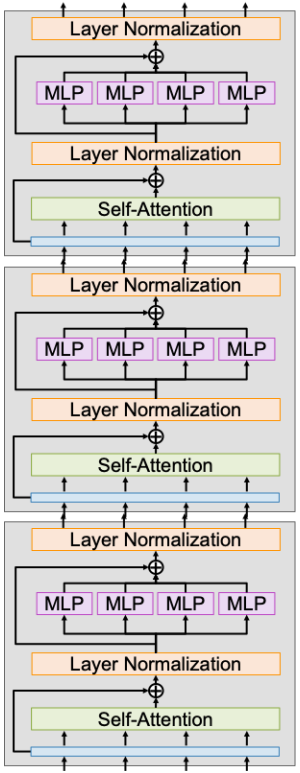
LayerNorm and MLP work on each vector independently

Highly scalable and parallelizable, most of the compute is just 6 matmuls:

4 from Self-Attention
2 from MLP

A **Transformer** is just a stack of identical Transformer blocks!

They have not changed much since 2017… but have gotten a lot bigger



Vaswani et al, “Attention is all you need,” NeurIPS 2017

The Transformer

Transformer Block

Input: Set of vectors x
Output: Set of vectors y

Self-Attention is the only interaction between vectors

LayerNorm and MLP work on each vector independently

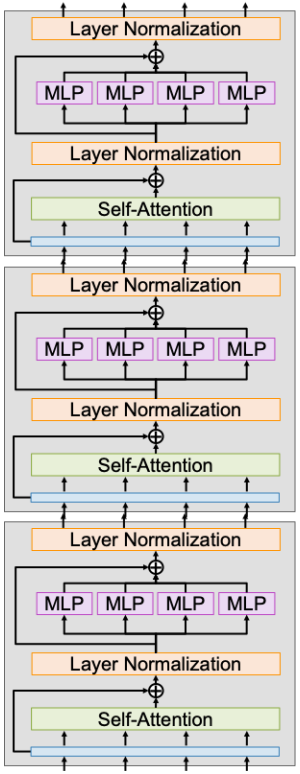
Highly scalable and parallelizable, most of the compute is just 6 matmuls:

4 from Self-Attention
2 from MLP

A **Transformer** is just a stack of identical Transformer blocks!

They have not changed much since 2017… but have gotten a lot bigger

Original: [Vaswani et al, 2017]
12 blocks, D=1024, H=16, N=512
213M params



Vaswani et al, “Attention is all you need,” NeurIPS 2017

The Transformer

Transformer Block

Input: Set of vectors x
Output: Set of vectors y

Self-Attention is the only interaction between vectors

LayerNorm and MLP work on each vector independently

Highly scalable and parallelizable, most of the compute is just 6 matmuls:

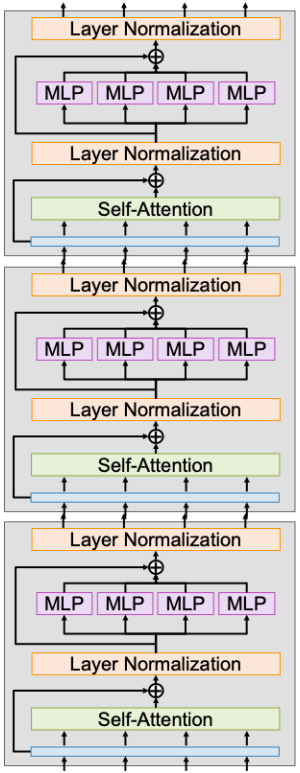
4 from Self-Attention
2 from MLP

A **Transformer** is just a stack of identical Transformer blocks!

They have not changed much since 2017… but have gotten a lot bigger

Original: [Vaswani et al, 2017]
12 blocks, D=1024, H=16, N=512
213M params

GPT-2: [Radford et al, 2019]
48 blocks, D=1600, H=25, N=1024
1.5B params



Vaswani et al, “Attention is all you need,” NeurIPS 2017

The Transformer

Transformer Block

Input: Set of vectors x
Output: Set of vectors y

Self-Attention is the only interaction between vectors

LayerNorm and MLP work on each vector independently

Highly scalable and parallelizable, most of the compute is just 6 matmuls:

4 from Self-Attention
2 from MLP

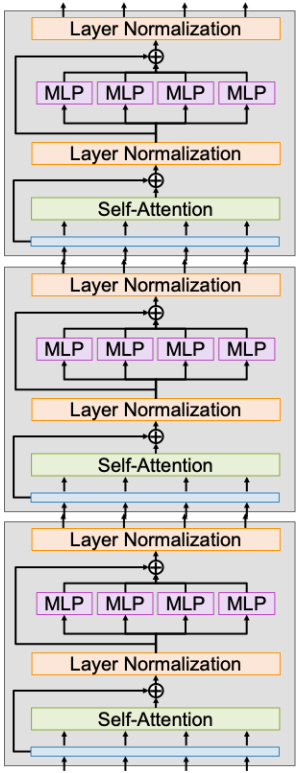
A **Transformer** is just a stack of identical Transformer blocks!

They have not changed much since 2017… but have gotten a lot bigger

Original: [Vaswani et al, 2017]
12 blocks, D=1024, H=16, N=512
213M params

GPT-2: [Radford et al, 2019]
48 blocks, D=1600, H=25, N=1024
1.5B params

GPT-3: [Brown et al, 2020]
96 blocks, D=12288, H=96, N=2048
175B params

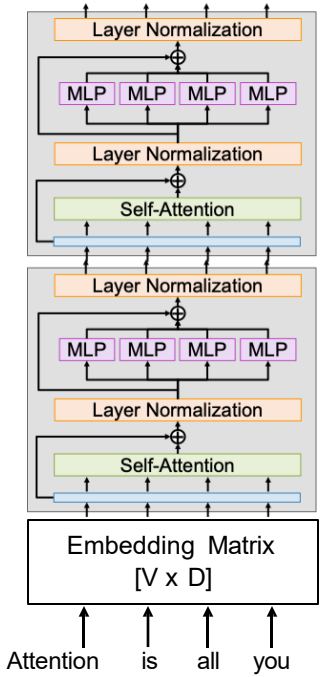


Vaswani et al, “Attention is all you need,” NeurIPS 2017

Transformers for Language Modeling (LLM)

Learn an embedding matrix at the start of the model to convert words into vectors.

Given vocab size V and model dimension D, it's a lookup table of shape [V x D]

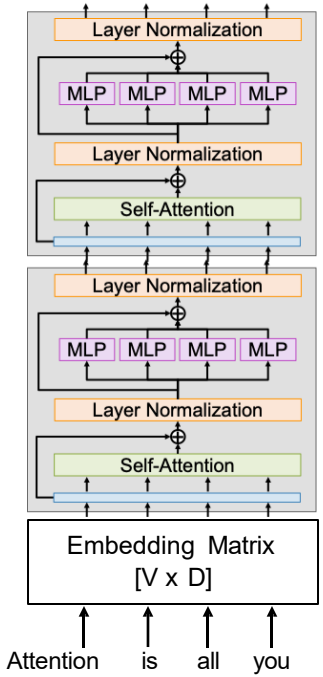


Transformers for Language Modeling (LLM)

Learn an embedding matrix at the start of the model to convert words into vectors.

Given vocab size V and model dimension D, it's a lookup table of shape [V x D]

Use masked attention inside each transformer block so each token can only see the ones before it



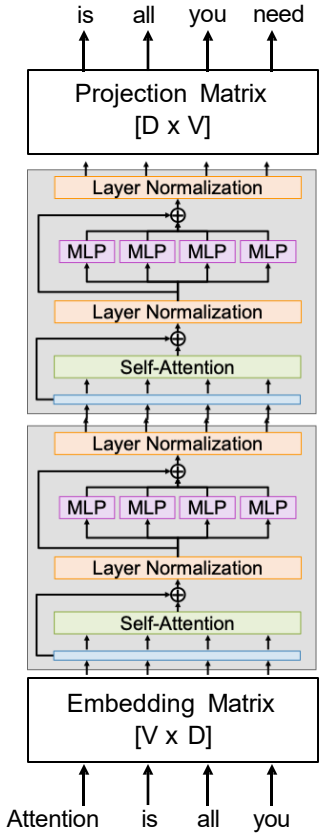
Transformers for Language Modeling (LLM)

Learn an embedding matrix at the start of the model to convert words into vectors.

Given vocab size V and model dimension D, it's a lookup table of shape [V x D]

Use masked attention inside each transformer block so each token can only see the ones before it

At the end, learn a projection matrix of shape [D x V] to project each D-dim vector to a V-dim vector of scores for each element of the vocabulary.



Transformers for Language Modeling (LLM)

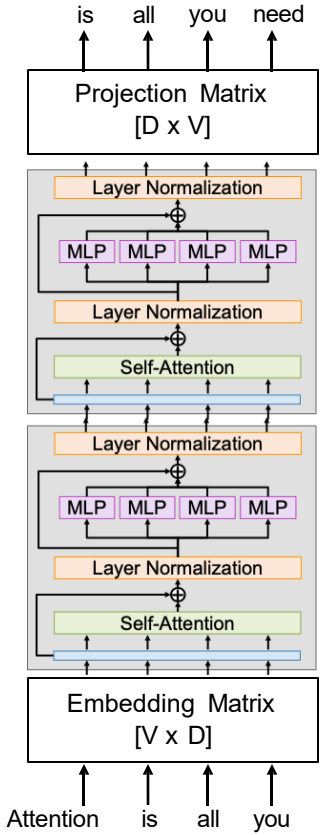
Learn an embedding matrix at the start of the model to convert words into vectors.

Given vocab size V and model dimension D, it's a lookup table of shape [V x D]

Use masked attention inside each transformer block so each token can only see the ones before it

At the end, learn a projection matrix of shape [D x V] to project each D-dim vector to a V-dim vector of scores for each element of the vocabulary.

Train to predict next token using softmax + cross-entropy loss



Vision Transformers (ViT)



Input image:
e.g. 224x224x3

Dosovitskiy et al, “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”, ICLR 2021

Vision Transformers (ViT)



Input image:
e.g. 224x224x3



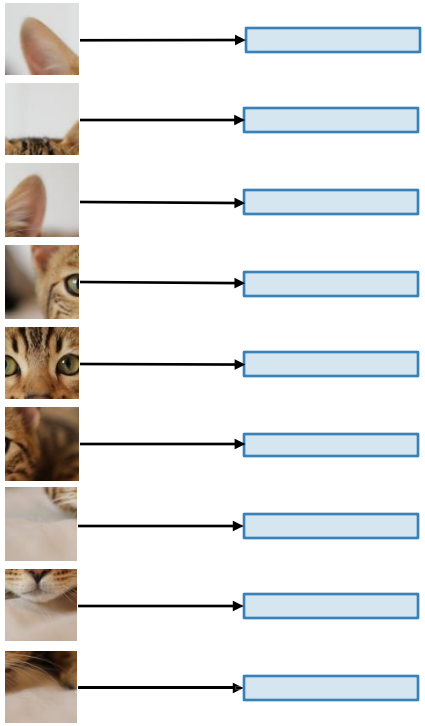
Break into patches
e.g. 16x16x3

Dosovitskiy et al, “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”, ICLR 2021

Vision Transformers (ViT)



Input image:
e.g. 224x224x3



Break into patches
e.g. 16x16x3

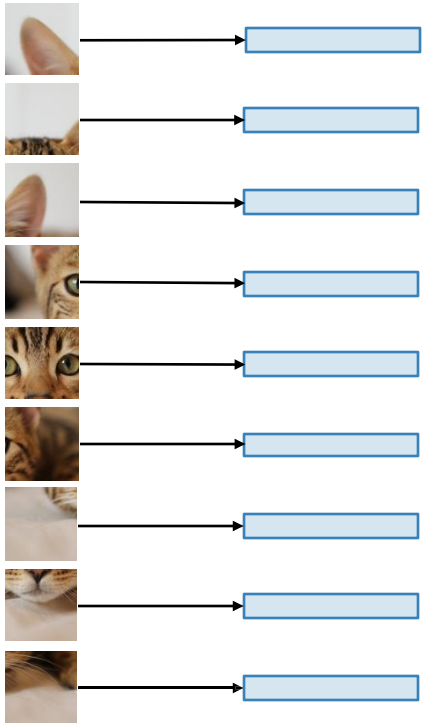
Flatten and apply a linear transform 768 => D

Dosovitskiy et al, “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”, ICLR 2021

Vision Transformers (ViT)



Input image:
e.g. 224x224x3



Break into patches
e.g. 16x16x3

Flatten and apply a linear transform 768 => D

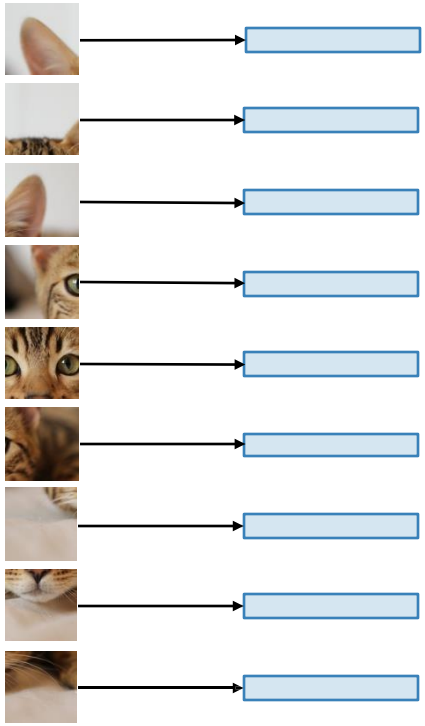
Q: Any other way to describe this operation?

Dosovitskiy et al, “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”, ICLR 2021

Vision Transformers (ViT)



Input image:
e.g. 224x224x3



Break into patches
e.g. 16x16x3

Flatten and apply a linear transform 768 => D

Q: Any other way to describe this operation?

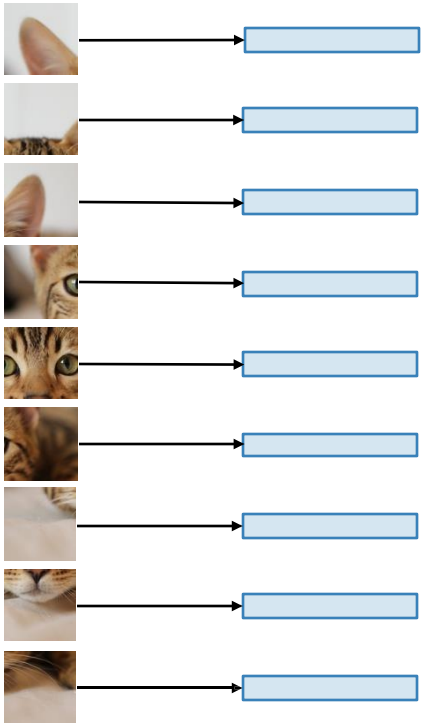
A: 16x16 conv with stride 16, 3 input channels, D output channels

Dosovitskiy et al, “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”, ICLR 2021

Vision Transformers (ViT)

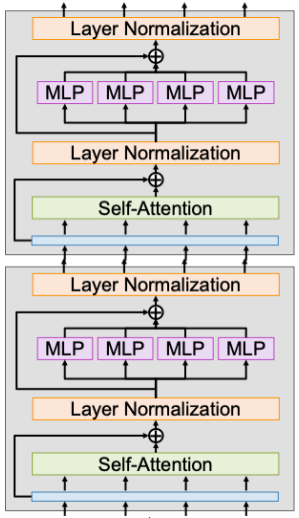


Input image:
e.g. 224x224x3



Break into patches
e.g. 16x16x3

Flatten and apply a linear transform 768 => D



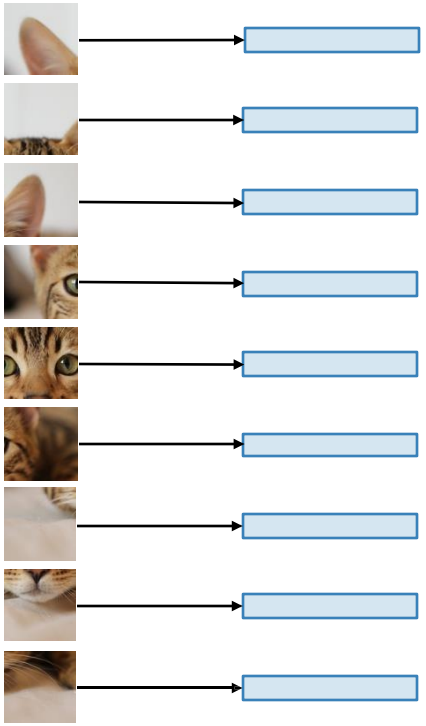
D-dim vector per patch are the input vectors to the Transformer

Dosovitskiy et al, “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”, ICLR 2021

Vision Transformers (ViT)

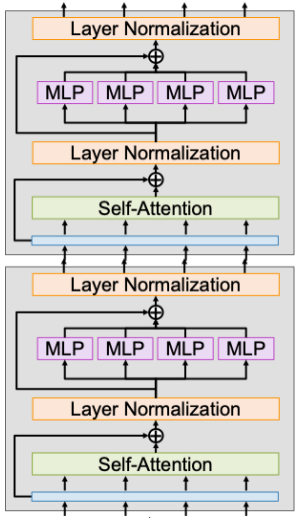


Input image:
e.g. 224x224x3



Break into patches
e.g. 16x16x3

Flatten and apply a linear transform 768 => D



D-dim vector per patch are the input vectors to the Transformer

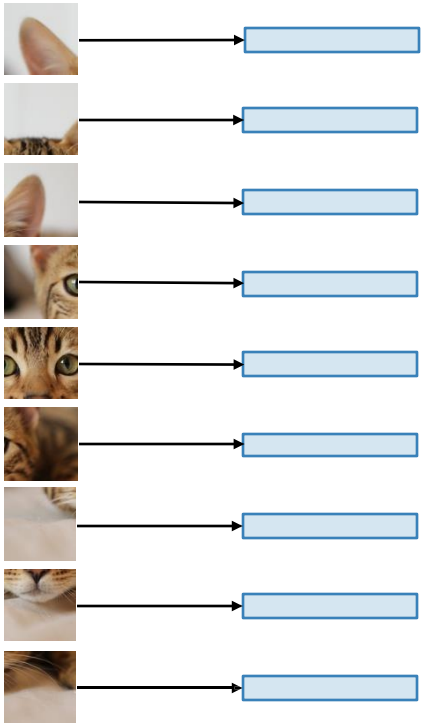
Use positional encoding to tell the transformer the 2D position of each patch

Dosovitskiy et al, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale", ICLR 2021

Vision Transformers (ViT)

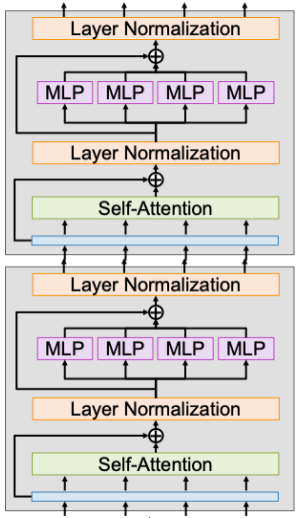


Input image:
e.g. 224x224x3



Break into patches
e.g. 16x16x3

Flatten and apply a linear transform 768 => D



D-dim vector per patch are the input vectors to the Transformer

Don't use any masking; each image patch can look at all other image patches

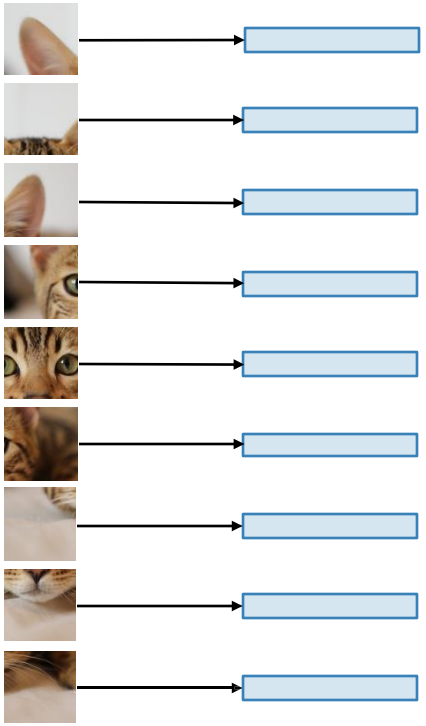
Use positional encoding to tell the transformer the 2D position of each patch

Dosovitskiy et al, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale", ICLR 2021

Vision Transformers (ViT)

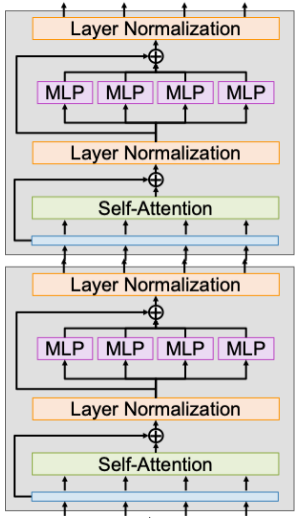


Input image:
e.g. 224x224x3



Break into patches
e.g. 16x16x3

Flatten and apply a linear transform 768 => D



D-dim vector per patch are the input vectors to the Transformer

Transformer gives an output vector per patch

Don't use any masking; each image patch can look at all other image patches

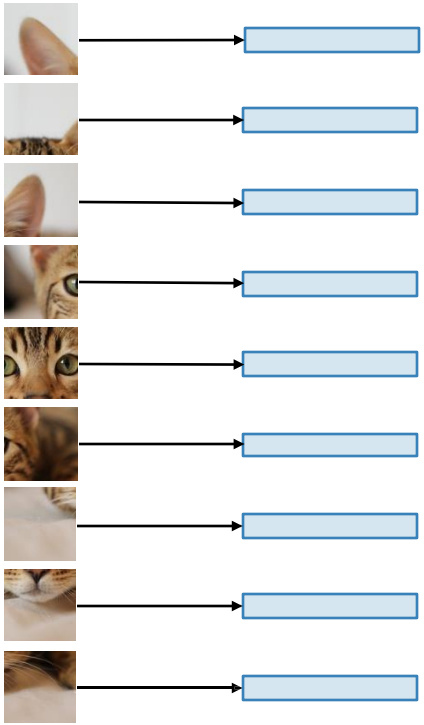
Use positional encoding to tell the transformer the 2D position of each patch

Dosovitskiy et al, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale", ICLR 2021

Vision Transformers (ViT)

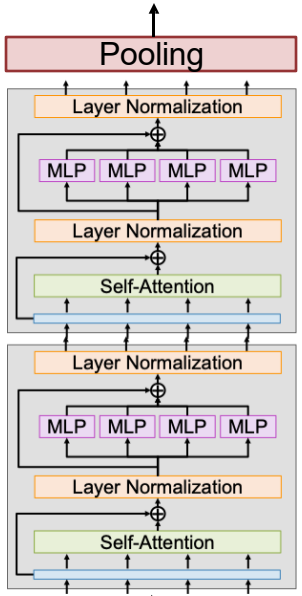


Input image:
e.g. 224x224x3



Break into patches
e.g. 16x16x3

Flatten and apply a linear transform 768 => D



Average pool NxD vectors to 1xD, apply a linear layer D=>C to predict class scores

D-dim vector per patch are the input vectors to the Transformer

Transformer gives an output vector per patch

Don't use any masking; each image patch can look at all other image patches

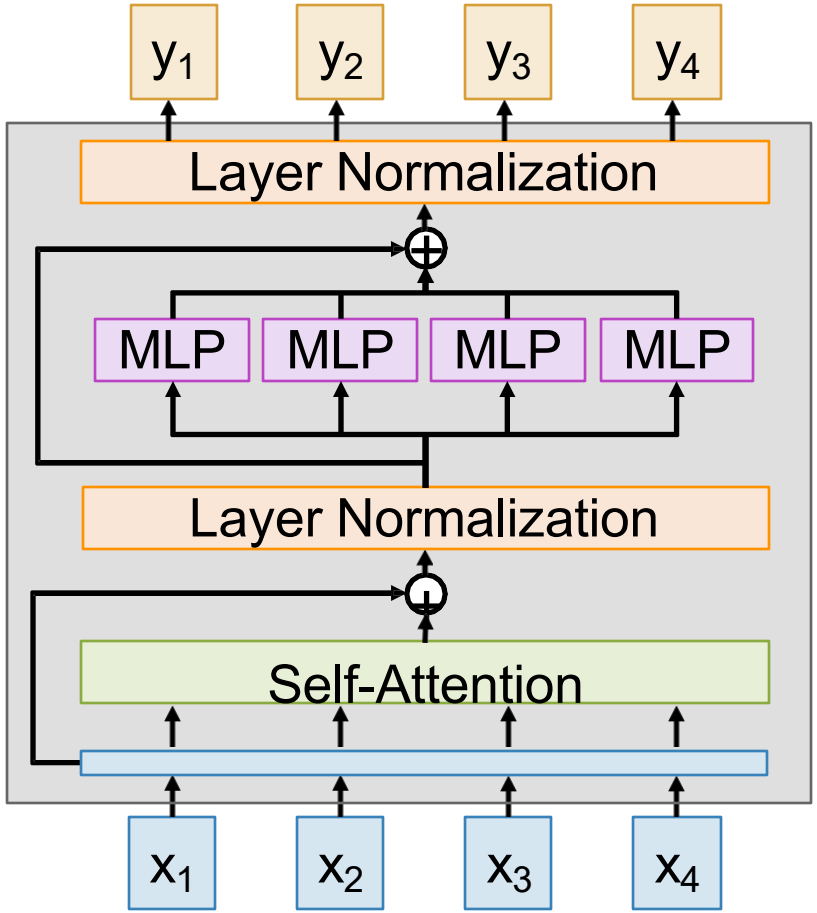
Use positional encoding to tell the transformer the 2D position of each patch

Dosovitskiy et al, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale", ICLR 2021

Tweaking Transformers

The Transformer architecture has not changed much since 2017.

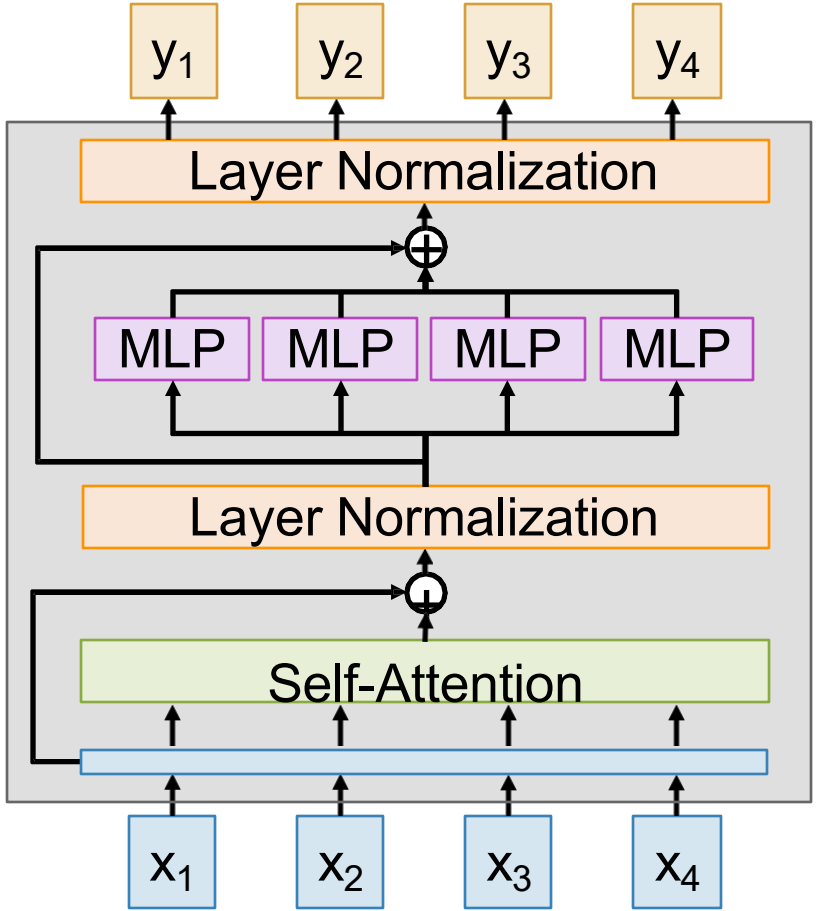
But a few changes have become common:



Pre-Norm Transformer

Layer normalization is outside the residual connections

Kind of weird, the model can't actually learn the identify function



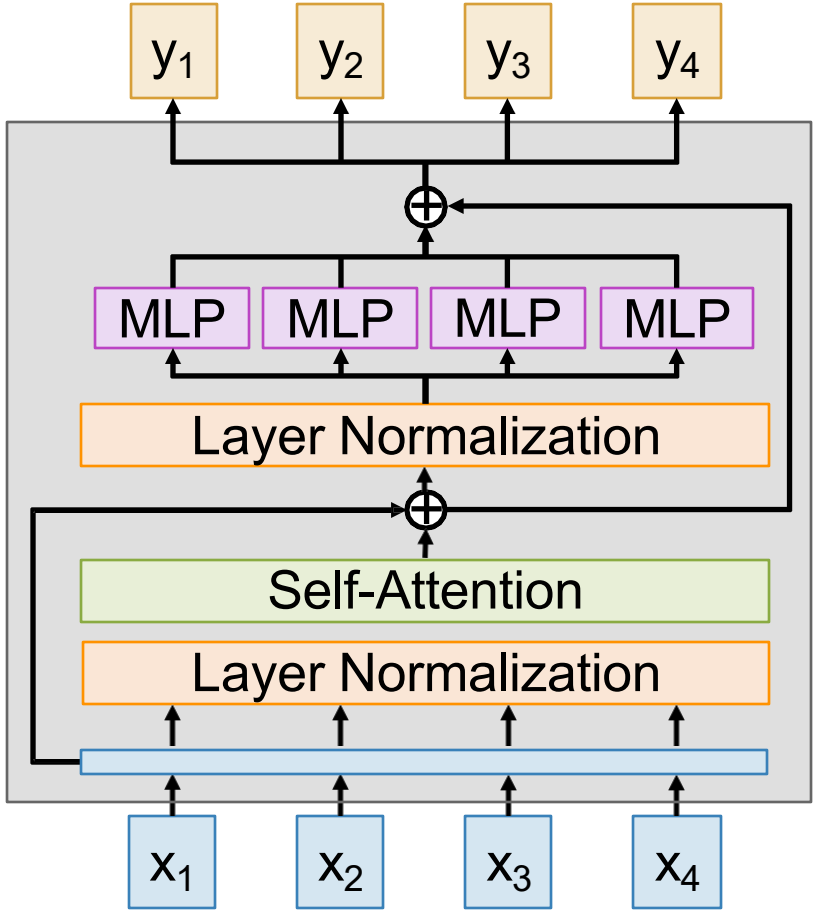
Baevski & Auli, "Adaptive Input Representations for Neural Language Modeling", arXiv 2018

Pre-Norm Transformer

Layer normalization is outside the residual connections

Kind of weird, the model can't actually learn the identify function

Solution: Move layer normalization before the Self-Attention and MLP, inside the residual connections. Training is more stable.



Baevski & Auli, "Adaptive Input Representations for Neural Language Modeling", arXiv 2018

RMSNorm

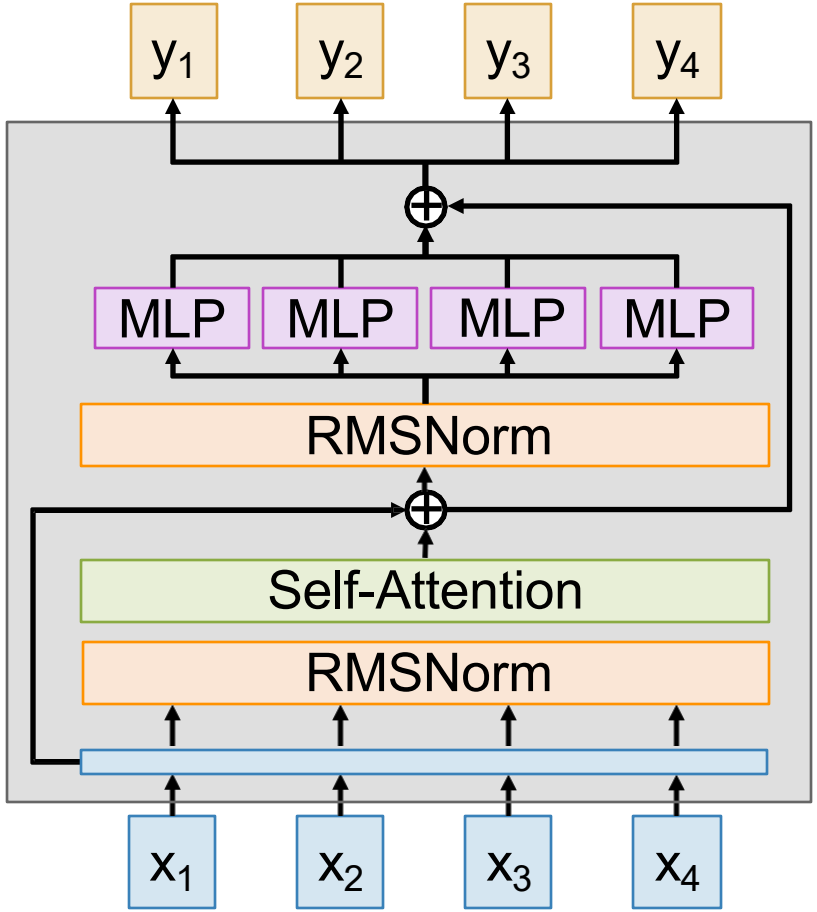
Replace Layer Normalization with Root-Mean-Square Normalization (RMSNorm)

Input: x [shape D]
Output: y [shape D]
Weight: γ [shape D]

$$y_i = \frac{x_i}{RMS(x)} * \gamma_i$$

$$RMS(x) = \sqrt{\varepsilon + \frac{1}{N} \sum_{i=1}^{N} x_i^2}$$

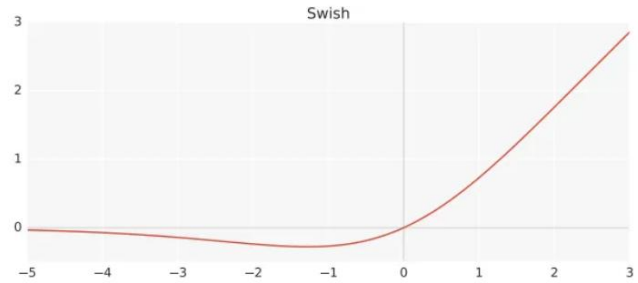
Training is a bit more stable



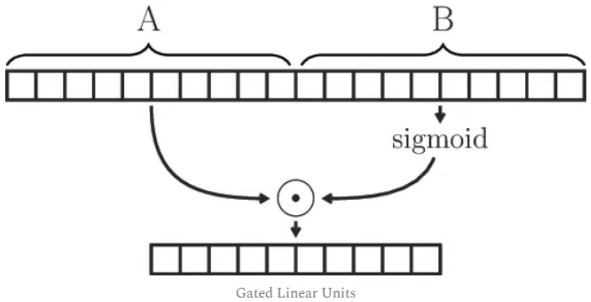
Zhang and Sennrich, “Root Mean Square Layer Normalization”, NeurIPS 2019

Идея SwiGLU

$$\text{Swish}(x) = x*\text{sigmoid}(\beta x)$$



$$\text{GLU}(x) = \text{sigmoid}(W1x+b)\otimes(Vx+c)$$



Gated Linear Units

$$\text{SwiGLU}(x) = \text{Swish}(W1x+b)\otimes(Vx+c)$$

$$\text{FFNSwiGLU}(x) = (\text{Swish1}(xW)\otimes xV)W2$$

```python
def swiglu(x, y):
    """
    Формула: SwiGLU(x, y) = Swish(x) * y
    В PyTorch функция Swish реализована как F.silu
    """
    return F.silu(x) * y

# 2. Полноценный слой SwiGLU (Feed-Forward Network block)
class SwiGLUMLP(nn.Module):
    def __init__(self, in_features, hidden_features):
        super().__init__()
        # В SwiGLU нам нужны две параллельные линейные проекции:
        # одна для вентиля (gate_proj), другая для основных данных (up_proj)
        self.gate_proj = nn.Linear(in_features, hidden_features, bias=False)
        self.up_proj = nn.Linear(in_features, hidden_features, bias=False)

        # Финальная проекция возвращает размерность к исходной
        self.down_proj = nn.Linear(hidden_features, in_features, bias=False)

    def forward(self, x):
        # Применяем SiLU (Swish) к одной ветке и умножаем на вторую
        activated = F.silu(self.gate_proj(x)) * self.up_proj(x)

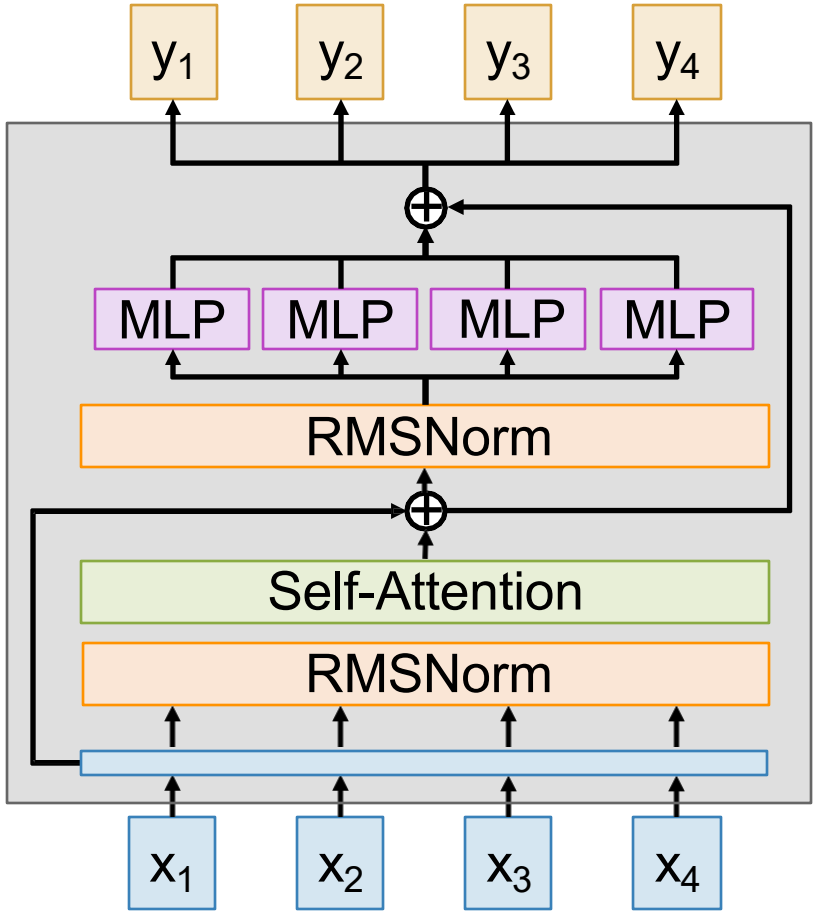
        # Пропускаем результат через выходной линейный слой
        return self.down_proj(activated)
```

Shazeer, “GLU Variants Improve Transformers”, 2020

SwiGLU MLP

Classic MLP:

Input: X [N x D]
Weights: W_1 [D x 4D]
W_2 [4D x D]
Output: $Y = \sigma(XW_1)W_2$ [N x D]



Shazeer, “GLU Variants Improve Transformers”, 2020

SwiGLU MLP

Classic MLP:

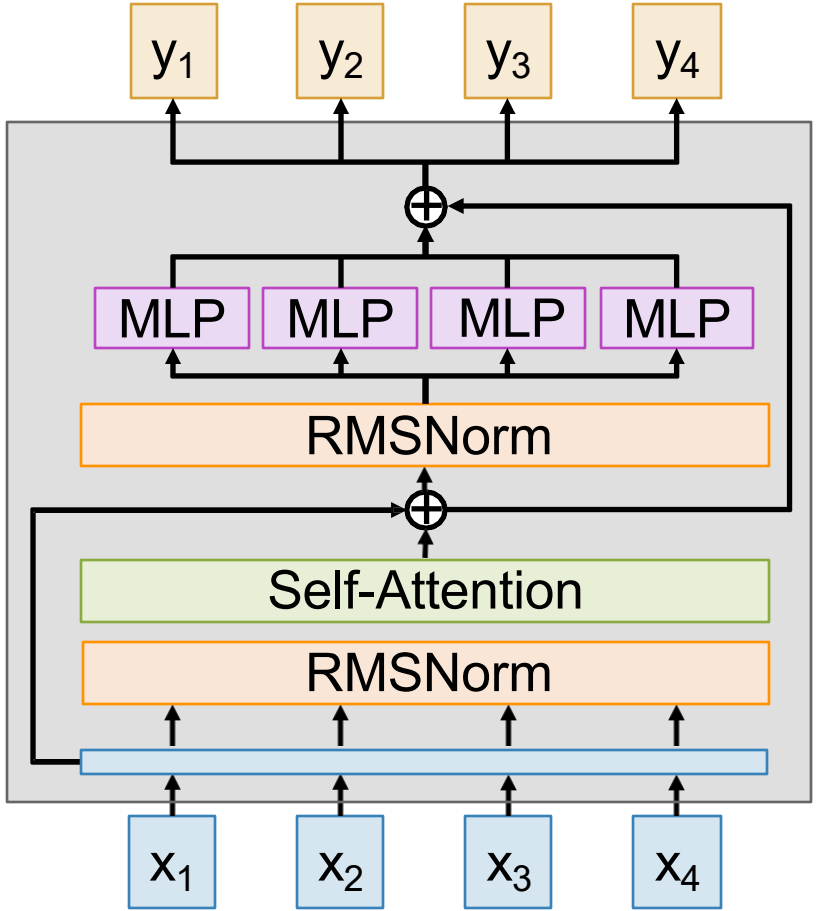
Input: X [N x D]
Weights: W_1 [D x 4D]
W_2 [4D x D]
Output: $Y = \sigma(XW_1)W_2$ [N x D]

SwiGLU MLP:

Input: X [N x D]
Weights: W_1 , W_2 [D x H]
W_3 [H x D]
Output:

$$Y = (\sigma(XW_1) \odot XW_2)W_3$$

Setting H = 8D/3 keeps same total params



Shazeer, “GLU Variants Improve Transformers”, 2020

SwiGLU MLP

Classic MLP:

Input: X [N x D]
Weights: W_1 [D x 4D]
W_2 [4D x D]
Output: $Y = \sigma(XW_1)W_2$ [N x D]

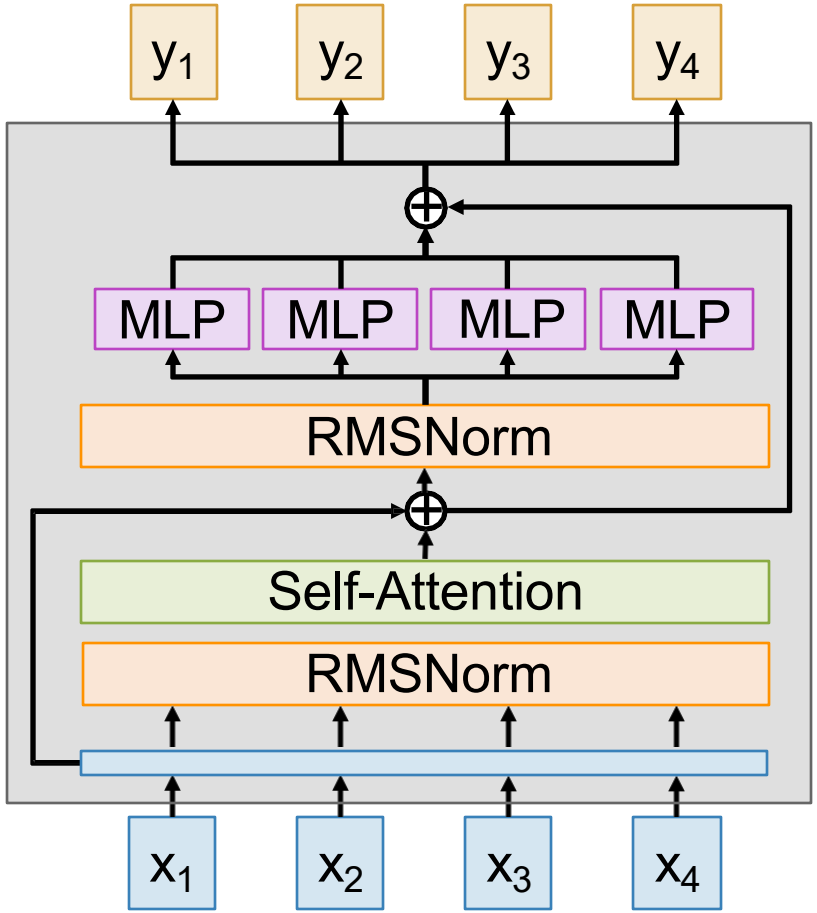
SwiGLU MLP:

Input: X [N x D]
Weights: W_1 , W_2 [D x H]
W_3 [H x D]
Output:

$$Y = (\sigma(XW_1) \odot XW_2)W_3$$

Setting H = 8D/3 keeps same total params

We offer no explanation as to why these architectures seem to work; we attribute their success, as all else, to divine benevolence.

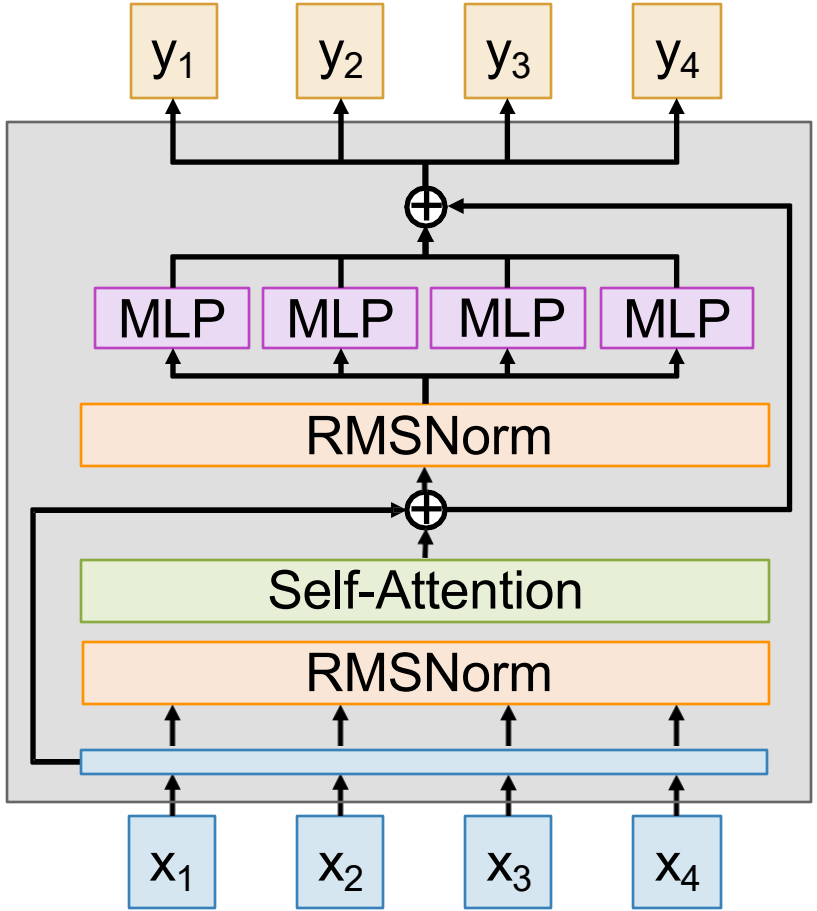


Shazeer, "GLU Variants Improve Transformers", 2020

Mixture of Experts (MoE)

Learn E separate sets of MLP weights in each block; each MLP is an *expert*

W_1: [D x 4D] => [E x D x 4D]
W_2: [4D x D] => [E x 4D x D]



Shazeer et al, “Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer”, 2017

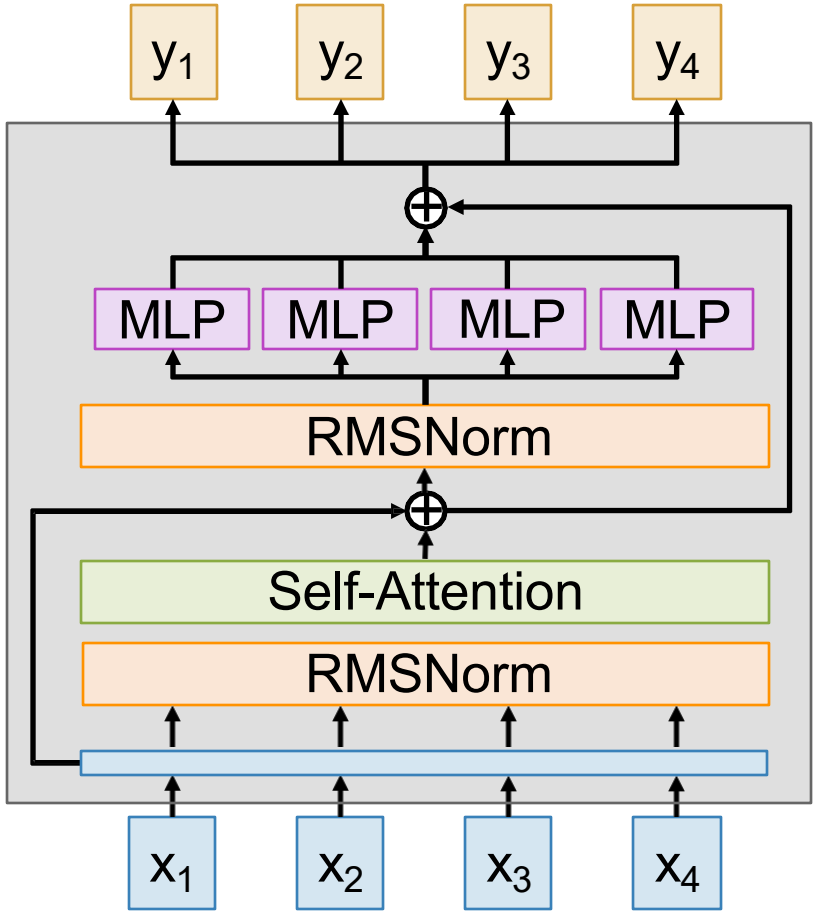
Mixture of Experts (MoE)

Learn E separate sets of MLP weights in each block; each MLP is an *expert*

W_1: [D x 4D] => [E x D x 4D]
W_2: [4D x D] => [E x 4D x D]

Each token gets *routed* to A < E of the experts. These are the *active experts*.

Increases params by E,
But only increases compute by A



Shazeer et al, “Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer”, 2017

Mixture of Experts (MoE)

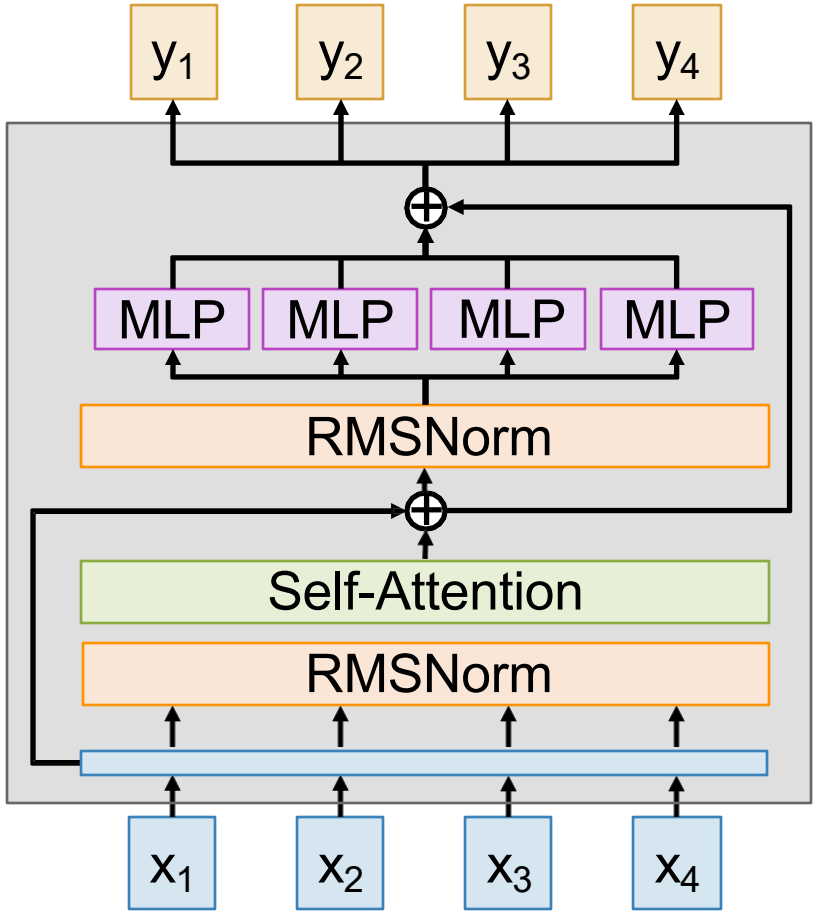
Learn E separate sets of MLP weights in each block; each MLP is an *expert*

W_1: [D x 4D] => [E x D x 4D]
W_2: [4D x D] => [E x 4D x D]

Each token gets *routed* to A < E of the experts. These are the *active experts*.

Increases params by E,
But only increases compute by A

All of the biggest LLMs today (e.g. GPT4o, GPT4.5, Claude 3.7, Gemini 2.5 Pro, etc) almost certainly use MoE and have > 1T params; but they don't publish details anymore

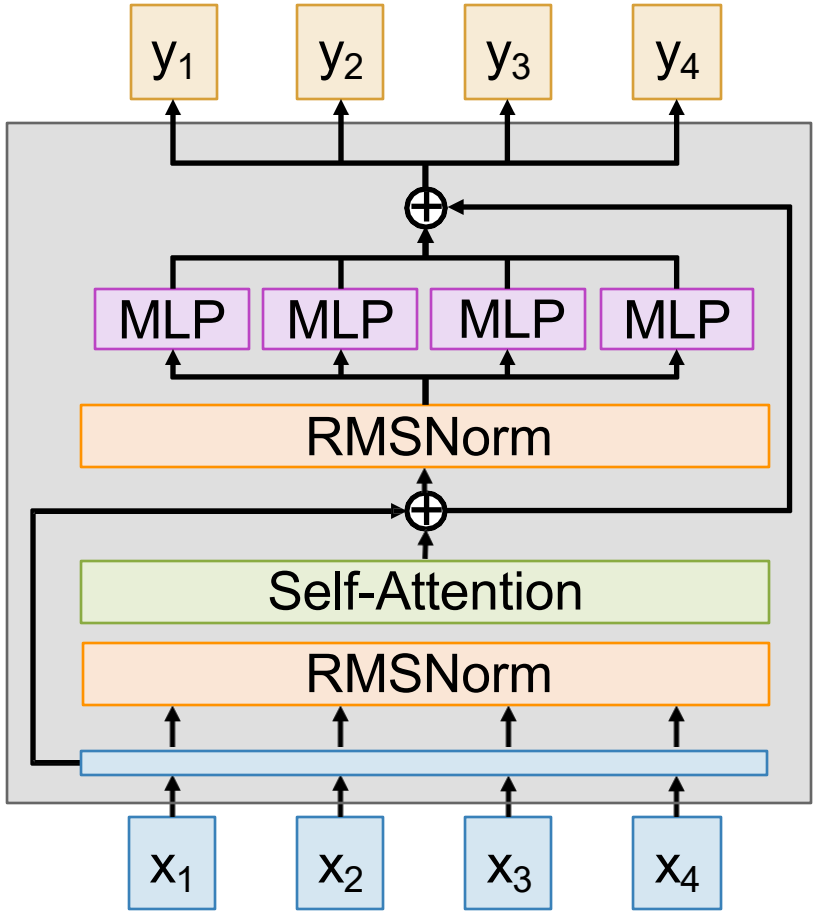


Tweaking Transformers

The Transformer architecture has not changed much since 2017.

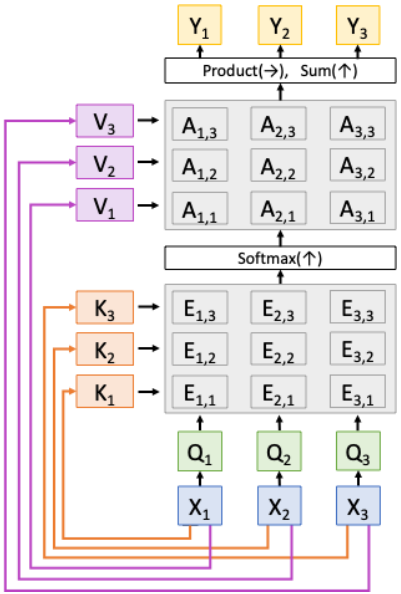
But a few changes have become common:

- **Pre-Norm:** Move normalization inside residual
- **RMSNorm**: Different normalization layer
- **SwiGLU**: Different MLP architecture
- **Mixture of Experts (MoE)**: Learn E different MLPs, use A < E of them per token. Massively increase params, modest increase to compute cost.



Summary: Attention + Transformers

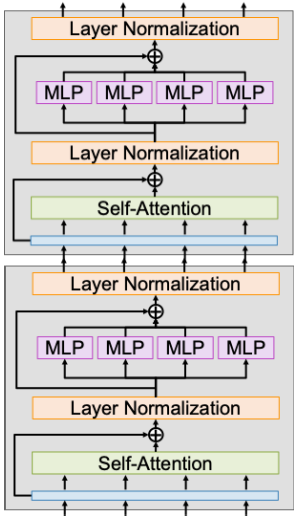
Attention: A new primitive that operates on sets of vectors



Transformers are the backbone of all large AI models today!

Used for language, vision, speech, …

Transformer: A neural network architecture that uses attention everywhere



Архитектура GPT

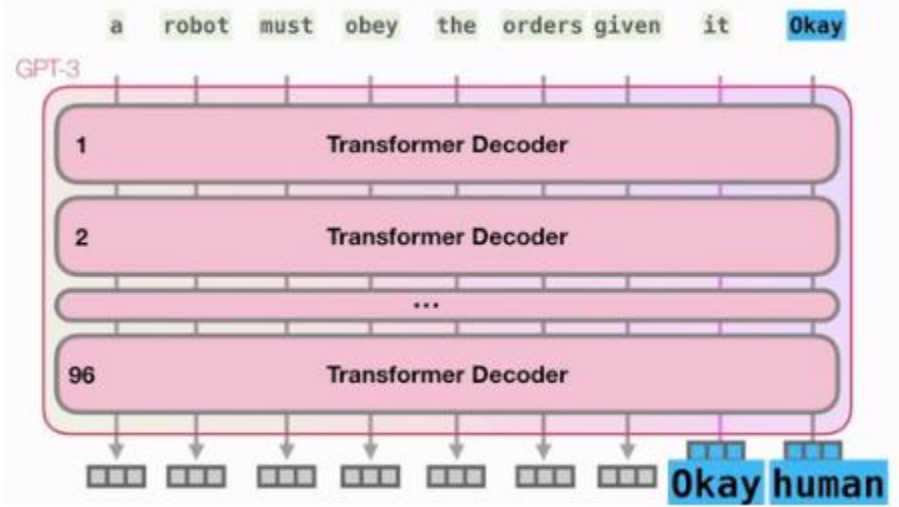
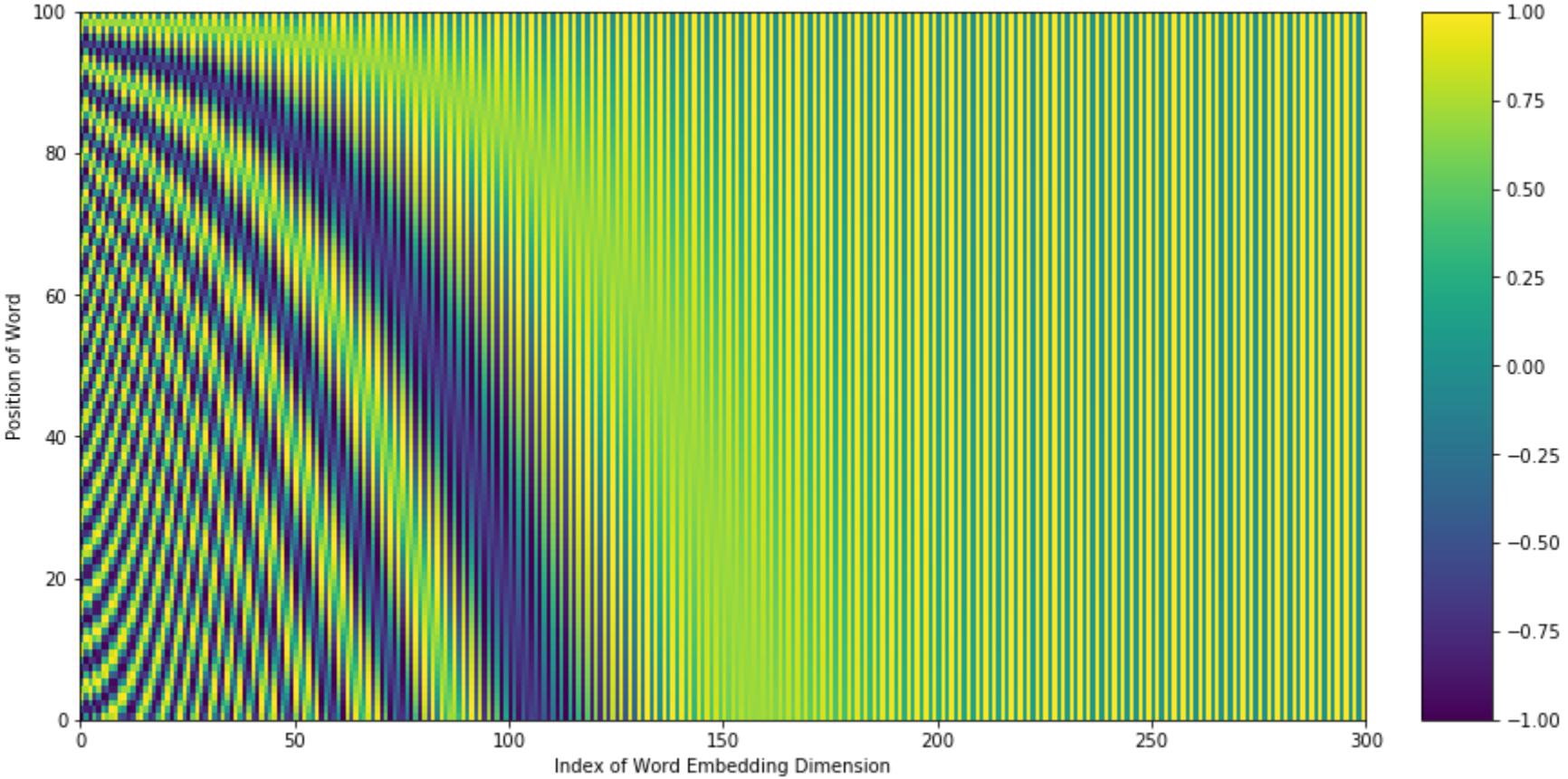


Схема работы GPT (продолжение текста)

Последовательность шагов GPT

1. Токенизация
2. Получение эмбедингов (похоже на word2vec)
3. Добавляем эмбединг позиции
4. Пропускаем через последовательность Transformer Decoding Blocks
5. Умножаем выход на вход? берем softmax – получаем вероятности следующего токена
6. Делаем семплинг из распределения

Позиционные эмбеденги



Позиционное кодирование

```
0:  0 0 0 0     8:  1 0 0 0
1:  0 0 0 1     9:  1 0 0 1
2:  0 0 1 0     10: 1 0 1 0
3:  0 0 1 1     11: 1 0 1 1
4:  0 1 0 0     12: 1 1 0 0
5:  0 1 0 1     13: 1 1 0 1
6:  0 1 1 0     14: 1 1 1 0
7:  0 1 1 1     15: 1 1 1 1
```

Бинарное представление наивного позиционного кодирования

$$PE_{(pos,i)} = \begin{cases} \sin\left(\frac{pos}{10000^{i/d_{model}}}\right) & \text{if } i \text{ is even} \\ \cos\left(\frac{pos}{10000^{(i-1)/d_{model}}}\right) & \text{if } i \text{ is odd} \end{cases}$$

Синусоидальное позиционное кодирование

A. Vaswani et al., Attention Is All You Need, 2017

```python
import numpy as np
import matplotlib.pyplot as plt

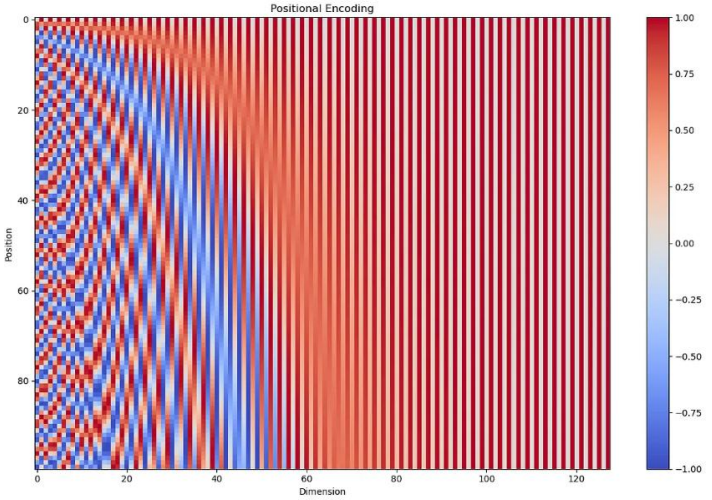
def sinusoidal_positional_encoding(max_position, d_model):
    position = np.arange(max_position)[:, np.newaxis]
    # The original formula pos / 10000^(2i/d_model) is equivalent to pos * (1 /
    # I use the below version for numerical stability
    div_term = np.exp(np.arange(0, d_model, 2) * -(np.log(10000.0) / d_model))

    pe = np.zeros((max_position, d_model))
    pe[:, 0::2] = np.sin(position * div_term)
    pe[:, 1::2] = np.cos(position * div_term)

    return pe

max_position = 100  # Maximum sequence length
d_model = 128       # Embedding dimension

pe = sinusoidal_positional_encoding(max_position, d_model)
```



RoPE – Ротационное кодирование

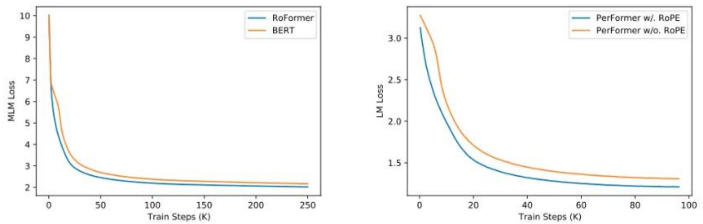
Классический подход сумма токена и вектора позиции.
RoPE(Rotational Positional Encoding) поворот токена **в многомерном пространстве на угол, пропорциональный его позиции**.

$$PE_{(pos,i)} = \begin{cases} \sin\left(\frac{pos}{10000^{i/d_{model}}}\right) & \text{if } i \text{ is even} \\ \cos\left(\frac{pos}{10000^{(i-1)/d_{model}}}\right) & \text{if } i \text{ is odd} \end{cases}$$

Скалярное произведение между Query ((m)-я позиция) и Key ((n)-я позиция) после применения матриц поворота естественным образом начинает зависеть только от относительного расстояния (m - n). Абсолютная позиция «сокращается» при перемножении.

Model	BLEU
Transformer-baseVaswani et al. [2017]	27.3
RoFormer	**27.5**

The RoFormer gives better BLEU scores compared to its baseline alternative Vaswani et al. [2017] on the WMT 2014 English-to-German translation task




```python
import numpy as np
import matplotlib.pyplot as plt

def get_rotary_matrix(context_len: int, d_model: int) -> np.ndarray:
    """
    Generate the Rotary Matrix for ROPE
    Args:
        context_len (int): context len
        d_model (int): embedding dim
    Returns:
        np.ndarray: the rotary matrix of dimension context_len x d_model x d_model
    """
    R = np.zeros((context_len, d_model, d_model))
    positions = np.arange(1, context_len + 1)[:, np.newaxis]
    # Create matrix theta (shape: context_len x d_model // 2)
    slice_i = np.arange(0, d_model // 2)
    theta = 10000. ** (-2.0 * slice_i.astype(float) / d_model)
    m_theta = positions * theta
    # Create sin and cos values
    cos_values = np.cos(m_theta)
    sin_values = np.sin(m_theta)
    # Populate the rotary matrix R using advanced indexing
    R[:, 2*slice_i, 2*slice_i] = cos_values
    R[:, 2*slice_i, 2*slice_i+1] = -sin_values
    R[:, 2*slice_i+1, 2*slice_i] = sin_values
    R[:, 2*slice_i+1, 2*slice_i+1] = cos_values
    return R
```

Su J. et al., RoFormer: Enhanced Transformer with Rotary Position Embedding, Neurocomputing, 2023

Оптимизация трансформера

MLA (Multi-Head Latent Attention) (DeepSeek v2, v3)

Рост контекста и размера батча забивает память видеокарты **KV-кэшем** (сохраненными ключами и значениями прошлых токенов). MQA (Multi-Query) и GQA (Grouped-Query) уменьшают кэш за счет Low Rank Approximation.

KV-кэш сжимается на **93%**

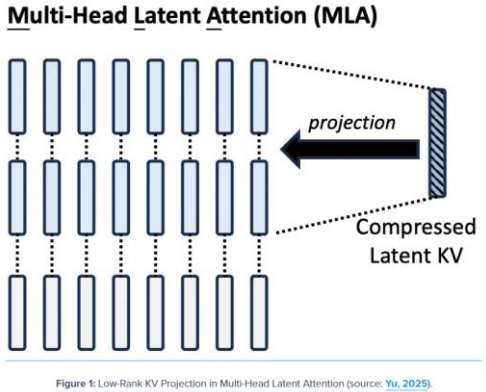


Figure 1: Low-Rank KV Projection in Multi-Head Latent Attention (source: Yu, 2025).

Спекулятивное декодирование

Генерация текста в авторегрессионных моделях происходит строго по одному токену. Чтобы выдать одно слово, огромная LLM должна прогнать через свои веса весь контекст. Инференс упирается в скорость чтения весов из памяти

Мы берем маленькую, «дешевую» и быструю модель-черновик (Draft Model) и просим её сгенерировать вперед, например, 5–7 токенов. Затем большая, умная модель (Target Model) проверяет эти токены за один единственный проход (параллельно).



Representation Engineering (инженерия представлений)

Activation Steering

$$h^{(l)}_{\text{modified}} = h^{(l)} + c \cdot v_{\text{steering}}$$

$h^{(l)}$ активации слоя,
v_{steering} вектор стиринга

Activation Addition

$$v_{\text{steering}} = h^{(l)}_{+} - h^{(l)}_{-}$$

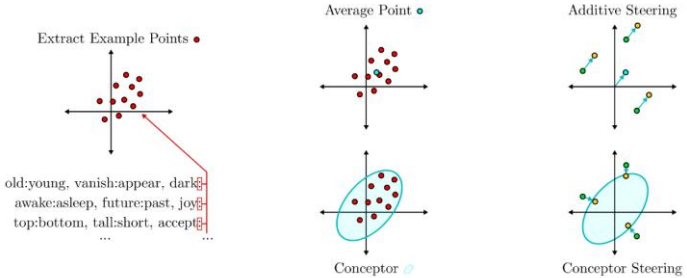


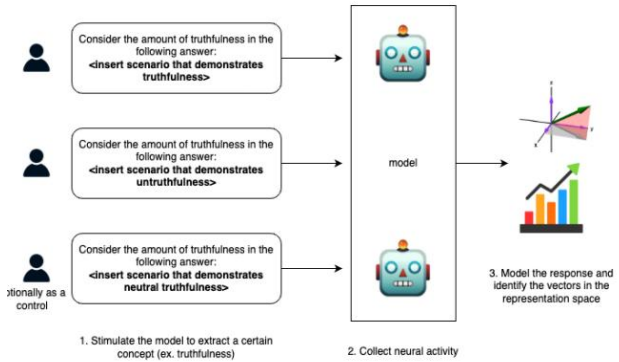
Illustration showing the basic geometric difference between additive and conceptor steering using a set of activations for the antonym task. Additive steering acts as a translation of the activation vectors by a fixed steering vector. Conceptor steering acts as a (soft) projection onto a target ellipsoid.

Условный AS (Conditional Activation Steering**)**

Вводим **вектор условия (Condition Vector)**. Во время инференса скрытое состояние сначала проецируется на этот вектор. Система оценивает: «Относится ли контекст к опасной теме?». Если да — вектор стиринга применяется, если нет — модель работает в штатном режиме.

Conceptor Steering – элипсоиды человеческих абстракций

Можно ли использовать в ризонинге?



Linear Artificial Tomography

L Bartoszcze et al., Representation Engineering for Large-Language Models: Survey and Research Challenges, 2025 https://arxiv.org/pdf/2502.17601